

SmartStride

*The Future of Mobility: Hands-Free, Affordable, and
Intelligent*

Midterm Report

EEL4914 – Senior Design I

Group 11



University of Central Florida
College of Engineering and Computer Science

Matthew Itskovich, CpE
Arturo Lara, CpE
Adam Lilly, EE
Evan Rees, CpE

Reviewers: Sonali Das, Daniel Hedlund, Qun Zhou
Sun, and Piotr Kulik

Mentor: Chung Yong Chan

Table of Contents

Table of Contents.....	2
List of Figures.....	4
Chapter 1 Executive Summary.....	1
Chapter 2 Project Description.....	2
2.1 Project Background and Motivation.....	2
2.2 Existing and Prior Related Projects.....	3
2.3 Objective and Goals.....	4
2.4 Requirement Specifications.....	6
2.5 House of Quality.....	7
2.6 Project Diagrams.....	8
Chapter 3 Research and Investigation.....	11
3.1 Sensor Technologies.....	12
3.1.1 Xbox Kinect V2.....	12
3.1.2 Lidar Sensor.....	13
3.1.3 UltraSonic Sensor.....	15
3.1.4 Comparison of Sensors.....	15
3.2 Power Technologies.....	16
3.2.2 Power Supply - Batteries.....	20
3.2.3 Battery Level Indicator.....	25
3.3 Processing Unit Research.....	30
3.3.1 Raspberry Pi.....	30
3.3.2 Nvidia Jetson.....	33
3.3.3 x86 Computers: A Familiar Foundation with Significant Drawbacks.....	35
3.3.4 Processing Unit Conclusion.....	38
3.4 Wireless Processing Unit Research.....	39
3.4.1 Communication technologies.....	40
3.4.2 ESP32.....	41
3.4.3 Raspberry Pico W.....	42
3.4.4 Arduino Nano RP2040 Connect.....	42
3.5 IMU Chip Research.....	44
3.5.1 MPU-6050.....	44
3.5.2 GY-9250 MPU9250.....	44
3.5.3 BNO055 Absolute Orientation Sensor.....	45
3.6 LED Screen Research.....	45
3.6.1 2.0" 320x240 Color IPS TFT Display - ST7789 EYESPI.....	45
3.6.2 3.5" TFT 320x480 - HX8357D.....	46

3.6.3 2.8" 320x240 ESP32-2432S028R.....	47
3.6.4 Display Results.....	47
3.7 Button Research.....	48
3.7.1 Adafruit 560.....	48
3.7.2 APIELE Stainless Steel Push Button Switch.....	49
3.7.3 E-Switch PR144C1900.....	49
3.7.4 Button Results.....	50
3.8 3D Printed Material Research.....	51
3.8.1 Solid Thermoplastic Filament.....	51
3.8.2 Liquid Photopolymer Resin.....	51
3.8.3 Printer Results.....	52
3.9 Software Research.....	52
3.9.1 Kinect Libraries.....	52
3.9.2 LCD Programming Research.....	53
3.9.3 ESP32-Display Program Research.....	54
3.9.4 Button Reaction Program Research.....	55
3.9.5 Buzzer Program Research.....	56
3.9.6 Collaborative Code Platforms.....	56
3.9.6.1 Git.....	57
3.9.6.2 GitHub.....	57
3.9.6.3 GitLab.....	57
3.9.6.4 BitBucket.....	58
3.9.7 Programming Languages.....	58
3.9.7.1 Python.....	58
3.9.7.2 C/C++.....	58
3.9.7.3 Coding Overview.....	58
3.9.8 Repeatable Software Environments.....	59
3.9.8.1 Docker.....	59
3.9.8.2 Podman.....	59
3.10 Digital-to-Analog Converter Research.....	60
3.10.1 Analog Devices AD5664.....	60
3.10.2 Texas Instruments DAC7614.....	61
3.10.3 Analog Devices AD5696.....	61
3.10.4 Texas Instruments PCM4104.....	61
3.10.5 Analog Devices ADAU1978.....	61
3.10.6 Texas Instruments PCM1690.....	62
3.10.7 DAC Research Comparison.....	62
3.11 AI Vision Models.....	63
3.11.1 Kinect Skeletal Data.....	63
3.11.2 RGB Object Detection.....	64

3.11.3 Semantic Segmentation.....	66
Chapter 4 Standards and Design Constraints.....	68
4.1 Industry Standard Constraints.....	68
4.1.1 Universal Serial Bus (USB) 3.0 Standard.....	68
4.1.2 Bluetooth Standard.....	70
4.1.3 LCD Screen Standard.....	71
4.1.4 Button Standards.....	72
4.2 Practical Constraints.....	75
4.2.1 Time.....	75
4.2.2 Economic.....	76
4.2.3 Environmental.....	77
4.2.4 Ethical.....	77
4.2.5 Adaptation of Current Wheelchair.....	78
4.3 ADA Constraints/Compliance.....	78
4.3.1 ADA Physical Dimension Constraints.....	78
4.3.2 ADA Visual Constraints.....	80
4.4 Personal & Mission Constraints.....	81
Chapter 5 ChatGPT and other Similar Platforms.....	83
Chapter 6 Hardware Design.....	89
6.1 Power Supply Systems.....	89
6.1.1 3.3V Regulator.....	89
6.1.2 5 V regulator.....	90
6.1.3 Battery Level Indicator.....	91
6.2 Interfacing with WheelChair PCB.....	93
6.3 Wireless Controller Schematic.....	93
6.4 Pi DAC & Power Delivery.....	95
Chapter 7 Software Design.....	97
7.1 Emergency Stop Button.....	97
7.2 Selfdriving/Cruise Control Algorithm.....	98
7.3 Joystick Workflow.....	100
Chapter 8 System Fabrication/Prototype Construction.....	101
8.1 PCB Fabrication.....	101
8.2 Controller Casing Prototyping.....	101
8.3 Sensor Casing/Mounting Fabrication.....	102
8.4 Pi to DAC & Power Delivery PCB.....	102
8.5 Wireless ESP32 Controller PCB.....	104
Chapter 9 System Testing and Evaluation.....	105
9.1 Hardware Testing.....	105
9.1.1 LCD Screen Testing.....	105

9.1.2 Button Testing.....	106
9.1.3 Buzzer Testing.....	106
9.1.4 Battery Testing.....	106
9.1.5 Motor Testing.....	107
9.2 Software Testing.....	110
9.2.1 LCD Screen Testing.....	111
9.2.2 Emergency Stop Testing.....	111
9.2.3 Freenect2.....	111
9.2.4 ESP32 BLE Server.....	113
Chapter 10 Administrative Content.....	114
10.1 Project Milestones.....	114
10.2 Budget Estimates.....	116
10.3 Division of Labor.....	117
Chapter 11 Conclusion.....	119
Works Cited.....	121
Appendix.....	124
List of Images.....	129

List of Figures

Chapter 1 Executive Summary

Chapter 2 Project Description

2.5.1 House of Quality.....	7
2.6.1 Hardware Diagram.....	8
2.6.2 Software Diagram.....	10

Chapter 3 Research and Investigation

3.1 Sensor Technologies	
3.1.1 Kinect V2.....	12
3.1.2 Lidar FHL-LD20.....	14
3.2 Power Technologies	
3.2.3.1 Battery Voltage Regulator.....	25
3.2.3.2 LM324 Battery Indicator.....	28
3.6 LED Screen Research	
3.6.1.1 ST7789 EYESPI.....	45
3.6.2.1 HX8357D.....	47
3.6.3.1 2432S028R.....	48
3.7 Button Research	
3.7.1.1 Adafruit 560.....	49
3.7.2.1 APIELE Push Button.....	50

3.7.3 E-Switch.....	50
3.9 Software Research	
3.9.2.1 Flow of Information.....	55
3.9.4.1 Button Program Example.....	56
3.9.5.1 Buzzer Program Example.....	57
Chapter 4 Standards and Design Constraints	
4.1 Industry Standard Constraints	
4.1.1.1 xkcd Standard Comic.....	69
4.3 ADA Constraints/Compliance	
4.3.1.1 ADA Floor Space Diagram.....	80
4.3.1.2 ADA Clearance Chart.....	80
4.3.1.3 ADA Knee Clearance Chart.....	81
Chapter 6 Hardware Design	
6.1 Power Supply Systems	
6.1.1.1 3.3V Regulator.....	91
6.1.2.1 5 V regulator.....	92
6.1.3.1 Battery Level Indicator Schematic.....	93
6.2 Interfacing with WheelChair PCB	
6.2.1 Original Controller PCB.....	94
6.3.1 Wireless Controller Schematic.....	95
6.4.1 Pi to DAC & Supply Schematic.....	96
Chapter 7 Software Design	
7.1.1 Emergency Stop Visual.....	98
7.1.2 Button Programming Diagram.....	99
7.2.1 Self Driving/Cruise Control Algorithm Flowchart.....	100
Chapter 8 System Fabrication/Prototype Construction	
8.3 Sensor Casing/Mounting Fabrication.....	102
8.4 Pi to DAC &	

Chapter 1 Executive Summary

While cures for diseases and medical technology continue to advance, there are still many individuals out there that suffer from irreparable damage that has made their daily livelihood far more difficult than the average person. Some people become weaker and others will unfortunately lose abilities that most people take for granted such as being able to walk without the assistance of any medical apparatus. As much as we wish we could, we can not provide a cure to debilitating effects like paralysis, but with dedication and ingenuity we can make the tools most people have at their disposal to make their lives just a little bit easier. This project, SmartStride, is a means of converting both old and new motorized wheelchairs into a new and more accessible means of transportation for the mobility impaired. With the addition of modern day computers and sensors we can provide a cheap and efficient means of automating the movement process for those in need of an easier to use wheelchair.

The SmartStride wheelchair project has two main focuses, the first is to design a way to revamp old wheelchairs or to adapt newer, more modern, wheelchairs to be compatible with the systems we are using to collect and process data. This includes adding an entirely new function to the chair, that being an automatic movement setting. This automatic mode will allow users to set the directional inputs of their desired destination, so that those that have trouble controlling the chairs motor functions can have a more accessible option of navigating their environment. This of course will require the addition of multiple computers to be installed so that we can ensure the chair has the processing capability of taking in directional input while being able to accurately move the chair with the motors. The refitting of the chairs controllers will also need to occur as a new layout and visual informational outlet will need to be installed to ensure that the user can manage the system and receive any information regarding the current status of SmartStride's functionality.

Along with updating the hardware and improving the functionality of the wheelchair SmartStride will also be incorporating sensors to the chair that will be used in tandem with the automatic movement mode. This is to ensure that the safety of the passenger is not compromised by the chair's inability to perceive any dangers around it, or to identify if the route the wheelchair is trying to take is blocked or not. One of the sensors that will be used in our system will be that of Microsoft's Xbox Kinect camera, a surprisingly advanced piece of hardware that incorporates multiple types of sensors besides just a visual input. With this piece of technology we can easily scan the user's surroundings and find distant objects that may or may not prove to be a threat to the automatic systems of the chair.

This document contains all the information collected for the project along with all the work needed to have SmartStride fully operational. It lists a collection of components and information used in the process of designing the wheelchair along with the overall cost of building it.

Chapter 2 Project Description

This chapter provides a detailed blueprint for the SmartStride project, an initiative focused on augmenting an existing electric wheelchair with affordable, autonomous capabilities. We begin by exploring the project's background and motivation, establishing the need for a cost-effective, intelligent mobility solution. An analysis of prior related projects will then be presented to contextualize our design choices. Subsequently, the chapter outlines our specific objectives and goals, followed by detailed requirement specifications for the system's core components. To conclude, we will present a House of Quality analysis and high-level hardware and software diagrams that illustrate the proposed system architecture and operational logic.

2.1 Project Background and Motivation

Ever since the earliest humans started walking earth, there has been an issue at play for the physically disabled to get around in a smooth manner. This issue has posed as an attempt for engineers since the beginning of the occupational concept to come up with a solution for the handicapped. One of the best-known solutions engineers have come up with to date is the wheelchair. The first ever case of the wheelchair seems to come in as early as in 12th century Europe, but the first ever documented case happened in the 17th century in Germany. The German maker of this wheelchair, Farffler, initially created these three-wheeled contraptions for the wealthy [1]. Over time, the design of the wheelchair has gotten a lot more optimal; they have gotten very expensive as well.

A more specific optimal design we want to focus more on here is the electric wheelchair. The earliest known case of the design was in 1953. George Klein and his fellow companions wanted to make a motorized wheelchair mainly to help injured veterans on their return home from WWII [2]. Ever since 1953, the electric wheelchair has continued to change drastically in both its features and in its popularity. It grew in popularity so much that even the World Health Organization declared access to the motorized wheelchair a basic human right. Even though the electric wheelchair is now declared a basic human right and people can get them for cheap or even free through government assistance programs and charities, there is still a big problem at hand for people that don't have access to them because of the costs.

While the issue of affordability is present in electric wheelchairs, there is always room for improvement in the design. A very useful improvement would be to have a smart autonomous electric wheelchair that helps the user to avoid obstacles without even having to control it. This idea is what our group intends to work on. The name we came up with for our project is the SmartStride. The SmartStride would be useful in the sense that it will empower users with severe disabilities as well as for it to enhance safety and navigation.

The motivation behind wanting to create an automated electric wheelchair stems mainly from our want to help people with disabilities. There are a lot of similar and well documented projects out there but it seems like the optimality of the designs could always be improved. Another big reason we would like to create a functioning smart wheelchair is due to the already high costs. An electric wheelchair without any smart features on the lower tier level starts around \$1,500 and only goes up from there. Our group aims to design a smart wheelchair that's affordable enough to be accessible to people in lower-income communities—something we believe would be deeply meaningful and fulfilling!

2.2 Existing and Prior Related Projects

The good thing about our project idea is that there have been a lot of attempts at creating an electric wheelchair. There have also been so many attempts at it that Vocic.com even created an article walking people through the basics of how to convert a manual wheelchair to an electric wheelchair. In the article, they lay out different ways of implementing this idea. The different ways are installing a front-powered wheel, wheelchair conversion kit, joystick control system, power assist system, full electric conversion, and DIY conversion. Also in the article, they lay out costs and different safety measures (legal) that should be taken into consideration when designing this project [3]. We will be starting with an already designed electric wheelchair so we won't need to change out the hardware too much.

We came across different project ideas online. One electric wheelchair project that captured our attention was created by a team of engineers at UC Davis. Their project is called the DIY Electric Wheelchair, and they put a big emphasis on creating a cost-effective wheelchair, which is what our team is trying to do. This wheelchair is not automated but it is a good reference for hardware. On their official website, they have a bill of materials that shows all of what they bought. Some important things we noted that we may try to add to our design are a 12 V battery and 12V DC brushed motor. The BOM for this project came out to a little over \$2,000. While \$2,000 is particularly on the cheap end of the electric wheelchair spectrum, our goal is to keep the chair under well under \$2,000. On the team's project website, they highlight their needs of three different regions of the wheelchair: The power chain needs, body needs, and overall needs. The power chain highlights the overall motor's abilities, ability to climb road curbs, and good acceleration. The body needs show the teams want to have a durable wheelchair that can meet the ADA specifications. Overall needs are very similar to the other two types of needs. This section of the website is very useful for us to organize what we are looking for in our design [4].

Another project we found that is like what we are trying to do comes from a team at Rutgers university. They created a conversion kit that attaches to a manual wheelchair frame to turn it into an electric one. Even though this wheelchair isn't an automated one, it is a good reference for what parts we are

going to use for our project. It consists of two motors and a control system that is user-friendly. The motors are directly attached from the bottom frame to each of the wheels. One thing that stood out to us about this project is that the team was able to keep their budget within \$750, which is where we'd like our total cost to be when we start demoing our prototype. This team's project report does a really good job of laying out the control system diagram showing how the remote flows to the Arduino, then to the batteries, and finally to the motors to control the wheelchair. The team also lays out torque calculations for different surface types and then designs the wheelchair around that. They also do horsepower calculations. The torque needed for a standard concrete wheelchair ramp calculated is 2.5815 ft-lbs. Other important calculations they laid out are horsepower needed for an appropriate range of torque and RPM. The desired speed of a wheelchair according to their project came out to about 2 feet/second which equals about 754 rotations per minute. These types of calculations in their report gave our team an idea of how to properly gauge what type of motors and the voltage batteries we are going to need to be able to power our wheelchair given different variables (slope, weight of load, terrain type) we need to consider.

When trying to do research on autonomous wheelchair designs, it seems like a lot of companies have severely invested in these types of projects and have come out successful. One smart wheelchair design that stands out is the Mobilis Autopilot, located out of Poland. This company takes existing wheelchairs and helps people to convert their wheelchairs into a smart AI assisted chair that you can even control and modify all by just using your smartphone [5]. The autopilot feature on the wheelchair is an AI assisted feature that uses a camera on the wheelchair to automap things nearby. For example, if someone ventures into the living room with their chair, the chair will self learn how to map out objects in the living room and avoid them for the future. The user can use their smartphone to set up and name desired destinations as well as. When thinking about how we want to focus on our own automated electric wheelchair, the Mobilis is a good reference point for the basis of our project research.

2.3 Objective and Goals

This project's basic idea is to augment an existing electric wheelchair and give it auto driving capabilities. Since we are modifying a broken chair (dead battery) we also plan to restore it to working condition. This can be useful for automating movement and making automated safety features, particularly for users with vision impairments. We also hope we can make this system affordable, so that other users can do this cheaply themselves. As well as give it intuitive user features that can be easy to control and preferable than bypassing the system.

Overall Goals

- A modular design that can be easily maintained

- Cost effective
- Maintain feature set of base wheelchair

Basic Goals

- Be able to drive forward automatically
 - A user can define speed and it can basically “Cruise Control”
- Be able to detect and stop at an obstacle
 - If it detects a obstacle, it will stop and alert the user
- Manual override button
 - This button or function will stop the system and allow for user correction
- Battery level estimate and time estimate
 - Either via a screen or LED array we can show the battery level, as well as a time estimate till empty

Advanced Goals

- Automatic Rotation from obstacle
 - Able to detect and point chair away from the obstacle
- Incline Detection
 - The Chair can detect the incline and juice the motors based on incline
- Edge Detection
 - Be able to detect curbs or sharp drops and prevent driving
- Auto driving from point A to B (Stop at Obstacle)
 - We can set the chair to move to a location via a predetermined route, stopping when that route is disturbed
- Bluetooth control
 - A controller, via bluetooth, can allow for the user to control and set commands for the system
- Tool-less or minimal tool battery replacement
 - Since the battery is a consumable part, we hope the battery can be made in a way that it can be replaced or upgraded

Stretch Goals

- Dynamic Routing
 - When auto driving from A-B, a user doesn’t need to define a path and the system will navigate to it
- Obstacle avoidance and bypassing
 - The system should be able to detect an obstacle and drive around it without user interaction
- Doorway alignment
 - The chair can detect a doorway and align with it and drive through
- Unauthorized Braking

◦ If the user has an unwanted person trying to push the chair, the brake will engage. Or if the chair is on a steep incline and the user doesn't want to movement

With these goals in mind, outside of the basic goals, we may not get to all of them. However to make a fully competent Wheelchair driving system, comparable to other self driving systems these goals must be understood and pursued. Our hope is to provide a system that is useful for users with limited range of motion or vision

2.4 Requirement Specifications

Motor Requirement Specifications:

- Should support a load of 120 kg (including user and chair).
- Should enable a top speed between 4km/h
- Must be capable of climbing a ramp with a gradient of up to 5 degrees.
- Must operate quietly with noise levels not exceeding 50 dB.
- Should have thermal protection to prevent overheating above 120 degrees Fahrenheit.
- Motor casing should be weather resistant
- Each motor must weigh less than 5kg.

Controller Requirement Specifications:

- Must provide smooth acceleration and deceleration without jerks.
- Should support speed modes: slow, normal, and fast.
- Should include over-current, over-voltage, and thermal protection features.
- Response time to input signals should be less than 200 milliseconds.
- Should integrate with a 24V to 36V battery system.
- Total controller unit must weigh less than 1.5 kg

Control Unit Requirement Specifications:

- Should allow operation via joystick
- Must have user-friendly interface for either impaired and non-impaired users.
- Joystick/controller response delay must be less than 300 milliseconds
- Must support Bluetooth connectivity for remote operation (range $\geq$ 6 meters).

- System should log basic usage data (speed, battery level).
- Should be able to operate continuously for at least 60 minutes on full charge.
- Control unit housing should be rugged and lightweight, under 1 kg.

2.5 House of Quality

Our House of Quality is a diagram that will help guide us through what may be complimentary or against each other. This allows us to visualize what solutions will be engineered to work with each other, or what tradeoffs we need to consider. Hopefully when we use this, we can find a nice balance between what we want and what we can achieve.

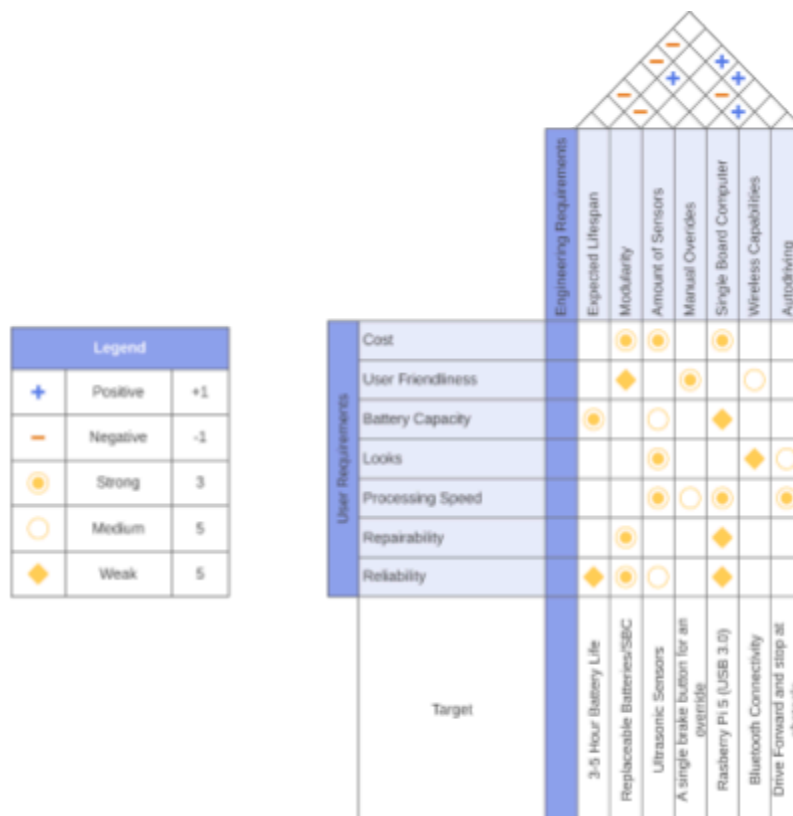


Figure 2.5.1: House of Quality

2.6 Project Diagrams

These diagrams are the basic outline of how the wheelchair will be constructed and connected together. It is intentionally high level to give a general overview and guidance to how the whole system is put together. The diagrams give us:

Hardware Diagram

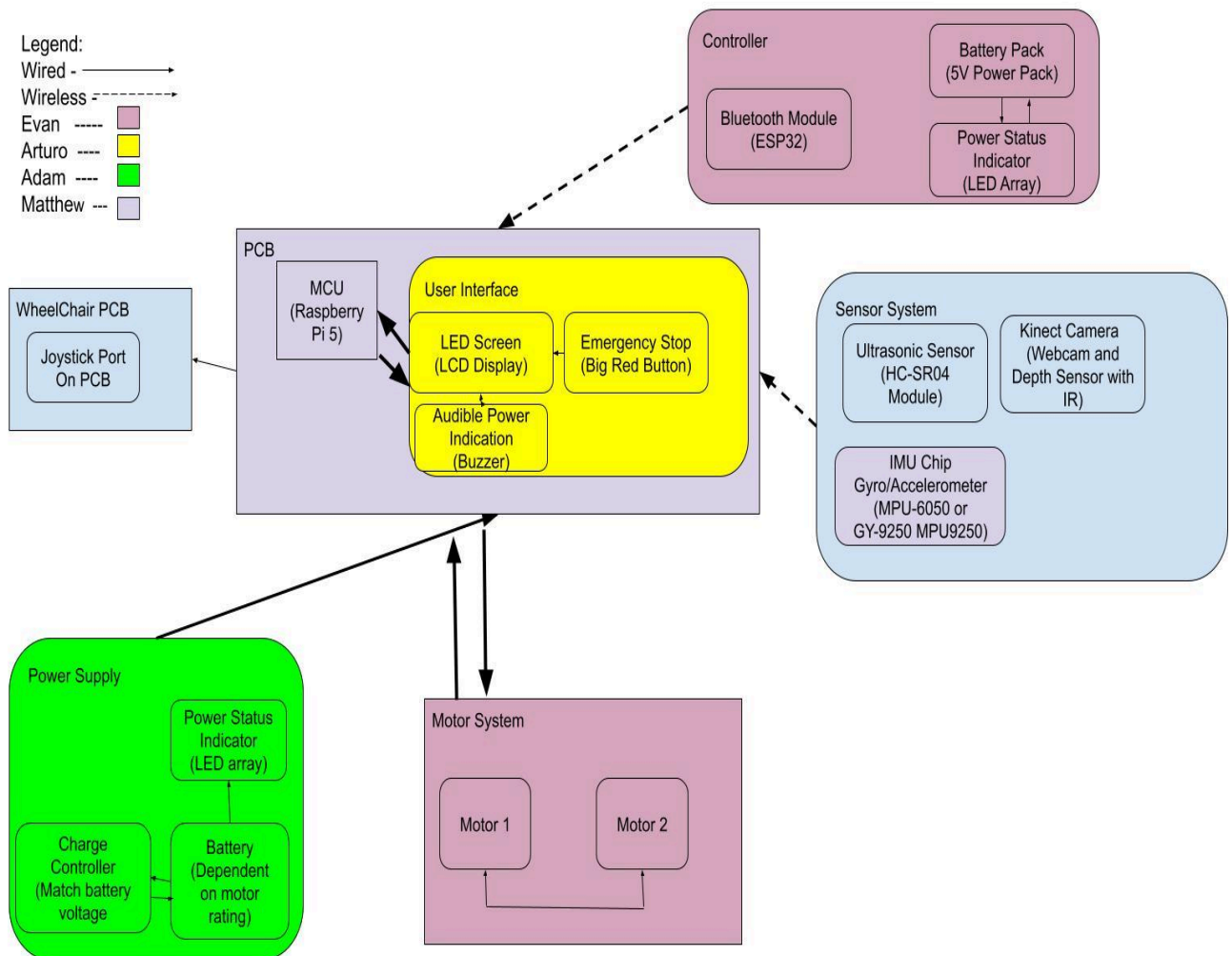


Figure 2.6.1: Hardware Diagram

The diagram above explains the hardware architecture of the system. The main core of this diagram is the MCU (Raspberry Pi 5) which serves as the central processing unit and performs all operations. The hardware systems performance is divided into many interconnected blocks. The user interface provides interaction points with the user which features an LED screen for visual feedback, a buzzer for notification, and an emergency stop button feature. Both the user interface and the MCU will be hooked up on the **PCB**. Connected to the PCB will be the joystick interface where the user can manually override the autonomous working wheelchair to control it themselves.

A **system of sensors** will be located throughout different places on the wheelchair. This system will consist of an ultrasonic sensor (distance detection), IMU Chip (gyro/accelerometer), and Kinect for more advanced visual and depth sensing. All of the data from the sensors will be sent to the MCU for processing. From there, the MCU will give commands to the two **motor system** based on the data. The power supply will power the whole system. The **power supply** features two 12V batteries, a charge controller to charge the batteries, and a power status indicator to track the current charge of the batteries.

Additionally, on the chair, located on the right arm is the **controller** block. Here, the bluetooth module will be hooked up as well as the joystick. The bluetooth module will be wireless communication for the user interface and external devices. A 5 V battery pack will be placed in this to power all of the components inside the controller. Lastly in the controller, a visual display will be shown to the user of the current battery charge.

As one can see from the graph, the color coding represents who in our team will research and work on the certain parts of the project. The dashed and solid lines represent whether the connections between the components and the MCU are wired or wireless. As one can see, most of the connections will be wired. Overall, the design emphasized user interaction, environmental awareness, and robust motor control, all managed by the central Raspberry Pi.

Software Diagram

Main Logic

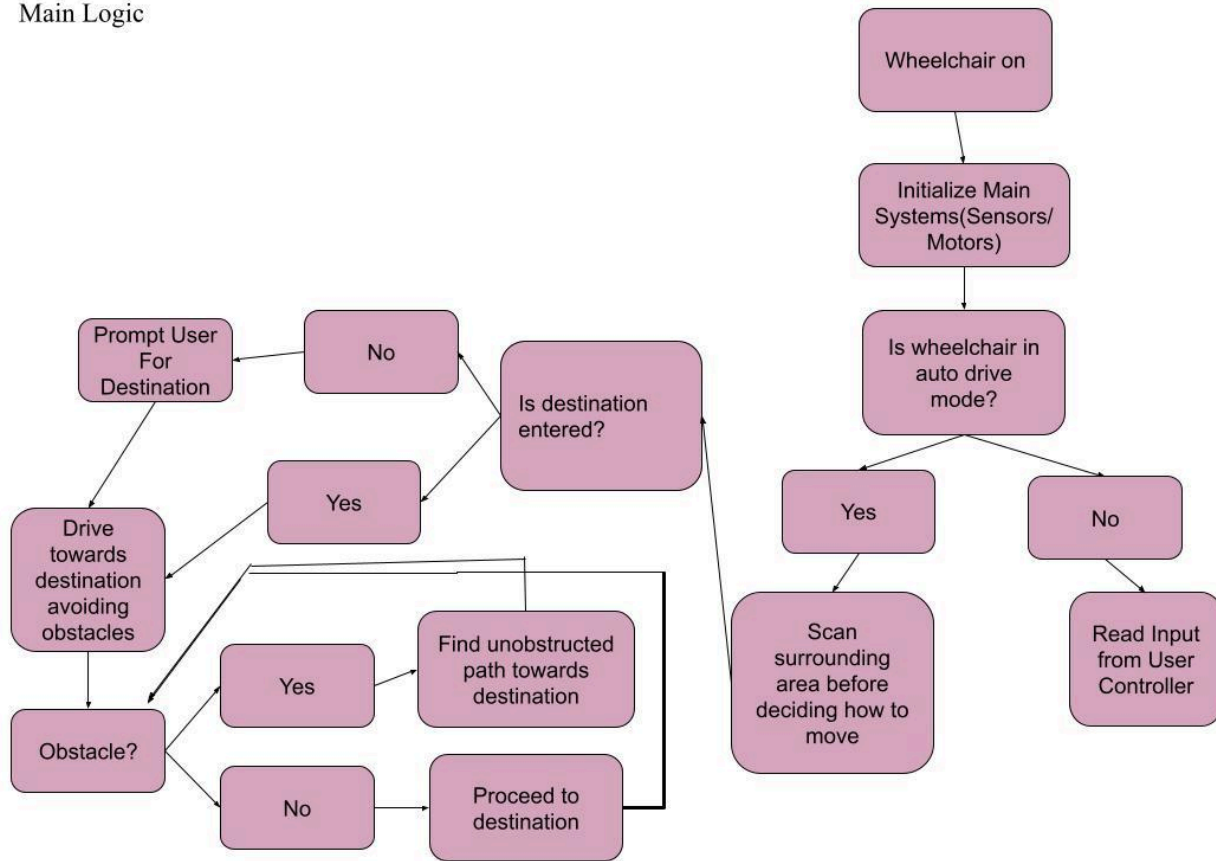


Figure 2.6.2: Software Diagram

The diagram above shows the min software logic for our wheelchair described in a simple form. The logic begins when our wheelchair turns on which leads to the initialization of the main systems. After initialization, the system checks whether the chair is in auto-drive mode or not. If it is not in auto-drive mode, the wheelchair operates under direct user input. If the wheelchair is in autodrive mode, the system will first scan the surrounding area before deciding how to move. After the scan, then the system will have the user input a destination. If no destination is entered after a certain amount of time, the system will prompt the user to enter a location. Once a destination is entered, the system attempts to drive towards the destination while avoiding obstacles. During the driving process, the system continuously checks for an obstacle. If an obstacle is detected, then the system will scan and try to find an unobstructed path with the sensors. If nothing is detected, then the wheelchair will continue along its path already set in place. This marks the end of the software logic sequence. Overall, this diagram is very clear to understand and easy to implement.

Chapter 3 Research and Investigation

Research is the foundation upon which our project decisions are built. Before moving into design or implementation, it is critical that we fully explore the landscape of existing technologies, approaches, and user needs. This chapter serves as a comprehensive investigation into the tools, systems, and methods that will inform our project direction. Through systematic research, we ensure that every decision is grounded in evidence, technical feasibility, and relevance to the user experience.

We begin by examining the key components and concepts central to our design goals. This includes reviewing similar mobility systems, comparing hardware and control schemes, and evaluating past efforts in the development of accessible wheelchair technologies. Our goal is not to reinvent what already works, but to understand what has been done and build on it more effectively. Research also provides the data needed to avoid inefficiencies. Without a structured investigation, we risk making assumptions that lead to ineffective solutions or technical limitations down the line. Whether analyzing motors, joystick responsiveness, or structural modifications, this chapter ensures our decisions are data-driven and backed by engineering best practices. Ultimately, this chapter sets the intellectual groundwork for the design process that follows. The insights we gather here will directly influence our prototyping strategy, component selection, and system integration in the next phases of the project.

Hardware Research

When thinking about the scope of the research for this project, hardware research is going to take up the bulk (if not all) of the budget we set for the project. When researching our hardware, we will focus on the power supply, pcb design, controller, sensor, and the user interface. Some of these components will be more straightforward to research than others such as the power supply and motor system. While going through the research for our hardware, we want to keep in mind the requirement specifications in the overall design as well as the budget. Another big thing to keep in mind while doing research is the quality and durability of each component needed to withstand certain factors like outside temperature, load power, vibration, and UV exposure. Lastly, while doing hardware research, the most important thing to keep in mind is that the components that are being researched on to be put into the design user friendly and meet safety standards as outlined by the IEC 12184: International Standard for Electric Wheelchairs.

3.1 Sensor Technologies

Since we are making a Self Driving Wheelchair, we need to start with finding out what sensors we need. They are our eyes to the world and if we want to be able to detect obstacles then we need adequate sensors that are versatile and accurate. We also looked into self-driving cars, and the sensor arrays they use.

3.1.1 Xbox Kinect V2

When looking into sensors, we found we needed to find a way to find depth so that we can detect how far an object is. We also wanted a way to detect how many objects, as well as map out the environment around the wheelchair. We turned to sensors that we knew of, however one team member had a Xbox Kinect v2. This gamer peripheral has a history of being used in research and experimental settings, as it had quite advanced sensors integrated into it at an affordable price. This sensor array has 2 high quality sensors:

- A RGB 1080p 30Fps Camera
- A 512x424 Pixel 30Fps IR Depth Camera
 - Operates from 0.5 meters to 4.5 meters of distance
 - Has a FOV of 70 degrees horizontal and 60 degrees vertical

These two sensors are quite helpful, mainly the depth sensor. A benefit of this implementation is that it is widely documented and used, whilst also having the RGB camera that can be used for image recognition and such. It also takes a power input of 12V, which is advantageous as will be discussed later.

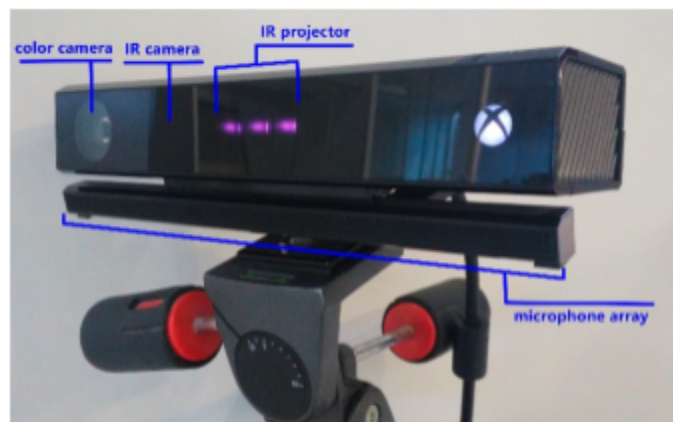


Figure 3.1.1: Kinect V2

When deciding on a sensor we also need to decide a sensor with software that can utilize it. We won't go into detail here and will list technologies in the software portion but it can utilize many popular libraries that utilize the depth sensor. This is the more useful portion of the sensor, as we can recognize and measure our surroundings with remarkable detail. This comes at the cost of

requiring a powerful processing unit to ingest the data produced by the sensor, as it requires OpenGL acceleration to function and most embedded systems would struggle with it. We also require a USB 3.0 connection at a speed of 5Gbps. If we don't have a sufficient CPU and a fast enough data interconnect, the sensor will not provide data properly and will not be helpful for a real time sensing such as self driving.

Related Kinect Projects

Advanced RealTime Road Detection and Autonomous Vehicle Control using Convolutional Neural Networks and Human Learned Driving Models

This project uses some fairly low tech practices to make a self driving ATV. However it fully utilizes the features of the Kinect v1, a similar but less powerful version of the Kinect v2. It utilizes both Vision and AI data to make a fully self-driving vehicle with self steering, variable speed, and utilizes AI models for decision making. This project is more sophisticated with its self-driving algorithm than planned for our project, utilizing AI for vision analysis, but is a very good example of the sensor's use in an application like ours. The reason we will not use the Kinect the same way as this project is due to the power hungry GPU required to run the AI models necessary for vision based detection. Since we are trying to minimize power consumption this would not be a direction we can follow. It is a very interesting project however and some of it's discoveries may be useful for our project

<https://github.com/sieuwe1/Advanced-Real-Time-Self-Driving>

Self-Driving Wheelchair (Graduation Project)

This project is entirely in software, and has code and ways to implement the Kinect if you were to recreate this project in the real world. It also uses a wheelchair design and was a graduation project interestingly enough. Unfortunately it doesn't seem to be easily adapted nor very helpful for our specific use case. However it shows a cool example on how we would use sensors to map out an area.

<https://github.com/YousifAdel170/Autonomous-Wheelchair>

3.1.2 Lidar Sensor

Lidar is a sensor type that is quite new compared to Ultrasonic and IR. LIDAR stands for Light Detection And Ranging. Lidar uses many small light dots to map out an area with depth and color. It's used in a variety of applications, such as:

- Surveying

- Such as geological surveying or geography mapping
- Satellite imaging
 - Seismology
 - Global Mapping
 - Laser Guidance
- Drone Guidance
 - Height Detection
 - Surveillance

Most interesting for our case is for self-driving applications. It's very helpful for self driving implementation as the light dots are not impeded by bad weather conditions and have quite a high resolution. The specific model that we are considering is a 360 degree sensor model FHL-LD20. 360 Degree sensors are common on many advanced self-driving cars like Waymo. This sensor spins at 60 rpm giving a fairly fast mapping at 6 times per second. However this is a 2D



Figure 3.1.2: Lidar FHL-LD20

sensor, which means it is both cheap but has a limit in scope of vision. There do exist 3D sensors but are quite expensive, and for our cases will not be necessary for our self driving implementation. It will sense depth of a single point every degree around us, this will be useful for finding the distance of obstacles in a 360 view around us. This is a 8 meter range around the wheelchair, which is perfect for a wheelchair where obstacles shouldn't be encountered very quickly, but in a self driving high speed vehicle.

Another benefit is this sensor uses Pins to communicate and require much less data throughput. Where other sensors will send lots of data down the data lanes, this uses UART and simpler data delivery. This is fantastic in comparison to the Kinect from the previous section, which requires as much as 5Gbps of data throughput to function properly.

3.1.3 UltraSonic Sensor

The UltraSonic sensor is a simple sensor that is the lowest tech sensor in this comparison. It also has the lowest range of the other sensors at 450 cm of effective maximum distance. However, like Lidar and the Kinect, it provides accurate depth data of the closet object in a 30 degree FOV from the sensor. This could be helpful in making a simple virtual “bumper” so that if another sensor fails to detect an obstacle we can use its input as a last resort. It can’t create a map of its environment like the other sensor mentioned, but its data can still be helpful and is used alongside many self-driving systems. The technology even exists in current cars for crash detection. As such it’s a perfect sensor that can be implemented into our project. It also has a very low power draw, which is perfect for our battery operated system that cannot be tethered to the wall. It also has an extremely fast response time, making it perfect in the real time system we are programming in.

It is also a very popular sensor with much documentation on how it works. We also used them in Junior Design for our distance sensor, so our team has familiarity with the sensor. In fact, we have a few in our possession which means we can reduce costs if we only require a few. Though they are remarkably cheap and easy to implement, thus they are a perfect supplemental sensor.

They are also remarkable low powered and easy to wire, so making edge sensors paired with wireless low powered systems is ideal. They have limited FOV so we will need many of these supplemental systems to be effective, or to fill in holes other sensors have issues seeing

3.1.4 Comparison of Sensors

With all these sensors in mind, we decided to go with the Kinect Sensor as our main sensor. Even though we may use the other sensors as supplemental, the kinect is so much more cost efficient for how much data we can use that it will become the backbone to our sensor system.

Table 3.1: Tech Comparison of Sensors

Criteria	Kinect v2	Lidar	Ultrasonic
Cost	~\$10-\$40	~\$45-\$150	\$2.50
Range	0.5m to 4.5m	0.1m to 8 m	10cm to 450cm
Resolution	IR 512x424 pixels	0.8cm	0.3cm
Power Consumption	32 Watts	0.3 Watts	0.1 Watts

FOV	30 Degrees	360 Degrees	70 Degrees
Frequency	60 hz	6 hz	100000 hz
Link	https://www.ebay.com/sch/i.html?_nkw=kinect+v2&_sop=12	https://a.co/d/1tHBdP7	https://a.co/d/5wJWHeX

3.2 Power Technologies

When designing the SmartStride, it is very important to keep in mind how the power will be supplied to the hardware components. While supplying power, we needed to determine how to regulate the output to prevent damage to components and ensure efficient power management for long-term product reliability. This section will figure this out and in the process will talk about voltage regulators, batteries, and battery level indicators.

3.2.1 Voltage Regulator

The two batteries that will be supplying all the power to the wheelchair will fluctuate a bit in terms of their voltage. Both batteries are connected in series meaning the total voltage will be around 24 V. When the two batteries are fully charged, the voltage can fluctuate 1 to 2 volts higher. If there is a moderate load that the batteries have to supply to the wheelchair, the voltage won't be right at 24V. More likely, the voltage being pushed out will be somewhere between 22.5 V and 23.5 V. If there is a heavy load, especially if the wheelchair user puts the chair at full speed, the voltage may fluctuate even more to somewhere between 20 and 21 V. With the voltage of the batteries fluctuating a lot, this creates a problem for all of the components hooked up to the SmartStride. Components such as the Kinect need a stable voltage. If it gets a fluctuating voltage, it may fry the internal system within the component rendering it unusable. To address this issue, we will need to create voltage regulators for each additional component on the wheelchair as a precaution.

There are many types of voltage regulators. When choosing one, we need to ensure it can maintain a constant output voltage, whether the battery supplies more or less than the nominal level. Also, it is important to keep in mind that we are working with DC voltage so we don't need to look into any types of regulators that convert AC to DC. Since the voltage from the batteries can vary widely depending on the load, having a regulator in place helps prevent that fluctuation from reaching sensitive components. This ensures that all devices receive a consistent and reliable voltage during operation. Shown below are the types of

regulators we looked into, as well as the final selection and an explanation of why we chose that type:

Linear Regulators

A linear voltage regulator in short terms is a device that makes sure there is a constant DC output voltage for any difference with the load conditions or input voltage. This type of device will use a BJT or MOSFET which is controlled by an op-amp usually. The linear voltage regulator acts as a voltage divider to get the regulated output voltage. This type of regulator is also called a step-down converter because the output voltage is always less than the input voltage. This regulator would definitely help in our project, but its downside is that it is not ideal for us because we also need to protect our components from low voltage. Getting more into linear voltage regulators, there are two main types: a series voltage regulator and shunt voltage regulator. The series regulator controls the voltage by placing a transistor in series with the load. The series acts like an automatic potentiometer that adjusts its resistance to maintain the constant voltage at the output. The shunt regulator works by placing the regulating element in parallel with the load. A constant output voltage is kept by “shunting” the extra current away from the load by adding a transistor to ground. If we were to pick a linear regulator for the project, we would definitely choose the series regulator because it is more efficient and doesn't need extra power supplied to it like the shunt type, just to be wasted. If we were to select a linear regulator for the project, we would opt for a series regulator due to its greater efficiency and the fact that it does not consume additional power unnecessarily, unlike the shunt regulator.

Switching Regulators

What makes a switching regulator different from a linear regulator is that a switching type turns off and on a component like a MOSFET to carry out the voltage conversion. Switching regulators are available in three main types: Boost, Buck, and Buck-Boost. Boost regulators are the case for when you want the input voltage less than the output voltage. Buck regulators are the opposite of boost regulators where the input voltage is greater than the output voltage. Using a Boost or a buck regulator would be nice for the SmartStride but they aren't ideally what we are looking for because the regulators only regulate the output voltage for either the case of the input voltage being higher, or lower than the output voltage, not both at the same time. The third type of switching regulator is the buck-boost regulator. It is a mix of both the buck and boost regulator, hence the name. A buck-boost regulator maintains a stable output voltage regardless of whether the input voltage is higher or lower than the desired output voltage. This is the type of regulator that most aligns with our goal of protecting the components on the chair from getting fried from the unstable output from the batteries.

Table 3.2: Tech Comparison of Switching Regulators

Criteria	Buck Regulator	Boost Regulator	Buck-Boost Regulator
Average Cost	4\$	8\$	20\$
Durability	High	High	Moderate to High
Operating Range	3V to 20V	5V to 30V	3V to 30V
Size	Compact	Compact to Moderate	Moderate to Larger
Lifespan	5 to 10 years	5 to 10 years	3 to 7 years
Maintenance	Minimal	Minimal	Minimal to Moderate

After comparing the three different switch regulators above, we decided to go with the buck-boost regulator. Our reasoning for picking this component was due to the operating range. It was a must have for the components on the SmartStride to be protected by both undersupplied and oversupplied voltage. There are some setbacks from choosing this component over the other two. One setback is the maintenance. The buck-boost regulator has many more components than the buck and boost regulators. More components means a bigger possibility of something going wrong with the regulator. The good thing is we can buy in bulk for this type of regulator just in case if anything goes wrong, that way we have many back ups.

Another downside of this type of regulator is that it is the most expensive out of the three. The regulator ranges in price depending on the current capacity it needs and the range of input voltage that is expected from the fluctuating power supply. Most of the components we will be putting into the smart stride shouldn't be drawing anything more than 2 amps at a time. To lower the total cost when buying buck-boost regulators, we should find different converters based on minimal need for each component. For example, we don't need to buy a twenty-dollar 12V, 2A regulator for some small IR sensor that needs 5V 5mA. We could find something way less costly like a LT1302 that costs in the range of \$7-\$10.

Unfortunately the buck-boost has other downsides compared to the other two regulators like size and cost but it serves the need of supplying power within a range that the other two converters can't fulfill. The buck-boost regulator will be our regulator going forward. Below this section is a table that we made to help us

find the most suitable choice buck-boost regulator for the Kinect and other similar power-drawing components:

Table 3.3: Part Comparison of Buck-Boost Regulators

Manufacturer	Model	Price	Size	Max Current Output	Supplier
Onsemi	LM2596DS ADJR4G	\$2.41	7 X 6 X 4.5 (mm)	3A	Digikey
National Semiconductor	LM2577SX -12/NOPB	\$3.27	10.16 X 4.57 X 13.97 (mm)	3 A	Digikey
HiLetgo	XL6019	\$8.99 (3-pack)	50 X 28 X 13 (mm)	5 A	Amazon

Out of the options above, the most suitable choice is the LM2596 made by Onsemi. It is the cheapest out of all of the options per unit and when we buy it in bulk, it will be even cheaper. It makes sense to go with the LM2596. The input voltage range is from 4.5 V to 40 V. A wide input range is nice even though we only needed a range from 22V to 28V. Just in case if we decide to hook another battery up in series to supply more power, we have that extra room to go to 40 V if we wanted. Another reason we decided to go with the LM2596 is because of the sizing. This model had the smallest size out of the other models. An overall goal our team has kept in mind during the research phase is to find appropriate sized components, that way they fit on the wheelchair to where it doesn't add too much weight, or get in the way when the user operates the chair. For a project like this, the voltage regulator needs to be compact to avoid overcrowding the chair's base, which houses all of the electrical components. That's why the LM2596 is a good fit.

While the LM2596 is going to be used with the Kinect, its wide varying input voltage range as well as adjustable output option could be used for other components. If we decide to buy in bulk with this type, it will probably be cheaper to do this than to buy many different single unit regulators from different suppliers. This will be a good thing to keep in mind for when we figure out our total parts list and see how much power needs to be supplied to each separate component. We predict that we won't need more than 10 LM2596 regulators for the total project. Four to five will be needed for all of the components while the five to six of them will be used as backups just in case if something happens to the others.

Voltage Regulation When it Comes to the Motors

When it comes to figuring out how we are going to place the voltage regulators in the chair, we have to keep in mind the feedback the motor produces. The motor doesn't produce feedback the whole time while the chair is running but it does produce it quite a bit. Whenever the wheelchair slows down and changes direction is when the motors produce a different voltage than from the norm. These backed voltage spikes can cause our components to get fried. On top of the components possibly getting fried, the motor feedback can cause noise that could possibly interfere with our sensors and the controller. Also, this noise could cause the microcontroller to reset.

In order to fix this issue we will need to place a few more components and have a few more steps for safer measures. The first thing is to hook up a flyback diode in between the motor and the already built-in motor driver. The diode is placed in parallel with the motor terminals in reverse polarity. The motor driver already acts like a regulator making sure the voltage doesn't fluctuate too much. Without the driver, the motor would probably stop working really quickly. Even with the driver, there is still a decent amount of noise being supplied to the motor when it surges. When the switch (MOSFET, transistor, etc) that is built in to the motor turns off, the collapsing magnetic field caused from the motor action causes the voltage to flip polarity which causes the diode to be forward biased which will cause it to absorb and dissipate the energy in a way to where it won't cause harm to anything.

One more thing we are doing to take extra precaution is placing a capacitor in three different locations to further make sure the motor surges are minimal on the rest of the chair and its components. The first location we are going to place a capacitor is between the motor driver and the battery connection for each of the motors. The second location will be across the voltage regulator input at each device connection to make sure the regulators can handle sharp current more smoothly. The third location we will place a capacitor is across the regulator output so that it keeps the voltage as stable as possible across all the components. In order to find the capacitance needed for all of our capacitors for each of the three-places. The calculations for how we got our capacitances for each of the three areas can be found in the images section under the image "----"

3.2.2 Power Supply - Batteries

While doing research for the type of batteries that are going to be used to power the wheelchair and its autonomous capabilities, we got to keep in mind that the batteries must be able to fit into the existing wheelchair base that has been provided and it must meet the user manual specifications made by the MERIT company as well as the requirement specifications laid out in section 1.4 of this report. Most electric wheelchairs have a dual battery system with most of these batteries laid out in parallel. According to the MERIT MP3R user manual,

the wheelchair previously had two 12V batteries laid out in series providing an amperage of around 44 amp hour.

When looking into what is needed when researching batteries and what type we are going to use, it is definite that for this project, we need the two batteries to be 12 V. For the dimensions of the batteries, they need to be at or below the dimensions of the battery compartments located in the base of the wheelchair. In The MERIT MP3R user manual, it states that the batteries it takes are group 22. Group 22 refers to a Battery Council International (BCI) group size. Group 22 NF dimensions are 9.5" L X 5.5" W X 8.9375" H. The smaller the BCI number, the smaller the dimensions of the batteries. The desired battery dimensions for this project when conducting research will be anything ranging from group 22 or lower.

It is important to note that when looking for suitable batteries, they must be able to handle a decent load. The load for this project will be the two motors, kinect V2 sensor, and main control unit. The motors will be the biggest drawers of power. Looking up the motors for the MERIT wheelchair, it was found that continuous use of the wheelchair will draw about 10 A from each motor which will be around a total of 20 A in total. The controller barely draws any power from the batteries unlike the motors. It can be assumed that max amps drawn per hour from a controller can be under 1 amp. To be on the safe side, 2 amps will be added to the total for the controller in the search for finding proper batteries. Lastly, these batteries will need to supply a load for the kinect sensor. According to an article, the V2 draws about 1.17 A on average and it peaks at 1.57 A. To be on the safe side, we will add 1.5 A for the V2 on average. Another 2 amp hour will be added to the total load calculations just in case. This is the last of our predicted load calculations.

Total load that is projected between the motors, controller, kinect v2, and any other additional add-ons is expected to draw 25.5 Amps an hour. When thinking about cost and how long the wheelchair should last considering how much the load is predicted to draw every hour, it would be nice to be able to run the chair for at least an hour and a half. In order to figure out what type of battery output is needed to supply the load for at least 1.5 hours, this equation is used: Minimum battery output needed = (Desired Run time (hours)) * (predicted load draw in an hour (amps)). Minimum battery output = (1.5)* (25.5 A) = 38.25 amp-hours. It is ok if a battery below this amp hour threshold is chosen. When choosing, we should aim for a minimum of 35 amp-hour or better. It is imperative that when researching for the proper batteries, the specs of the batteries need to be 12 V and have a minimum amperage of 38.25 amp-hour.

Sealed Lead-Acid Batteries (SLA)

The sealed lead acid battery has been around the battery industry for a long time . In fact, this was the first rechargeable battery invented for commercial

use. A good thing about the lead acid battery is that it is one of the more preferred batteries because of its durability and cost-per-watt. According to Battery University, there are “few other batteries that deliver bulk power as cheaply as lead acid.” A downside of lead acid is that it is very heavy. From the MERIT user manual, A group 22 NF battery is about 31 pounds. Having two lead acid batteries brings the load for the batteries alone to 62 pounds which is already a lot of weight for the wheelchair to support. Another downside of this type of battery is their charging capabilities. These batteries are not fast charging by any means. A typical full charge takes 14-16 hours, which isn't the most appealing selling point for consumers of the SmartStride. Even though the Sealed lead-acid battery has its cons, it has a lot of positives that would make it a good fit for the wheelchair.

Gel Cell Batteries

Gelled lead acid batteries date back to the 1950s. There is this jelly like substance within the battery that is made by mixing sulfuric acid with a silica-gelling agent. The gelled material within the battery gives it a higher tolerance to vibrations than most batteries. This is a good thing to keep in mind for the wheelchair. The chair will definitely be used on uneven surfaces as well as go over cracks and mini objects that will cause the interior of the wheelchair to be disturbed. The gel-like material will help keep the battery more durable. Another thing to keep in mind is the gel cell has a higher tolerance to heat than a lot of other batteries. The wheelchair will want to be used outside where the conditions can get hot. This is a good thing to keep in mind. Some limitations of this battery is their sensitivity to overcharging and the manufacturing costs for these batteries are more expensive than Sealed-Lead Acid.

Lithium-Ion Batteries

These types of batteries are pretty new. In 1991, Sony commercialized the first Li ion. The make up of these batteries are very promising and these batteries are the most popular on the market. They have a high energy density and are very light weight. These types of batteries are very ideal to put into the wheelchair. They have no charge memory meaning they do not need exercising to keep it in good shape. Their lifespan is way much more than the SLA and gel cell. A con with these batteries is their costs. They are way much more expensive than both the SLA and gel cell.

Table 3.4: Tech Comparison of Batteries

Criteria	Sealed Lead Acid	Gel Cell	Lithium Ion
Cost	\$100 - \$250+	\$120 - \$300+	\$300 - \$800
Lifespan	2-4 years	4-6 years	8-15+years
Durability	Least Durable:Sensitive to over discharge, heat, and vibration.	Intermediate. Handles deep discharge better than SLA and resists heat better.	Most Durable. Longest cycle life, strong resistance to physical wear.
Operating Temp Range	-15 to 50 degrees celsius.	-20 to 50 degrees celsius	-20 to 60 degrees celsius.
Size and Weight	Group 22 NF: 38.6 pounds	Group 22 NF:37 pounds	Group 22 NF: 10.62 - 15.5 pounds (depending on type)
Safety	Can cause gassing or swelling if overcharged.	Intermediate - more sensitive to overvoltage.	Safest out of all three. Protected by battery management system.
Maintenance	Make sure to use proper voltage and avoid overcharge or deep discharge.	Use a precise charger. More sensitive to overvoltage.	Simple: don't have to worry about overcharging because BMS handles protection.

Out of the three batteries mentioned, we decided to go ahead and obtain two sealed lead-acid batteries for the SmartStride. Even though there are some downsides to this type of battery, there are several significant advantages that come with choosing the SLA. The main reason this type of battery was chosen

was because of the costs. It is the cheapest battery out of the three, ranging from \$100 to over \$250 for the largest size being considered. Despite being one of the cheapest to manufacture, it still delivers good quality. It has a decent battery life and has a decent operating temperature range so the user can take it out in warmer climates if they would like to.

The Lithium Ion battery would have been the first choice between the three types researched but it unfortunately goes through a good chunk of the budget set in place already. Choosing SLA batteries keeps the budget on track. The wheelchair base used as the foundation for our project was originally equipped with sealed lead acid batteries. Since this type of battery previously powered the chair reliably, continuing with the same battery type is a practical choice.

Table 3.5: Part Comparison of Batteries

Manufacturer	Model	Price	Size	Battery Capacity	Supplier
Mighty Max	ML35-12	\$148.99 (2-pack)	7.76" L X 5.12" W X 6.02" H	35 ah	Amazon https://a.co/d/esJkFh4
Duracell	SLI22NF	\$109.99 (each)	22 NF 29 pounds	35 Ah	Batteries Plus
AJC	EB1255 ONB	\$90.19 (each)	22 NF	35 Ah	Battery Clerk https://batteryclerk.com/products/excel-eb12550nb-group-22nf-12v-35ah-sealed-lead-acid-replacement-battery?utm_source=chatgpt.com

The Mighty Max 2-pack deal seems to be the best option out of the three. It comes in a 2-pack bundle that is relatively cheap putting the individual cost of each battery at \$74.95. This is a little over \$15 cheaper than buying the next lowest cost battery out of the options above. We also took our familiarity with the supplier as an option to look into when making this decision. Both Battery Clerk and Batteries Plus are reputable businesses but no one in our group has dealt with them before so that is another reason for why we went with the Mighty Max batteries.

It really doesn't matter too much about the selection of the battery as long as something has the proper specs we are looking for. The specifications we were looking for were the right voltage, size and reasonable range of amp hour. The decision making for choosing a proper battery has proven to be very straightforward in this project as compared to most components. The reasoning is because each of these batteries we were researching were pretty much the same besides the cost. Mighty Max is a very good brand and also these batteries are specifically made for wheelchairs even though they can be used for a broad array of things.

3.2.3 Battery Level Indicator

While first testing our initially given wheelchair without modifications, we noticed that the battery indicator was not reading proper measurements from the batteries. One of the very first tests of the operation of the wheelchair, we noticed that with one of two of our batteries not charged, the indicator still read that both batteries were full. When designing the prototype, we would like to take out the battery gauge that's on the remote and replace it with a simple design concept for a battery level indicator. Below are some different types of battery level indicators we are thinking about implementing:

LM3915 Battery Voltage Level Indicator

This first type of battery level indicator is an integrated circuit. It takes the IC (LM3915) which drives a simple 10 LED display to show the voltage level of an input signal. Each LED represents a 3 dB increase or decrease. What is nice is we can use this type of indicator for charging our wheelchair as well. Below is a picture of the schematic for this IC:

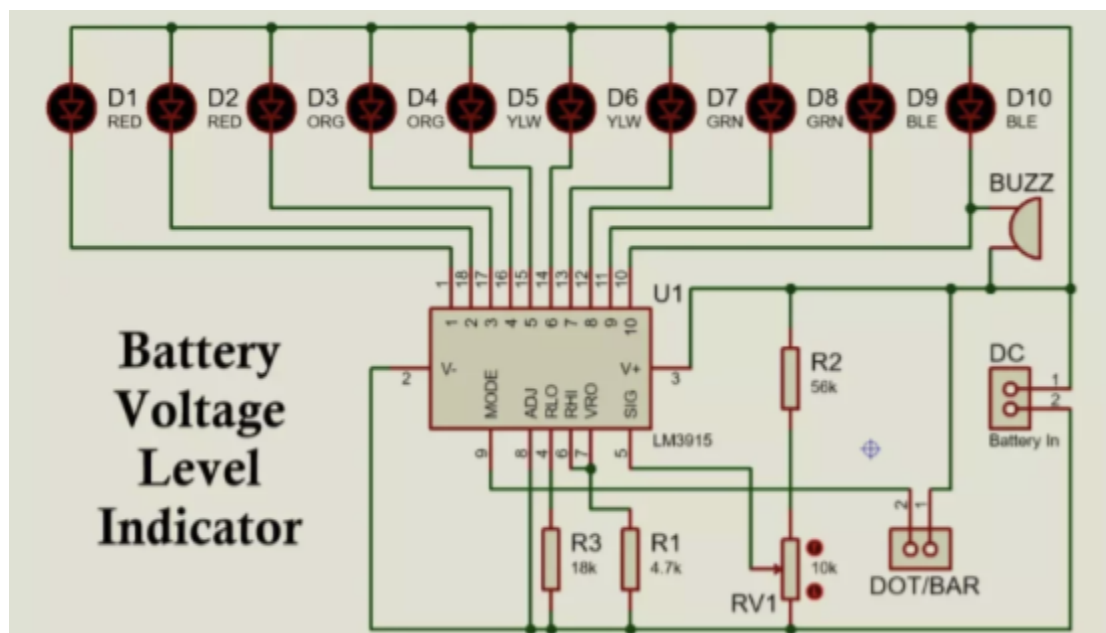


Figure 3.2.3.1: Battery Voltage Level Indicator

One side of the LM 3915 has the 10-pinouts strictly for each of the LEDs. The left side of it facing downwards in the picture has various usages for its pinout. Pin 5 is the signal input where the voltage will come through. It has a 10 K Ω potentiometer hooked up with a 56 K Ω resistor. Pins 4 and 6 on the picture of the schematic are the reference voltage pins. These pins are hooked up to the maximum and minimum voltage levels the IC will detect which is set by the external resistors. Pin 7 is used to control the brightness of the LEDs. Pin 7 is hooked up to pin 6 as well. Pin 8 is hooked up directly to the battery input. This pin will be part of the LED connections. For this pin, as the voltage increases more, more LEDs will light up. Pin 9 is the mode selection pin. This pin will allow you to switch between two different modes to show the LED display. One mode will be dot mode; this mode will show only one LED at a time. The other mode is the “bar mode” which will show multiple LEDs light up to the proper charge that the batteries are currently at. Pins 2 and 3 (located on top and bottom of the IC) are the V- and V+ inputs that are hooked up to the battery terminals. If we do decide to go forward with this design, we can either make another PCB just for this indicator or we can make it part of our main PCB that we are designing for the control unit.

Components

- LM3915 IC
- 10 LEDs
- Capacitor (optional)
- Voltage Input
- Resistors: 4.7k Ω , 18k Ω , 56k Ω and potentiometer that ranges 10k Ω .

16x2 LCD Hooked Up to Raspberry Pi Voltage Indicator

This type of voltage indicator may be the way to go since we can directly hook it up to the Raspberry Pi and implement it on the main PCB board design for the project.

Components

- 4 male pins
- Two pin female header
- 16x2 LCD with I2C adapter.
- Mini Breadboard (optional for testing)
- Raspberry pi 5

- Two same value resistors (10k Ω) - the smaller the value, the lesser the wasted power.
- MCP3008 - 8-channel Analog to Digital Converter

The Pi 5 does not have analog inputs, so we would need an ADC like the MCP3008. For the pinout, refer to the table below:

Table 3.6: Raspberry Pi to ADC(MCP3008) Pinout

MCP3008 Pin	Pi GPIO Pin (BCM)	Role
VDD	3.3V	Function
VREF	3.3V	Reference
AGND	GND	Analog Ground
DGND	GND	Digital Ground
CLK	GPIO18	SPI Clock
DOUT	GPIO23	SPI MISO
DIN	GPIO24	SPI MOSI
CS/SHDN	GPIO25	SPI Chip Select

Table 3.7: I2C LCD Wiring

LCD pin	Pi Pin	Description
VCC	5V	output
GND	GND	Ground
SDA	GPIO2 (pin3)	I2C Data
SCL	GPIO3 (pin 5)	I2C Clock

The general wiring of the whole entire circuit is essentially a voltage-divider. The bottom of the divider will be ground. The middle tap will go to the MCP3008 (directly connected to the pi which is then connected to the LCD display) and the top of the divider to the 10k Ω . The two resistors in series mark the different parts of the divider circuit. Overall, for this design, it looks like since it

is compatible with the pi which we are already using for our project, this would be the best bet for the type of voltage indicator to use.

LM324 Battery Indicator

This type of circuit uses two quad Op-amps (LM324). Each of these two components has four outputs and 8 total inputs for the four actual op-amps which makes sense. You have an inverting and non-inverting input for each. Each of the four op-amps is used as a comparator in this circuit. There are a total of 12 resistors in this circuit. The main function of Resistors R1 through R6 Resistors are meant to prevent damage to the LEDs and maintain LED brightness consistency. R7 through R12 divide the 24V total power supply into decreasing reference voltages and feed those into the non-inverting inputs of each op-amp. Doing this will create thresholds at different thresholds. The op-amps will turn on at different voltage levels activating the LEDs that are hooked up to them. For example, the top op-amp can be set to turn on at the 24.5 to 25V voltage mark which indicates a full charge, that op-amp can be connected to the two green LEDs(one darker than the other) indicating a full charge.

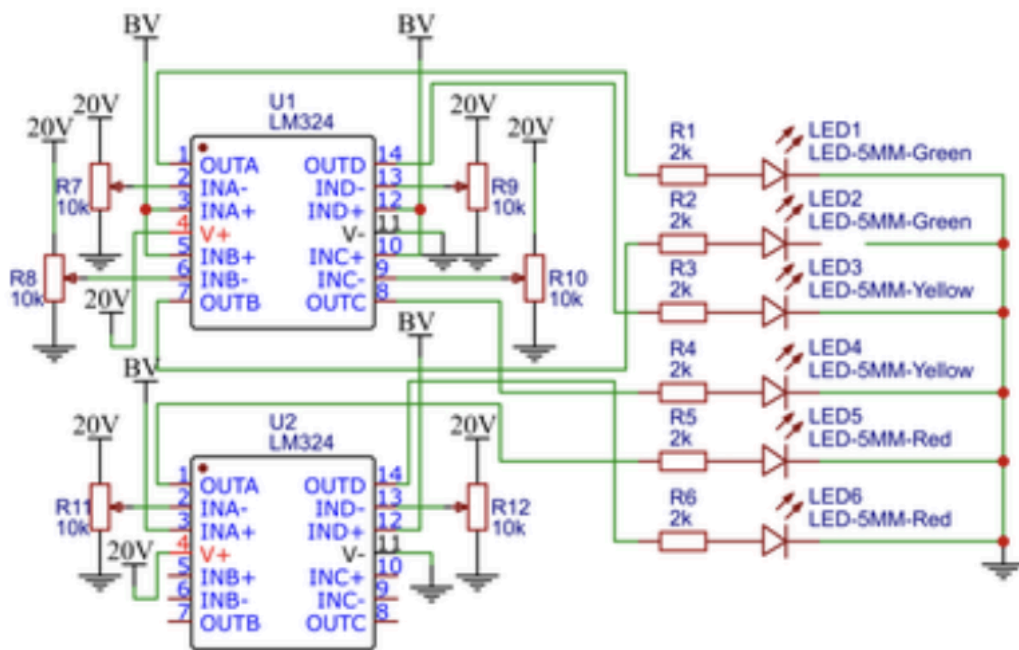


Figure 3.2.3.2: LM324 Battery Indicator

Components

- 12 Resistors: 6-2k Ω , 6-10k Ω ,
- 2 LM324's
- 6-5mm LED: 2-green, 2-yellow, 2-red

In order to implement a design like this, we need to find a good voltage range to set for our op-amps to turn on or off which will control the LEDs. So, since we have two 12V batteries in series, we can expect our battery range from full charge to total discharge is something like 25 V to 21.6 V. It would be ideal for each LED being a + or - 20% increment of the total battery life. Looking at the table below:

Table 3.8: Ideal LED Fluctuating Voltage Range

LED#	Lights Up Above	% Charge
LED1	> 25V	100%
LED2	>24.5V	80%
LED3	> 24V	60%
LED4	> 23.5V	40%
LED5	> 23V	20%
LED6	> 22.5 V	0%

The highest reference voltage goes to the comparator tied to LED1. Each comparator will use a +/- 0.5 V differential when reading the battery status. When the battery voltage is below a certain threshold, that comparator will stay off and that LED will not come on. Overall, this type of design would work very well. The only thing that we do not like about this design is the amount of components in it. It has a total of 20 components.

Table 3.9: Tech Comparison of Battery Voltage indicator

Feature	LM 3915 LED Bar Graph	16x2 LCD +Raspberry Pi + MCP3008	LM324 Comparator
Display Type	10 segment LED bar graph	16x2 LCD Screen	6 LEDs
Voltage Threshold	Logarithmic scale (3 dB steps)	Programmable	Fixed (0.5V steps)
Component Count	15	3	20
Estimated Power Consumption	0.48 to 0.96 W	5 W	0.24 to 0.48 W
Cost	\$15-17	\$25-40	\$13-15
Complexity	Medium: hooking up 10 LEDs indirectly to one op-amp.	Little: hooking up 16x2 LCD to MCP3008 then to pi and coding	High: Working with 8 op-amps technically. They are known to cause issues

Based on the table above, we made our decision to go with the LM3915 LED bar graph. One of the biggest reasons we decided to go with this design was mainly due to power consumption and cost. When we picked our batteries out, we specifically set aside about 1.5 A more of current draw from them for unexpected loads we haven't thought about yet. The battery indicator was something we initially didn't think of implementing into our project, so when we decided to implement this, we had to keep in mind the 1.5 A. Cost was the second biggest reason because it is one of our biggest priorities to stay within our planned budget as shown in chapter 10.

We really wanted to go with the 16x2 LCD indicator with how easy it is to set up and there aren't many parts. The main reason we decided not to go with this and go with the LM3915 is because its power consumption was way too high. 5 W is what it will draw around 210 mA at 24 V. The power draw for the LCD 16x2 is still well within the 1.5 A load that we have left to work with but we would rather go with something that will draw significantly less just in case if we have to add more onto the general load later. By the way, the LM3915 draws 20 to 40 mA. The only thing we do not like about the LM3915 voltage indicator design is the amount of parts. We could lower our LED count from like 10 to 3 but it would be better to have more LEDs so we can be more precise with the battery status. The design is time consuming to put together on a PCB because of the component count but this was to be expected and is not a big obstacle for us.

3.3 Processing Unit Research

We need a processing unit that will interpret the sensor and run the self-driving algorithm. As such we need to pick a processing unit that is fast enough since this is a real time system and needs to have low processing times. Also, since we are considering sensors with high data throughput requirements, we need to spec a unit that has these requirements in mind if we want to be able to use these sensors. We also need to make sure it has the proper pinouts we need to connect to said sensors and also preferably easy to program for the task. Most processing units we will look at are called Single Board Computers (SBC) since our project requires lots of processing power.

3.3.1 Raspberry Pi

The Raspberry Pi family of boards are very popular for many projects. As such it has much documentation of its use and also is quite powerful compared to similar SBCs of its class. However due to its popularity, its demand is quite high and as such pricing can be high. It also is a general purpose SBC so it doesn't excel in our specific use case. These problems can be overcome by savvy purchasing and a sufficiently powerful board to forgo any inefficiencies of being a general purpose SBC

Raspberry Pi 3 Model B+

This SBC is probably the most recognized and popular as it's been out the longest. It sports 4 USB 2.0 ports, 1GB of LPDDR2 RAM, and a 1.4GH Quad Processor. At the time it wasn't powerful, but it had very low power consumption, easily accessible GPIO pins, and a very low price point. Normally for our project this would fit the bill as GPIO based sensors don't require that much processing power. In our case however, we need a 5Gbps USB port and OpenCL acceleration that the Pi 3 is not capable of. As such this SBC is not enough.

Raspberry Pi 4 Model B

The Pi 3 is quite similar to the Pi 4. The main difference is the larger RAM size (a variety of RAM sizes from 1GB to 8GBs) and faster ram. Much faster processors that consume less power. The main draw from the Pi 3 is 2 USB 3.0 ports in place of 2 USB 2.0 Ports. This fulfills the requirement of a 5Gbps data connection for the Kinect v2 we are planning to use. It does have a slight problem however, it doesn't have GPU acceleration for OpenCL applications. We need this if we want to properly process the Kinect v2's data. We can offload it to the CPU for processing, but that increases power draw and can be problematic for our real time component of this system.

Raspberry Pi 5

The most powerful Raspberry Pi to date, with many features over the Pi4 but has similar specs. The main difference is OpenCL acceleration, which allows for the cpu to be freed for our programs when accelerating processing the Kinect's data. It also has a cooling fan that we can put on it to have stable operation. Since this device may be outside in hot weather, it is helpful to have a cooling system built in to keep the system stable.

It does consume more power which may be a problem for our use case, and it runs at a weird 5V 5 Amp power draw to get full performance which is a nonstandard USB PD profile. However we can still power it via the GPIO pins or run it at the lower 5V 3 Amp profile (which reduces the usb port's power management and other nonessential systems) if we can't find a way to power at the 5V at 5 Amps.

It also has the same pinout and formfactor of the others whilst being the most popular amongst hobbyists. This means many users have documented projects using this model. It also means it is the most supported of the libraries we need to run our sensors.

Table 3.10: Tech Comparison of Processor

Criteria	Raspberry Pi 3 Model B+	Raspberry Pi 4 Model B	Raspberry Pi 5
Processor	1.4GHz quad-core ARM Cortex-A53	1.8GHz quad-core ARM Cortex-A72	2.4GHz quad-core ARM Cortex-A76
RAM	1GB LPDDR2	1GB, 2GB, 4GB, or 8GB LPDDR4	4GB, 8GB, 16GB LPDDR4X
USB Ports	4x USB 2.0	2x USB 3.0, 2x USB 2.0	2x USB 3.0, 2x USB 2.0
Video Output	1x Full-size HDMI	2x micro-HDMI	2x micro-HDMI
Power Consumption	~3.5W (idle), ~5.1W (load)	~3.4W (idle), ~7.6W (load)	~4.0W (idle), ~9.5W (load)
Bluetooth	Yes	Yes	Yes
UART	Yes	Yes	Yes

PWM	No	Yes	Yes
SPI	Yes	Yes	Yes
Special Features	-	-	PCIe 2.0 interface, Active cooling header
OpenCL Acceleration	No	No (Possible with CPU)	Yes
Price	~40\$	~70\$	~90\$ + (7.99 for Active Cooler)

Some more notes as all the Pi's share some of these limitations and also some of these benefits.

The Raspberry Pi's all have GPIO pins that have SPI, I2C, and UART functionality. So we can lean on those communication protocols for sensors and parts that need this connection. The GPIO pins are also limited to digital outputs only, and have only a few mAmps that it can push out. This is limiting as we may not be able to power a sensor purely on a few pins as that adds complexity. Also without Analog outputs, it means we will need to integrate Digital to Analog Converters to allow this functionality.

3.3.2 Nvidia Jetson

The Nvidia Jetson is a very similar competitor to the Raspberry Pi, however its main claim to fame is its very powerful GPU that is integrated. As we require a very powerful GPU for our Kinectv2, it makes sense to look into these Single Board Computers that have solid GPU performance whilst also being low powered. On the surface this seems to be an easy choice to forgo Pi's or other Single Board Computers. However, there are some major considerations to be made. Mainly cost and niche appeal means that it is difficult to acquire and also difficult to program for with much less user documented projects. Even the cheaper models are out of production so it is difficult to find on the used market. This is compounded by its niche appeal so it is often not found on the used market. Since we want this project to be made with easily found parts and cheaply, we need to consider the pros of using this SBC. We also need to remember that most of these systems need a development board that may not be a part of the product, which is highly unusual for

Jetson Nano

The Jetson Nano is the first and oldest of the Jetson line. It is also the cheapest but prices fluctuate since it is discontinued. Despite that it is a very capable SBC, with 4 USB 3.0 ports and a 128-core Maxwell GPU. This makes the GPU performance in even this older module much higher than any other SBC

mentioned in this paper. It also has very developed video encoding and decoding performance so it is capable of many video streams if we elected to use RGB camera sensor arrays. It has 4 Gigabytes of RAM, which is modest for its power and is quite fast for SBC of its weight class.

Jetson Orion Nano 4GB

This model is a much more modern version of the Original Nano, with a lot more GPU compute and a marginal amount of CPU compute. It has the same amount of RAM as the previous Nano, which was fairly generous initially so it's not too strange here. The main benefit of the Orion over the original Nano is that it has an Ampere architecture as its GPU. These cores power the 3000 series GPUs from Nvidia, and are very powerful for AI workloads. The AI special Tensor cores are the real claim to fame, meaning we can have AI workloads running in tandem to other GPU workloads since AI can gobble up a lot of GPU compute without guard rails. Ampere also brings more advanced video transcoding support.

To note too is that we get 2 more ARM cores compared to the original Nano, which are faster, meaning that the CPU power is also quite respectable. However, we only found modules listed online, which means we would need to source a dev kit if we wanted to utilize this powerful computer module.

Jetson Orion Nano 8GB

Even though this SBC is named the same as the 4GB module, it is doubled in both GPU and RAM as opposed to just RAM. This means that anything the Orion 4GB can do, this 8GB model does better. This SBC also has some consideration where it can consume much more power if enabled, which means that we can get much higher clock speeds from the SBC. This in turn means that we can enable a “Super” Mode that unlocks the power limit. This leads to much higher performance overall at the cost of power consumption. This is a blessing for embedded systems with Power Supplies connected to the grid as they don't need to consider power consumption as greatly. However for our battery focused project this Super Mode would be much more limited for our uses as we don't want to bring excessive power consumption.

Table 3.11: Tech Comparison of Nvidia Jetson Type

Feature	Jetson Nano (Original)	Jetson Orin Nano 4GB	Jetson Orin Nano 8GB (Super Dev Kit)
MSRP	~\$150 (Discontinued, market price)	~\$159 (Module only)	\$249 (Developer Kit)
GPU Performance	0.472 TFLOPS (FP16)	20 TOPS (INT8, sparse)	40 TOPS (INT8, sparse)

GPU	128-core Maxwell	512-core Ampere, 16 Tensor Cores	1024-core Ampere, 32 Tensor Cores
CPU	Quad-core ARM A57 @ 1.43 GHz	6-core Arm Cortex-A78AE	6-core Arm Cortex-A78AE
Memory	4 GB 64-bit LPDDR4	4 GB 64-bit LPDDR5	8 GB 128-bit LPDDR5
Memory Bandwidth	25.6 GB/s	34 GB/s	68 GB/s
Power Modes	5W / 10W	7W / 10W	7W / 15W (25W Super Mode)

The Jetson line of Single Board Computers are very compelling, due to their high performance ceiling and GPU compute. Which brings many benefits to our systems and sensors. However it has a much more aggressive price that must be weighed with our price to performance targets. Okay, here's a rewritten and expanded version of all the sections regarding potential computing platforms, incorporating more detail and aiming for better flow, based on the provided context.

3.3.3 x86 Computers: A Familiar Foundation with Significant Drawbacks

The x86 architecture consistently emerges as a potential consideration when designing the control unit for our wheelchair project. Its primary appeal lies in its widespread adoption and inherent software compatibility. Virtually all existing software is designed to run on x86 systems, a major advantage simplifying development and integration. Furthermore, the architecture's longevity means it's readily available in legacy hardware, though this often comes with a trade-off in efficiency. However, a key challenge is the inherent power inefficiency of x86 systems. This presents a significant hurdle for a battery-powered wheelchair application, making them generally unsuitable without careful consideration and mitigation strategies. Despite this, we will analyze them as potential options, reserving the possibility of integration should their benefits outweigh the power consumption drawbacks.

LattePanda 3 Delta

The LattePanda 3 Delta stands out as a unique offering within the x86 landscape. This Single Board Computer (SBC) distinguishes itself with its Intel N5105 processor and a robust 2.5 Gigabit Ethernet connection. While x86 systems aren't typically known for extensive GPIO capabilities, the LattePanda 3

Delta bucks this trend, providing a comprehensive suite of GPIO pins alongside USB and audio connectivity. It also boasts NVMe SSD support for fast storage and provides expansion slots for 4G/5G cellular connectivity. A significant benefit is the ability to program it using familiar tools and environments, mirroring standard computer development workflows. Specifically, the ability to run Windows environments is valuable for leveraging existing Kinect v2 integrations and providing a comfortable development experience for a portion of our team. However, the core issue remains: power consumption. As with all x86 systems, the LattePanda 3 Delta suffers from significant inefficiencies, posing a serious challenge to battery life within the wheelchair application. Furthermore, the integrated Intel CPU delivers a comparatively slow GPU, limiting its suitability for computationally intensive tasks like real-time image processing.

x86 Mini Office PC

One team member possesses a compact x86 system—a Dell Optiplex 9020M. This system is equipped with 8GB of DDR3 RAM and an Intel Core i7-4785T processor, a quad-core, eight-thread CPU with a Thermal Design Power (TDP) of 35 Watts. While offering reasonable performance, the 35-Watt TDP immediately raises concerns about power efficiency for our application. The pre-built nature of the system offers a convenient starting point, but customization options are limited. The older DDR3 RAM and the relatively modest GPU capabilities of the integrated Intel graphics further restrict its potential.

Desktop DIY System

Constructing a custom desktop system presents an attractive prospect, potentially delivering the highest performance for the lowest cost, particularly given the availability of spare parts among our team members. However, this approach introduces substantial complexity and significant power consumption challenges, extending the concerns we've already noted with existing x86 systems. A key hurdle is creating a suitable power supply capable of providing the multitude of voltages required by a desktop PC. This would likely necessitate a substantial power supply PCB and potentially require DC-to-AC conversion, leading to significant power loss and the need for larger, more powerful batteries. Implementing a bespoke system of this nature presents considerable engineering challenges and would likely be a complex and time-consuming endeavor.

Server/Cloud Processing

Leveraging server or cloud processing represents a fundamentally different approach, moving computation offboard the wheelchair itself. This offers the advantage of functionally unlimited processing power, and the potential for cost savings through a “pay-per-compute” model. This is particularly appealing for computationally intensive tasks that don't require immediate, real-time

responses. However, the primary drawback is increased latency due to the need to transmit data to and from a remote server. This latency could be detrimental to the responsiveness of our self-driving system. While cloud computing is utilized in various self-driving applications, a reliable and fast cellular connection would be essential for mitigating latency issues. Services like AWS VPSs could be utilized to set up a remote processing system, potentially enabling remote wheelchair control, provided sufficient bandwidth is available.

Table 3.12: Comparison of Processing Platforms for System Integration

Feature	LattePanda 3 Delta	x86 Mini Office PC	Desktop DIY System	Server/Cloud Processing
Processor	Intel Celeron N5105	Intel Core i7-4785T	AMD Ryzen 5 2600	Variable
Graphics (GPU)	Intel UHD Graphics (Integrated)	Intel HD Graphics 4600 (Integrated)	NVIDIA GeForce RTX 2060 (Dedicated)	None / High-End
Memory (RAM)	8GB LPDDR4	8GB DDR3	16GB DDR4	Variable (e.g., 0.5GB - 2GB)
Upfront Cost	~\$280 - \$330	\$0	~\$500 - \$700+	~\$0 (Free tiers available)
Power Profile	Excellent. Very low consumption. Idle: ~1.5W Load: ~15-18W	Poor. Moderate consumption. CPU TDP: 35W System Load: ~40-50W	Impractical. Extreme consumption. System Load: ~300-350W	N/A (Handled by provider)
Physical Size	Pocket-sized. (125 x 78 x 16mm)	Compact. (Small Form Factor PC)	Large. (Standard desktop tower)	N/A (Cloud-based)
Connectivity & Expandability	Excellent. Wi-Fi 6, 2.5Gb Ethernet, GPIO, NVMe, 4G/5G option.	Fair. Standard ports (USB, Ethernet), limited internal expansion.	Excellent. Highly customizable with PCIe slots, multiple drive bays, and ports.	Internet connection is the only interface. Need on site computer to encode inputs

Development Environment	Easy. Familiar Windows/Linux on modern hardware. Supports Kinect v2 well.	Easy. Familiar Windows/Linux, but on older architecture.	Easy. Familiar Windows/Linux on the most powerful hardware.	Complex. Requires remote development, managing instances, and network programming.
Key Pros	• Best balance of performance and power. • Onboard GPIO for sensors. • Modern I/O (NVMe, 2.5GbE).	• No cost to acquire. • More powerful CPU than LattePanda. • Convenient pre-built form factor.	• Highest local performance. • Unmatched for intensive tasks. • Fully customizable/upgradable.	• Virtually infinite, scalable power. • Cost-effective "pay-per-compute" model.
Key Cons	• Weaker CPU/GPU performance compared to desktops.	• Poor power efficiency for battery use. • Very weak integrated GPU.	• Extreme power consumption. • Complex power delivery required. • Large size and weight.	• High latency • Requires constant, fast internet. • Can become expensive with usage.

x86 Conclusion

With all these options in front of us, the Mini PC seems like the best choice. It is the cheapest of the bunch, whilst providing as many benefits of x86 we can muster. It does provide some challenges, such as its weaker GPU and higher power draw compared to the SBC. Its lack of latency with cloud services and small size make it an ideal middle ground and get us closer to the non-x86 computers listed.

3.3.4 Processing Unit Conclusion

Lets take the 3 choices from the last sections and compare them. We can then see which one we should choose ultimately between all of them.

Table 3.13: Comparison of Compact Computing Platforms

Feature	x86 Mini PC (Dell 9020M)	Raspberry Pi 5	Original Jetson Nano 4GB
---------	--------------------------	----------------	--------------------------

Processor (CPU)	Intel Core i7-4785T (4-Core)	2.4GHz Quad-core ARM Cortex-A76	1.43GHz Quad-core ARM A57
Graphics (GPU)	Intel HD Graphics 4600	VideoCore VII (OpenCL)	128-core Maxwell (CUDA)
Memory (RAM)	8GB DDR3	8GB LPDDR4X	4GB LPDDR4
Power Profile	Poor (~40-50W Load)	Excellent (~9.5W Load)	Excellent (5W / 10W Modes)
Cost & Availability	\$0 (Owned)	~\$90 (Widely Available)	Discontinued (High used price)
Ecosystem	Familiar PC Environment	Vast Community & Support	Niche AI Community
Key Pros	• No upfront cost • Strong CPU	• Low cost & available • Huge ecosystem • Meets all project needs	• Strong GPU for its time • 4x USB 3.0 ports
Key Cons	• Very poor power efficiency • Extremely weak GPU	• Weaker GPU than Jetson • Nonstandard power input	• Outdated & hard to find • Slower CPU

Our Team decided to go with the Pi 5. This was due to it being the solid middle ground over all the other options. The lower power consumption, small size, and GPU acceleration makes it most ideal. Another important reason is the fact it is a highly available and popular part. This project we want to make in a way that it can be somewhat applied to other chairs and using parts that can be easily sourced, utilizing non-standard parts and depending on niche availability is against this goal. So we finally come to our chosen part, the Raspberry Pi 5.

3.4 Wireless Processing Unit Research

Table 3.14: Evaluation of Wireless Protocols for Embedded Systems

Technology Name	Frequency Band	Typical Data Rate (Mbps/kbps)	Typical Range (Indoor/Outdoor, meters)	Latency (ms)	Power Consumption (Active/Sleep)	Interference Susceptibility	Key Security Features	Network Topology
BLE	2.4 GHz	1-2 Mbps	10-240m	Not	Low	Moderate	AES-128,	P2P,

	ISM			specified, but "fast connection"	(months-years battery)	(2.4 GHz band)	SSP/Secure Connections, ECDH	Star, Mesh
UWB	3.1-10.6 GHz	Up to 27 Mbps	Optimal: 1-50m; Max: 200m	<1 ms	Very Low (μ W-mW, long battery)	Very Low (wide band)	Time-hopping spread spectrum, concealment	P2P
Zigbee	2.4 GHz ISM	Up to 250 kbps	Indoor: 10-100m; Outdoor: 300m	Low (often <300ms)	Very Low (25% of Wi-Fi, years battery)	Moderate (2.4 GHz band)	AES-128, authentication, key management	Mesh
Z-Wave	900 MHz ISM	Up to 100 kbps	Indoor: 30m; Outdoor: 100m (extendable)	Low (often 300-800 ms)	Low (years battery)	Low (900 MHz band)	Advanced encryption (S2), authentication	Mesh
Proprietary RF	Sub-GHz, 2.4 GHz	Highly configurable	Highly configurable (long range possible)	Very Low (optimized)	Very Low (years battery)	Configurable (Sub-GHz is low)	Customizable, low-level access	Flexible (P2P, Mesh, etc.)
Wi-Fi	2.4 GHz, 5 GHz	Up to 1 Gbps	Optimal: 0-50m; Max: 500m	5-100 ms (variable)	High (80-260mW active, 5-20W router)	High (2.4 GHz band, congestion)	WPA2/3, enterprise security	Star (AP-centric)

3.4.1 Communication technologies

Multiple wireless technology standards should be compared to come up with one that meets the use case of our chair. We need a low latency, small power consumption and a decent range. We aren't expecting to move the chair from miles away but a feasible distance for remote control is ideal. Certain wireless technologies benefit more when in large clusters or in huge nodes of multiple devices. This isn't in line with our use case so while we may look into these technologies a bit they won't be our main focus.

3.4.1.1 Ultra-WideBand

Ultra-WideBand transmits data and signals over bandwidths larger than 500MHz. Ultra wide band systems are currently utilized to determine real time location thanks to reliability and accuracy. It functions well in indoor environments with examples being industry use in medical and healthcare facilities. Real time movement tracking and time of flight information are all regular applications behind this technology. Implementing this with actual equipped electronics could be costly compared to other technologies presented.

3.4.1.2 Zigbee

The ZigBee network technology has its primary use in mesh networks with multiple low power devices operating and communicating together on the meshed network. It operates in the 2.4GHz ISM band and supports rates up to 250kbps. It is designed for low latency communication. Zigbee networks are also much more power efficient in comparison to other technologies like Wi-Fi. Its primary specialties are mesh networking utilizing devices as relays that can extend the networks range. While useful this doesn't fall in line with our specific needs.

3.4.1.3 Z-Wave

Z-Wave is another Mesh network communication technology that focuses on low power and utilizing an incredibly low bandwidth range of below 1Ghz. This allows for less interference from other communication that may occur on more congested bands like the 2.4Ghz one. Z-Wave has uses in home IOT devices like lights and other low data rate hardware. It communicates at a pretty slow 100kbps but for its intended applications this is suitable.

3.4.1.4 Bluetooth Standard

The Bluetooth Standard is a widely known and adopted communication standard for consumer devices like phones and speaker bars. Bluetooth utilizes a master/slave architecture with the master communicating with up to seven slaves. The master sets the clock speed for packet exchange. The role of master and slave can change based on both devices agreeing and the function of the use case. This allows for some versatile applications of the communication technology when interfacing one device with multiple bluetooth connections. Bluetooth operates on the unlicensed 2.4GHz ISM band. This allows us to utilize these frequencies freely without needing to request authorization from anyone else.

3.4.1.5 Bluetooth Low Energy standard

Created with the intention of maintaining similar communication ranges and capabilities as regular bluetooth while using much less power. It operates on the 2.4GHz radio frequency. The power efficiency is met but using bursts of data transmission as well as regular intervals in low power states outside of transmission times. Devices usually need a profile, something that tells the

protocol how exactly the device is operating. Most use the (GATT) Generic Attribute Profile for sending and receiving data simply. BLE is also capable of a mesh effect where multiple devices on BLE can send information to one another in almost a cascading effect of data transfer. This allows for the control of multiple devices at once easily. BLE sends data using a 2MHz spacing that allows for 40 channels of data transmission.

3.4.2 ESP32

The ESP32 is a 2.4ghz dual core microcontroller with Wi-Fi and Bluetooth capabilities. A main point of the board is also its FreeRTOS framework. This preemptive OS is extremely well made for multi tasking and proper task scheduling and priority control. This allows for quick interrupts that can enable speedy control transfer between autonomous driving and controlled driving. The ESP32 also boasts a large number of ADC channels at 16 counts. It also has 2-8bit DAC channels for analog output. This is essential for our joystick signals to be read. The ESP32 is also very cheap and low power draw as well with an amp of draw when bluetooth and wifi are both running while cores are processing data. A 3.0v to 3.6v requirement allows for easily replaceable or rechargeable single cell batteries. Further lending to our accessible design. Avoiding proprietary design constraints helps allow our users to repair and continue to use the wheelchair as much as possible.

3.4.3 Raspberry Pico W

The Raspberry Pico is another small wireless controller board only boasting Wi-Fi integration. It is powered by dual ARM Cortex-M33 or Hazard processors, capable of running at up to 133- 150MHz. It's power draw of 1.0 - 3.3v for operation contributes to extremely small current drain. Raspberry Pico is capable of using C and Micropython. C lends to more in depth memory management and control to deal with limitations of its 4MB on board Flash. The most unique aspect of the Pico is its Programmable I/O(PIO). These may be useful when needing to implement custom or non standard communications with sensors or other peripherals we may include. This mainly helps if the sensors we aim to use don't use standard I2C or SPI.

3.4.4 Arduino Nano RP2040 Connect

The Arduino Nano Raspberry Pi® RP2040 silicon; a dual-core Arm Cortex M0+ running at 133MHz. It has 264KB of SRAM, and a capacity of 16MB flash memory being the largest of the bunch that we've seen. IT also has a radio module with full Wifi connectivity along with both bluetooth and BLE modes. This board also boasts a special array of sensors integrated into it. Such includes a six axis IMU with AI integration. This sort of sensing integration may be crucial in making a decision as IMU data is another thing we plan to consider in our

self-driving algorithms. Integrating MicroPython can also allow easy integration and leveraging of the Pykinect packages mentioned later in the software research sections.

Wireless Computer Comparison ESP32 VS Pico W

When comparing the ESP32 (specifically the ESP-WROOM-32 based board) to the Raspberry Pi Pico W, the ESP32 presents clear advantages in raw processor speed, the number of available ADC channels, and the presence of dedicated DAC channels. This benefits our use case of a wireless controller with wanting to read input from an analog joystick as well as receive a stop command from the emergency button. ESP32 and Raspberry Pico have comparatively low power draws but the Pico wins in that regard with lowest needed operating voltage and less current drain since the cpu is slower as well. Such minimal difference in power drain isn't as important as processing speed and power so the esp32 continues to be more beneficial for our use case. Higher clock speeds on the esp32 help with executing critical tasks on one core while being ready to work on a problem like regaining control from autonomous driving on the other core.

The FreeRTOS framework is also naturally suited to multi-tasking and concurrent task execution. And with it being a preemptive real time operating system. Interrupting the autonomous control and passing it to the user or emergency stopping is able to be done very well.

The Raspberry Pico does offer other advantages the esp32 forgoes. One including programmable I/O. This PIO allows the Pico to control GPIO pins with extreme precision independent of CPU cycles. This frees CPU cycles and assists in enabling the implementation of sensors or other communication interfaces that don't work properly or clear along standard communication protocols like I2C and SPI. While this may not be an issue in our implementation it may be beneficial in processing the Kinect sensor data compared to the ESP32 if standard protocols don't succeed as well. This is yet to be seen until prototyping but acknowledging this capability is important for future troubleshooting or perusal of our desired functionality.

Table 3.15: Tech Comparison of Microcontroller Development Boards

Criteria	ESP32 ESP-32S	Raspberry Pi Pico 2 W	Arduino Nano RP2040
Processor	32-bit LX6 microprocessor at 240MHz	Dual Cortex-M33 or Hazard3 processors at up to 150MHz	Raspberry Pi® RP2040 at 133MHz
Flash Memory	4mb SPI flash	4mb on board	16MB Flash IC

	memory	flash memory	
ADC Channels	16 ADC channels	3 ADC channels	8 ADC channels
DAC	two dedicated 8-bit DAC channels.	No dedicated DAC converter Can use PWM for rudimentary version	No Dedicated DAC but PWM is possible
Operating Voltage	Operating voltage of 3.0 - 3.6v	Operating voltage 1.0-3.3v	Operating Voltage of 3.3v

3.5 IMU Chip Research

IMU Chip comparisons/magnetometer

Any IMU(Inertial Measurement Unit) will be critical for certain decision making and the functional operation of our self driving wheelchair. Data such as real time speed, heading of the wheelchair, and angular velocity are crucial for a safe and comfortable drive. These readings give us a higher degree of control and system understanding. Without it we'd lack crucial information for the self-driving algorithm.

Real Time Speed data is critical for maintaining reasonable speeds depending on the environment the wheelchair user finds themselves in. This also helps provide smooth acceleration and deceleration during our self driving algorithm. Hard and quick jerkiness is the last thing we need when our wheelchair is course correcting and this data will prevent that. The gyroscope's angular velocity gives clear information on inclined or declined environments. This in combination with the real time speed data lets the wheelchairs algorithm decide proper speed dependent on incline. All this in combination with the Kinect's depth map and camera sensing will create an adaptive and functional sensing system we can utilize for the self-driving algorithm.

3.5.1 MPU-6050

The MPU-6050 is a cheap and easily accessible board regularly used for quick prototyping and incorporation into designs. Surface Mount versions exist This IMU features a built in 16 bit ADC converter with a 3 - axis accelerometer and 3- axis gyroscope. The chip also incorporates standard I2C communication which will work with any of our main computer options we provide. This falls

under a 6 - Degrees of Freedom(6-DOF) solution. Other IMU boards are able to offer higher degrees but the tradeoff is price and complexity.

3.5.2 GY-9250 MPU9250

This is almost like an upgraded version of the previously mentioned IMU chip. Also incorporating an accelerometer, gyroscope and additionally a magnetometer for more degrees of freedom sensing. Featuring 9-DOF that gives additional data such as absolute heading and orientation of the chair. These data streams can give our algorithm more to work with and have emergency responses for situations such as the chair tipping over or getting stuck somewhere. This chip is also cheap at 18 dollars a board for a prototyping board and has SMD options for final PCB fabrication.

3.5.3 BNO055 Absolute Orientation Sensor

The largest appeal for this 9-DOF sensing board is its capabilities of simplifying the fusion algorithm - that which takes all simultaneous streams of data and outputs the proper orientation - by doing all this processing on board and simply giving data in forms like Euler angles or vectors. This board can help us avoid struggling with developing a proper fusion algorithm compared to the previously mentioned ones. However it has drawbacks in its communication standards. Supposedly the I2C implementation of the BNO055 can violate the standard protocol at times. Certain boards may not play well with these issues but it may be something to consider anyway due to the fusion algorithm advantage it has.

3.6 LED Screen Research

In order to ensure that the user has both a clear understanding of the current condition of the wheelchair and any information regarding the automotive capabilities of the automated movement system a display of sorts must be implemented for the user's reading capability. This can be easily done by implementing a LED screen that is entirely within view of the user and consistently reading from the other modules built into the chair to accurately convey everything happening. When choosing a display we must consider three very important details to ensure reliability and efficiency. The display must be fully compatible with the central computer in charge of managing the controller and automated system as it will be sending out all the commands of the automated functions and must display them to the LED. The display must also be both large enough to be legible by anyone sitting down in the chair and compact enough to be built into the controller. Lastly it has to be able to read and display information related to the battery, mainly the charge status as it is important that the user has an easy way to know whether it needs charging or not.

3.6.1 2.0" 320x240 Color IPS TFT Display - ST7789 EYESPI

The smallest and cheapest of the LED displays we can use, the ST7789 is a 2-inch display with a resolution of 320 by 240 pixels with full RGB color capability. It's compatible with different microcontrollers including the ESP32, Arduino, and Raspberry PI with both a 18-pin flexible cable and FPC connector board allowing us to easily adapt it to whatever electronics we are using. It is a true TFT display giving it a quicker refresh time compared to other displays such as CSTN. Along with its great compatibility it already has a full graphics library that comes with enough information to adjust our code to display whatever information or images we want.

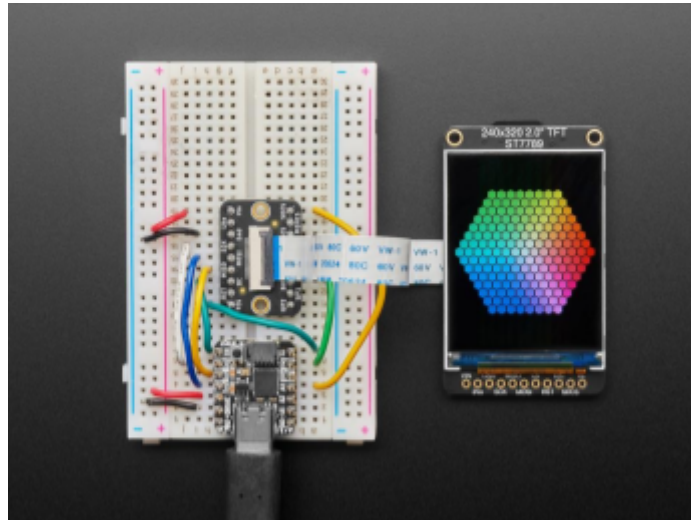


Figure 3.6.1.1: ST7789 EYESPI

3.6.2 3.5" TFT 320x480 - HX8357D

This display is of the same brand as the previous display but with a larger screen size, higher resolution, and touchpad compatibility. Its larger size and increased resolution makes it more ideal for better legibility as it will be both easier to see and allow us to write out larger texts or create more detailed images compared to its smaller counterpart. It also comes with its own full graphics library making it just as easy to adapt to suit our needs. The 5-pin layout will also make it easier to set up as it comes with details on how to properly use each pin and comes with a built-in controller with RAM buffering making the task of programming the LCD more than easy.

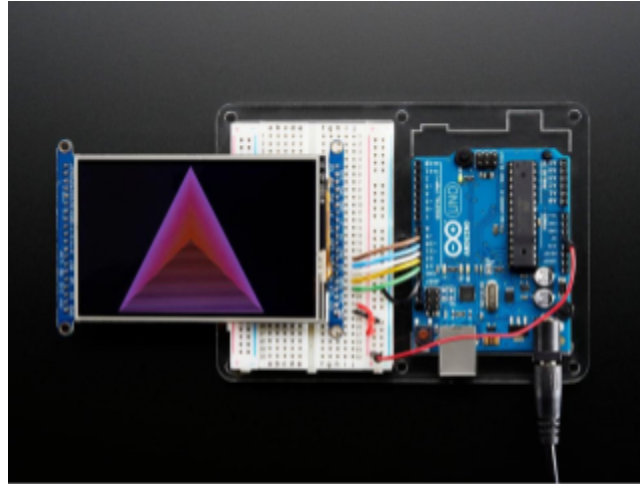


Figure 3.6.2.1: HX8357D

3.6.3 2.8" 320x240 ESP32-2432S028R

While this display is a step back in terms of size and resolution compared to the previous model it makes up for it with its main strength of having an ESP32 already installed within the display board. This gives it both wireless bluetooth capability and a USB port to allow us to connect to the board directly. As a bonus since we are already using an ESP32 for other modules of the wheelchair it will be easier to manage our software and remove the need of having to use another microcontroller to use the LCD. There is also a Github repository and community involved with designing software and other physical commodities such as 3D printed stands and casings giving us a plethora of different models and resources that can be used to design a mounting device for the display.



Figure 3.6.3.1: ESP32-2432S028R

3.6.4 Display Results

All three displays are viable options for our project, each with its own strengths and weaknesses. Out of all of them the 3.5 inch display would be the most effective at displaying information as its increased size will allow us to display either more data or make it easier to read for people with poor eyesight. The ESP32 display has the most built in functionality and will most likely save us time and space when attaching the display to the machine.

Table 3.16: Comparison of Display Modules

Name	Size (in)	Resolution	Cost	Source
ST7789	2.0	320x240	\$19.95	adafruit.com
HX8357D	3.5	320x480	\$39.95	adafruit.com
2432S028R	2.8	320x240	\$13.90	makerfabs.com

3.7 Button Research

Since the main principle of the emergency stop function is that it should be an independent system to the other controls in case the emergency is that they are malfunctioning, the emergency stop will be using a physical button to override the controls. The button will be built directly into the controller frame where it can have easy access to the user and so that it can connect directly to the controllers so the override can come from the same input as the joystick. In terms of the physical aspects of the button and the ergonomics behind it we need to purchase a button that will be usable to anybody that may be physically weaker than the average person so that in the event of a failure or error the user may prevent further injury or save their own life

3.7.1 Adafruit 560

The 560 is a very simple push button that has a glowing LED attached to the outside ring. It is 16mm wide with a total length of 30mm and with a recommended casing density of $\frac{1}{2}$ inches. It has five ports, one to set the switch to normally-open and another to normally-closed, along with two ports for operating the LED at 6V. The LED is the selling point of this button as it will make it easier for the driver to identify the location of the button when in either a dark setting or a stressful emergency situation.



Fig 3.7.1.1 Adafruit 560

3.7.2 APIELE Stainless Steel Push Button Switch

The APIELE push button is similar to the Adafruit in terms of size, however unlike that button this one does not come with an LED and only comes with a normally-open terminal. It is the most basic type of switch possible however it also comes with the added bonus of being water-resistant. This could prove quite useful as the complexity of the button is the least of our concern, we

are instead interested in ensuring that no matter the situation the emergency stop function will be fully operational.



Fig 3.7.2.1 APIELE Push Button

3.7.3 E-Switch PR144C1900

If you were to imagine a big red button that you would press in case of a dire emergency then this button switch is it. Like the APIELE push button this one only comes with two solderable ports that set the button to a normally-open state. It is also the largest of the buttons that have been researched, meaning that it will probably be the easiest to identify in an emergency situation and would be more visually obvious to what the function of the button is. The button has a total width of 22mm and has a total depth of 34mm, requiring us to create a significantly larger hole for the component into the controller casing.



Fig 3.7.3.1 E-Switch

3.7.4 Button Results

Since there isn't much of a need for a complicated component, each of these buttons will function perfectly well within our project scope. Both the APIELE and E-Switch will be extremely simple to setup and use but the E-Switch will be the hardest to integrate into the controller since it is larger than the other buttons and doesn't use a screw like casing to secure into the controller. The Adafruit has the bonus of having an LED and an alternative port so that we do not always need to run the normally-open function required to read the button.

Table 3.17: Feature Comparison of Candidate Pushbuttons

Name	Button Width (mm)	Hole width (mm)	Button Length (mm)	Price (\$)	Source
Adafruit	16mm	16mm	30mm	\$4.95	digikey.com
APIELE	16mm	16mm	22mm	\$6.99	amazon.com
E-Switch	20mm	22mm	34mm	\$2.42	digikey.com

3.8 3D Printed Material Research

In recent years, the use of 3D printers has expanded greatly and has grown to become an easy to use consumer product that allows the prototyping and development of engineering components without the need of having to source materials and process them through a factory. This usefulness is the main reason why we will be using 3D printers to develop the prototypes we will be using to secure our components to the refitted wheelchair being used in this project. We also have access to two types of printers, filament printers provided to UCF students thanks to the Engineering Building's TI Lab and resin printers that we have personally acquired. Since we do not need to source any printers for our designs, what we have to consider instead is what type of material to use for our components and which one will work best.

3.8.1 Solid Thermoplastic Filament

Filament printers are the most common types of 3D printers available to the public and one of the easiest to use. They don't need to worry about curing or being handled correctly to avoid any toxic chemicals from harming organisms as they just need to be set up and set to print without too much preparation. Filament printers are also generally larger than resin printers so if we have to create a large casing or component filament printers can easily create it so we can immediately move to testing it. In terms of the material itself, filament comes in a lot of options but something like PLA is both cheap and strong making it the

best choice for our prototyping phase. Filament does also come in a variety of other versions including some wear and impact resistant filaments. These versions can prove to be more useful if we plan on making a variety of components that would need more specialized filaments in order to increase their longevity and durability.

3.8.2 Liquid Photopolymer Resin

Resin printers have been increasing in popularity recently due to hobbyist consumers using them for more specialized projects such as model design and creating more detailed components. A resin printer doesn't use solid filaments but liquid resin to print objects by using UV exposure to create a whole layer at once. This makes it faster than filament when printing objects with layers that have a large surface area, however unlike filament resin must be washed and cured to avoid any chemical poisoning. The resin itself is also more brittle than filament and could prove to be less durable when creating parts that have a lot of tension put on them. Resin printers are more detailed than resin and can create smoother surfaces. Resin itself is also cheaper by volume and like filament comes in a variety of different versions that can prove to be tougher or more flexible giving us the ability to make up for some of resin's shortcomings while still keeping its details.

3.8.3 Printer Results

Reviewing both filament and resin printers gives us a clear understanding of what printer works best for each component. Parts like the casings of the computers and other components should be made with the resin printers as they would have a large surface area on multiple layers and will require a lot of precision if we want to save as much space as possible. Along with the fact that we do not need to worry about the casings receiving much stress and damage using regular resin should be enough to protect the components from being damaged from environmental and kinetic damage. Any component that we will be putting under duress will be needing a filament printer as we can not rely on resin being strong enough to resist the tension we plan to put these components under.

Table 3.18: Feature Comparison Between Filament and Resin Printing

Feature	Filament	Resin
Resolution	100-200 microns	25-50 microns
Build Volume	300x300x300 mm	130x80x160 mm
Durability	Very Strong	Brittle
Surface Finish	Visible Layer Lines	Smooth Surface

Cost (per kg)	\$20	\$35
---------------	------	------

3.9 Software Research

There are many software technologies we need to lean on when integrating the components and sensors.

3.9.1 Kinect Libraries

Since we are utilizing a Kinect, we need libraries to communicate with the proprietary protocols. Pykinect2 will allow communication and utilization of the 3D maps generated by the KinectV2. Using this data we can make decisions for the wheel chair to avoid obstacles and make anticipatory moves to anything it may see in advance. We can also extract Depth Maps and more advanced recognition content.

PyKinectTK allows for plotting of our data long plot libs for potentially easy debugging of data streams coming in the kinects sensors. This may allow us to potentially process the data during our self-driving algorithm or understand how the kinect is operating during real time use.

Kinect skeleton tracking capabilities may also open up to the wheelchair having simple following capabilities. It could enable the chair to follow a caregiver, for example a nurse in a hospital, reducing the need for the patient to have motor functions or controls and giving the nurse potential peace of mind knowing their patient is consistently behind them. KinectV2 is capable of tracking up to 6 individual skeletons at once and processing it all. Utilizing this capability can open more software features for the wheelchairs self driving capabilities. Capturing the flat back is possible and easily trackable and programmable using the

Utilizing and integrating these packages and data together with a self-driving algorithm will pave the way for our wheelchair to autonomously maneuver around obstacles with quick response time. We Will have a clear stream of data to compute and make the best possible decisions for our wheelchairs passengers safety and comfort. Having all these data streams will also allow us to stay in line with whatever safety standards as well as compliance with any ADA standards the wheelchair already met.

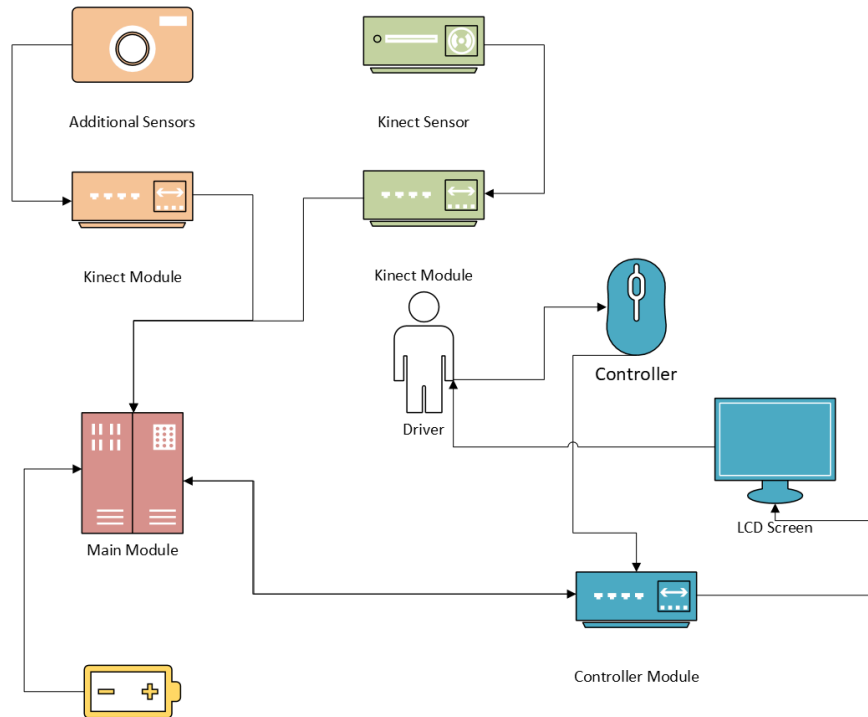
Another is Freenect2, a much more popular and up to date library that actually implements many of the PyKinect features. Most notably it has deep support with lower end hardware and non x86 architectures. With that known we can utilize it and PyKinect to interface with our KinectV2

3.9.2 LCD Programming Research

With the addition of a lot of advanced systems and sensors, at least compared to the original specs of the wheelchair we will be working on for this project, the addition of an LCD screen with a high enough resolution will be paramount to making sure that all the information the user could possibly need is readily available to them while being in a readable format. The LCD program needs to do three things:

1. Indicate the mode that the user is in, automatic or manual, using either text or an icon
2. Display total battery life, with an estimate of how long it will last in minutes and notify the user that battery is at max capacity when being charged.
3. Having the directional instructions of the automatic mode show what direction the chair is moving and how far it will go

Any further additions to the screen's program can be easily done as long as we have these three listed items already implemented, it would just be a matter of either making room on the screen or making sure that computer module that the screen is attached to can read the desired information. The screen will be attached to the controller itself so that the controls and LCD are connected to the same processor. The main computer module will be used mainly for running the chair's automated functions, collecting data from the sensors and processing the controllers inputs to the motors. The addition of the LCD to the controller will allow us to verify that the user's inputs are also successfully going from the controller's module to the main module as any response the main module creates will go right back to the controller and be indicated on the LCD screen. A diagram of the flow of information would look like this:



3.9.2.1: Flow of Information

3.9.3 ESP32-Display Program Research

This LCD, nicknamed the Cheap-Yellow-Display by its surrounding community[14], has a plethora of different programs made by people who have already had their fun testing the board and display together. The setup and configuration of the board is similar to that of any normal ESP32 project, but the repository we will be referencing for this screen already has a detailed guide on how to get it properly working out of the box. Along with having a full guide on its use there is also a detailed list of different programs that will allow us to test the functionality of the screen and give us an easier time modifying the code to display the desired results. The programs include a simple text display program, an image display program, a LED tester, a touch screen tester, and a program to use the ESP32's wireless functionalities. With these functions at our disposal configuring this board to that of our controllers specifications is more than easy, and with the integration of an ESP32 we can expect to save on hardware and design time as we can use the board itself as a computer to work along with the other components we plan to have it connected to.

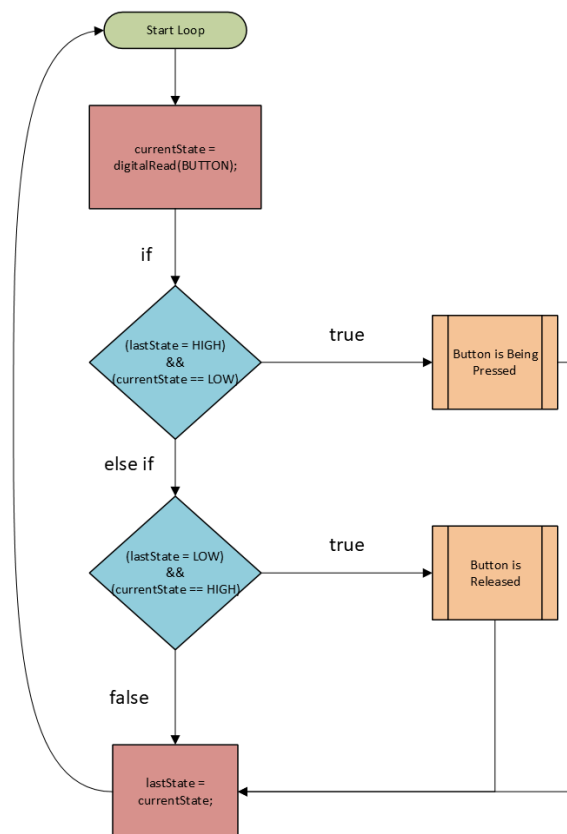
3.9.4 Button Reaction Program Research

The main use for this button is to have a way for the user to put an emergency stop to any automotive tasks the chair is currently conducting. If there

were for some reason an error or a serious emergency, such as an environmental hazard, then the user would need to have a way to cut off all functions beyond their control in order to prevent any further injury. When the button is pressed three things should occur.

1. The chair stops all current commands and resets back into a manual mode
2. The chair immediately uses its brakes to reduce the current velocity of the chair back down to 0 mph
3. An error message is displayed on the LED screen to the user with a noise coming from the buzzer to alert everyone that the emergency stop has been activated.

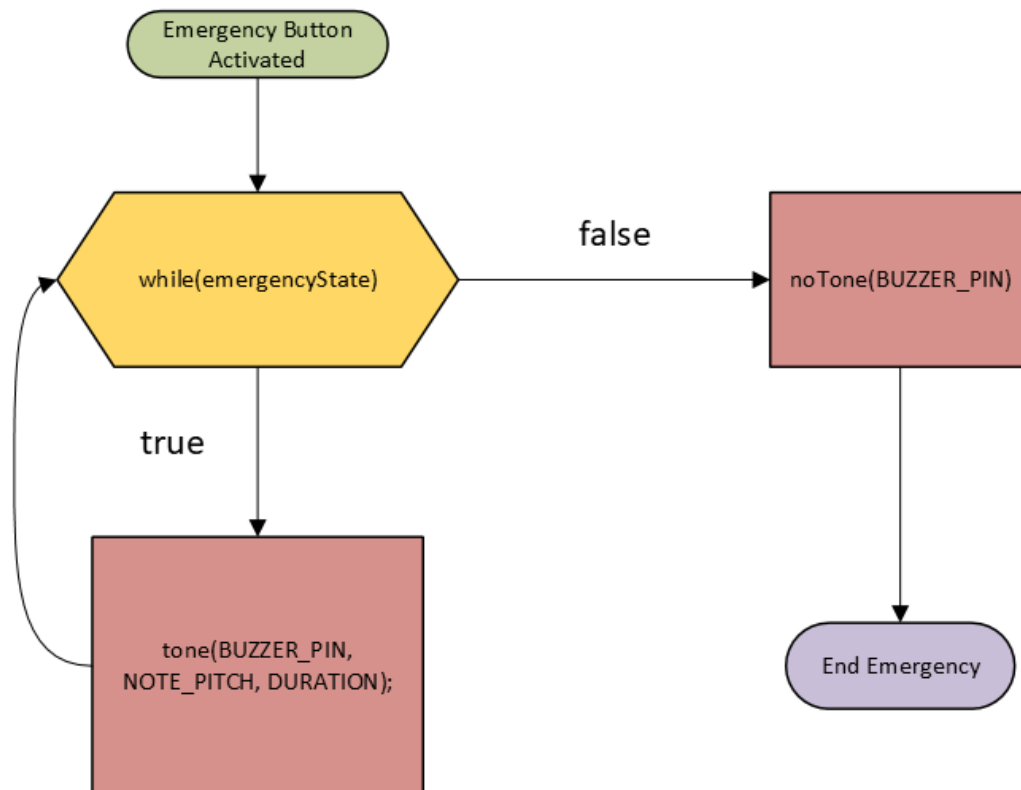
The data from the button can be read using an ESP32 and a simple button with at least 2 pins. Here is an example of a program that will detect whether the button is being pressed or not:



3.9.4.1: Button Program Example

3.9.5 Buzzer Program Research

When the button is pressed and the emergency brakes are set and the buzzer emits a loudly pitched noise to alert any around, including the driver, that the stop has been activated and to watch out as an accident may have occurred. The buzzer therefore has a single function, that being to activate whenever the emergency stop is running, and to keep on buzzing until the wheelchair has successfully reset. An example of the code looks like this:



3.9.5.1: Buzzer Program Example

3.9.6 Collaborative Code Platforms

There are many different collaborative code platforms out there. They mainly allow us to pool our development of software aspects together dynamically without worrying about conflicts or rewriting code. Almost all coding collaboratively uses git as a backend to manage itself, so often we are deciding less on base functionality but more on cloud features and accessibility

3.9.6.1 Git

Git is a tool that allows for easy version control of code and files. It was created by Linux Torvalds, the creator of the Linux kernel, as a way to prevent coding issues after having his BitKeeper license revoked when developing the kernel. Since no other free options existed at the scale he needed, he decided to develop it in April of 2005. Today it is ubiquitous with almost all coders and used in all forms of coding. It is even used with file management due to its efficiency of tracking and version control.

Git has many features, all of which the platforms soon to be discussed contain. Git's main feature is its distributed development workflow, enabled via "branches". Since it tracks and commits changes, it allows for many versions of a codebase to exist and then merge gracefully, and show conflicts that can be easily fixed. Branching can be local to a machine or on the git server and can then be added to the repository's data at any time and easily. Since all changes are tracked, it can be easily rolled back due to issues or tracked and collaborated easily.

A downside of this workflow is that it's declarative and cannot be manipulated easily. This is a blessing as it makes it reliable, but for small changes made without it can ruin a whole code base, fortunately you can overwrite these errors with old commits, but the history will always be a part of the repository.

3.9.6.2 GitHub

GitHub is probably the most popular of the platforms out there. It is based entirely on the git backend but has many GUI implementations with it and makes it quite approachable. It also has quite generous usage limits, meaning any small group of users can work on any project without worrying too much about the limits. It also is a massive website with many different projects and ways to bundle binaries for other users. We won't use those for our cases

The benefits of this platform is the familiarity (most of our team is used to this platform) and makes it easy to review code of others. It has comprehensive issue tracking and integration with Visual Studio Code, which is a boon as it's a very popular coding IDE.

3.9.6.3 GitLab

GitLab is very similar to GitHub. In fact, you would be excused for thinking it was related to Github as it looks and operates very similarly. However its claim to fame is the fact as it's easily self-hostable. So if our team wanted to make sure we were not limited by a cloud platform, we could use this to get around cloud limits this would be our solution. It also has robust Pipeline building support, so we could compile our code and detect issues before we need to run it locally and find the mistake. It's designed to be a full DevOps platform over Github.

3.9.6.4 BitBucket

BitBucket is pretty similar to the other platforms we mentioned prior, but they target themselves to enterprise systems. This is because they are modular and can have issues and pipeline integration with separate programs. This means Bitbucket can focus on reliability and keeping things simple. It has integration with Jira, an issues tracking system that is deeper than the integrated GitLab and GitHub solutions. This integration means it can be used with corporate systems easily and effortlessly.

3.9.7 Programming Languages

With all of these software technologies we need code to actually run the libraries and run the code. Here are a few programming languages we can use.

3.9.7.1 Python

Python, or Python3 in this case, is the programming language of choice for many. It has very simple syntax, plenty of libraries, and a lot of documentation behind it. However it does have a major flaw, this ease of use leads to bloated sizes and relatively poor performance. This is usually not a problem in powerful PCs but on our low powered devices we need to make sure we aren't programming something that can't run or is unnecessarily bloated.

Python is used a lot in pi projects and is documented well there, so for our Raspberry Pi 5, utilizing it will be necessary. Also the KinectV2's libraries exists with easily usable functions so that is a major boon.

It also has native support for LLMs and Vision AI Models and is documented the most in this language, meaning we can utilize the power of those tools in our programs easily.

3.9.7.2 C/C++

These languages are pretty common for embedded systems and are generally pretty fast compared to other higher-level programming languages. It's also the most familiar with our team, and often gives the most control to the developer. The main downside is we won't have access to the major libraries we need, such as those for the Kinect, and will need to either rewrite them or find implementations elsewhere. We get access to much lower-level control, which means we can squeeze more performance out of our computer if we need it, so it may be a good language to utilize if we need simple functions done and don't want the overhead of other languages.

3.9.7.3 Coding Overview

While Python is the language of choice for the high-level processing on the Raspberry Pi, C/C++ is indispensable for the performance-critical tasks on our embedded controllers. This creates a hybrid software environment where

each language is used for what it does best. The ESP32, programmed in C++, will act as a reliable, real-time node for user input, while the Raspberry Pi orchestrates the more complex tasks in Python. This separation ensures that core functions like manual control and emergency stops are handled with maximum speed and efficiency.

3.9.8 Repeatable Software Environments

When repeatedly deploying a software to mission-critical embedded systems reliability and reproducibility are paramount. Containerization of these softwares and their dependencies helps to simplify the deployment and management of our systems. This means that we know when we deploy the container it works the same as how it worked in development. Environmental stability and reliable deployment of said environment ensures our software will run safely.

3.9.8.1 Docker

Docker is a very popular containerization platform that has paved the way for deploying softwares and their libraries and dependencies in self isolated packages called “containers”. These containers are bundles of software that includes everything needed to execute the application in a totally portable and lightweight package. The benefits of this include reproducible environments. When working with embedded systems that depend on reliable performance from the software docker aims to reduce any issues with lacking configurations or dependency files.

Docker also lends to easy updates and software management of our containers. Images or the software version of the container is easily deployed and updated over previously running ones. This means as we improve our self-driving code and algorithms we can push updates easier and worry less about it working on our actual deployed systems. Docker also helps isolate software so for example with different sensors you may have to utilize different libraries or dependencies/configurations. Containerizing all this helps isolate different segments of our sensing system while still allowing for interoperability between them all. If one sensing system were to fail the other containers would still be able to run, increasing safety and reliability of our overall system.

3.9.8.2 Podman

Podman is another popular containerization platform and seen as an alternative to Docker. It runs differently utilizing a daemon-less approach for security reasons. A daemon is a constant running process that manages containerization and deployment and management of said containers. Docker

utilizes a daemon while podman does not. This distinction is important when looking at security of the embedded systems. Daemons run at root levels and are a critical operating component that may compromise the security of a system if unprotected. Podman forgoes this completely and allows for rootless management and containerization of processes. This could reduce power consumption and cpu usage at times due to one less process constantly running.

Podman manages containers by using systemd which is the base service manager for linux operating systems which our computing processor will be running on. This has it running at the user level reducing security risks. While security is a crucial aspect in embedded design there is a balance when picking for security and actual ease of usage. Docker has a daemon but has more support and documentation available than podman does. However podman has the benefit of added security which is integral to embedded system design and operation.

3.10 Digital-to-Analog Converter Research

Digital-to-Analog Converters (DAC) are a fundamental staple of electronic hardware design, used to convert the digital signals commonly generated by computer components and generate a new signal based on that data. These electronic components convert binary values that are processed and transmitted by microcontrollers, FPGAs and digital signal processors and turn them into a continuous voltage output that emulates the output of an analog signal. This gives computers the ability to communicate with real world analog components such as sensors, audio devices, and most importantly motors. When choosing a DAC to integrate in one's circuit designs, accuracy and speed need to be prioritized over everything else in order to ensure proper system behavior, particularly in control systems, real-time signal synthesis, and feedback-driven operations.

In the context of this project, a DAC has to be used in order for the main computer module to communicate with the motors and process information from the sensors into them. The Raspberry Pi model that we will be using has an output voltage of 0-3.3 V from its pins. In order for it to control the speed of the motors the Pi will need a DAC with a high bit input for close accuracy and four analog signals as there are four motors controlling the wheels on the wheelchair. We will be using a chip that would be 16 or 24 bits as the higher bit-width will allow the Pi's digital signals to be closer to the expected analog output.

3.10.1 Analog Devices AD5664

The AD5664 is a high-resolution DAC with four 16-bit outputs from Analog Devices that communicates via an SPI-compatible interface at 50 MHz. It operates with a supply voltage between 2.7V and 5.5V so it is fully compatible with the Pi's 3.3V output. This DAC is ideal for precision control applications such as programmable signal generators, instrumentation, or voltage output modules. It offers excellent performance with a rail-to-rail output range and a settling time

of 10 μ s. With a low glitch energy specifications, this DAC is well-suited for applications requiring smooth and accurate signal generation. It comes in compact TSSOP or MSOP packages, making it ideal for space-constrained designs.

3.10.2 Texas Instruments DAC7614

The DAC7614 is another 16-bit, 4 channel converter that also uses an ISP interface for receiving inputs. The DAC7614 differentiates itself with simultaneous or individual channel updating, which is particularly useful for control systems that require synchronized analog output. It operates at a similar voltage as the AD5664 at around 2.7V to 5.5V so the supply should just as compatible and has a settling time of around 10 μ s. The device communicates over a standard SPI interface at up to 30 MHz and is housed in a SOIC-16 package. This DAC is a strong candidate for multi-channel actuation or industrial signal generation where coordination of analog channels is necessary.

3.10.3 Analog Devices AD5696

The AD5696 is another model DAC provided by Analog Devices, similar to its cousin the AD5664 it comes at a 16-bit resolution with four analog channel outputs. However this DAC does not come in with an ISP interface package, but an I²C communication protocol with speeds up to 3.4 Mbps. Like the DACs mentioned previously, it operates with a voltage supply of 2.7V to 5.5V and features a rapid settling time of just 5 microseconds to ± 0.5 LSB. Power-down modes are also available, making this DAC particularly suitable for battery-powered or low-power embedded systems. Packaged in a TSSOP or LFCSP footprint, the AD5696 offers excellent linearity, low offset error, and stable performance across temperature, making it ideal for portable instruments, sensor calibration, or any precision analog generation over I²C.

3.10.4 Texas Instruments PCM4104

The PCM4104 is the first DAC we found that is compatible with the Raspberry Pi's output and has a higher resolution at 24-bits. This device was originally designed to support high quality audio devices, and while that isn't necessarily something that we are looking for when designing our boards it still proves to be an effective tool at signal conversion. The DAC supports PCM audio input up to 216 kHz and communicates via an SPI-like interface for control, in addition to using standard audio protocols such as I²S. The PCM4104 operates at a supply voltage of 3.3V making it ideal for the Pi's output, however, it does require a separate analog supply, typically 5V, to drive the high-fidelity output stages.

3.10.5 Analog Devices ADAU1978

The ADAU1978 is another 24-bit DAC with 4-channel output and operating specifications that match those of the Pi's digital output. It can use

either I²C and SPI for communicating and, unlike the PCM4104, has a single-supply operation between 3.0V and 3.6V, which allows it to run entirely on a 3.3V system without requiring an additional analog supply. The 24-bit resolution and multi-channel support make it suitable for signal conditioning, analog playback, or even sensor simulation. The ADAU1978 offers excellent range and while it was originally designed for audio applications like the previous chip, its precise analog output characteristics and flexible communication interfaces make it a strong choice for general-purpose high-resolution DAC requirements.

3.10.6 Texas Instruments PCM1690

The last DAC we found for this project is the PCM1690, a 24-bit DAC with 4-channel output and a design similar to the PCM4104. It has an operating supply voltage of 3.3V with control over I²C and SPI but requires a separate analog supply for full-scale voltage output. Although it is marketed primarily as a high-resolution audio DAC, its ability to be configured through SPI or I²C makes it suitable for repurposing in non-audio applications requiring synchronized or multi-channel analog signal generation at 24-bit precision.

3.10.7 DAC Research Comparison

While most people will assume that a higher resolution is automatically superior, that is not necessarily the case when it comes to integrating DACs into your computer systems. The 24-bit DACs, while being cheaper in price and higher in resolution, are designed less for embedded systems and more so for audio systems such as peripherals and consumer grade electronic equipment. They also require a more refined PCB design and due to the higher resolution they require more clocking precision and analog filtering in order to create the most optimized output. Along with all the design constraints generally the DACs need an external analog power supply separate from the digital signal that also supplies energy, this external supply uses up more power so it will require us to further modify the PCB to meet the DAC's needs.

The 16-bit DACs would be the most optimal choice when designing our motor controllers as these devices were specifically designed for controlling analog systems like motors. Analog motors don't actually need a high precision DAC to communicate with them, most motors will only need a resolution of around 12 to 16-bits to receive precise instructions so going with anything higher would only provide a minimal amount of efficiency that does not offset the overall cost to use it. The overall design of the PCB will also be simpler and interfacing with the DAC will be a lot easier as it doesn't require as much preparation and design to make the DAC fit our needs. A 16-bit DAC will also generally use less power but will be slightly more expensive than a 24-bit DAC but that doesn't matter as much when it comes to efficiently creating this project.

Table 3.19: Comparison of DAC Components for System Integration

Name	Resolution	Interface	Supply	Output	Price	Source
AD5664	16-bit	SPI	2.7-5.5V	Rail-to-Rail	\$19.38	mouser.com
DAC7614	16-bit	SPI	2.7-5.5V	0-VREF	\$21.89	mouser.com
AD5696	16-bit	I ² C	2.7-5.5V	0-VREF	\$21.35	digikey.com
PCM4104	24-bit	SPI/I ² C	3.3V	Differential	\$8.74	digikey.com
ADAU1978	24-bit	SPI/I ² C	3.0-3.3V	Audio-Grade	\$8.88	mouser.com
PCM1690	24-bit	SPI/I ² C	3.3V	Differential	\$5.72	digikey.com

3.11 AI Vision Models

Since we are developing a self-driving system, we need to look into AI vision recognition and detection and decide if it's a way we want to implement our self-driving algorithm. We have a limited Processing Unit and have a Kinectv2 for RGB input and IR depthmaps. With this input data we need to decide firstly what AI techniques we want to use and what models.

3.11.1 Kinect Skeletal Data

By default, the KinectV2 generates skeletal data that we can tap into. It gives us the coordinates for a few joints that lets us generate a wireframe skeleton of a pedestrian. This happens on the KinectV2, so it does not take processing power from our computer that would need to process the depthmap or other models. We are given quite a few joints:

Table 3.20: Kinect V2 Skeletal Joint Index Mapping

SpineBase = 1	SpineMid = 2	Neck = 3	Head = 4	ShoulderLeft = 5
ElbowLeft = 6	WristLeft = 7	HandLeft = 8	ShoulderRight = 9	ElbowRight = 10

WristRight = 11	HandRight = 12	HipLeft = 13	KneeLeft = 14	AnkleLeft = 15
FootLeft = 16	HipRight = 17	KneeRight = 18	AnkleRight = 19	FootRight = 20
SpineShoulder = 21	HandTipLeft = 22	ThumbLeft = 23	HandTipRight = 24	ThumbRight = 25

These joints (labeled by how they are reported) are reported every 10 frames from the Kinect. This means at 30fps we get skeletal updates every 333ms, which could be very useful for detecting pedestrians. These are given at coordinates of our depthmap, so without the depthmap it's not as useful as it could be. We can reliably detect up to two skeletons at once, however it is reported 3 or 4 skeletons are possible as well. We also get access to the thumb and wrist of the skeleton, which could be interesting for implementing very rudimentary hand tracking.

However we need to have a distance of 1.5 meters from the Kinect to get reliable skeletal data, as such user data and close persons will not be tracked accurately. This means we will be limited to pedestrians in front of the chair, and we are limited to predicting movements based on the skeletal data. This data was made more for gaming and static tracking purposes, so when the sensor is moving we will need to get creative with our data analysis in the context of this data.

3.11.2 RGB Object Detection

Since we have quite a clear RGB camera, we can use Camera Object detection to classify objects to work with our self-driving algorithm. There are many downsides to this approach, mostly due to the high computational power required for these models typically. We also need to keep in mind that these models may also hallucinate and give problematic info that we will need to control for.

The benefit is that it gives probably the most straightforward data we need for self-driving, and allows us to use any RGB camera rather than being stuck with more exotic sensors. If we use temporal data analysis we can also estimate distance and object velocity without utilizing a depthmap, which is also very helpful for the modularity and versatility of utilizing other sensors.

MobileNetV2-SSD

This model is an object detection model that exists to run in real time on low powered systems. It's quite inaccurate than other models, where it has a 88.6% confidence in objects, which we do not desire. The less confident it is, the more likely it will hallucinate, or give some very incorrect info. However it takes half the compute than the other models we will be looking at. This means we can

get more detection at a faster rate, which is very desirable in a self-driving scenario.

YOLOv3

This model is a step up from the MobileNet model mentioned before. This model has a quite high confidence of 94.6%, which makes it much more viable for use as an object detection model. This comes as a side effect of having two times the compute as the MobileNet, making it much slower in detection. This will be a problem as image detection will be useful only if it can stay as close to real time as possible. This may still be a viable model as it's twice the compute that the tiny MobileNetV2-SSD model, which may be below the tested real-timing required. This still has a sizable gap in its precision, so temporal data analysis must be used to cover its deficits.

YOLOv4

This model is the most precise model from our other models. It has a precision of 98%, which in our cases since we will be feeding it data constantly will be perfect for our uses in getting near flawless object detection. It does come at a large cost however, the compute is 6 times that of YOLOv3. This makes it much more unwieldy in lower power computing and will make it incapable of running in a real time format. That alone means this model will be too computationally intensive for our tasks, and therefore not as helpful.

Object Detection Comparison

Now that we have an understanding of the Object Detection Models, here is a comparison of the models:

Table 3.21: Comparison of Object Detection Models for Real-Time Applications

Feature	MobileNetV2-SD	YOLOv3	YOLOv4
Primary Goal	Speed and Efficiency	Balanced Speed and Accuracy	High Accuracy
Accuracy (Precision)	91.4% (Lowest)	96.5% (Balanced)	99.5% (Highest)
Accuracy (Recall)	88.6% (Lowest)	94.6% (Balanced)	98.0% (Highest)
Inference Speed	0.145 seconds (Fastest)	0.345 seconds (Balanced)	1.91 seconds (Slowest)

Since we need to prioritize lower compute time for our self-driving system, it needs to be as real time as possible. We opt to use the Model that has higher accuracy, whilst also being fairly fast. YOLOv3 seems to fit this requirement and as such we will utilize it for Object Detection.

3.11.3 Semantic Segmentation

Semantic Segmentation is utilizing a model to look for signage and the floor to look for where is drivable and where is not drivable. We need to analyze each pixel of an image and find patterns we can connect to roadway and sidewalk markings. Since we don't need a system to understand road signs for cars and pedestrian signage and markings are quite simple, we can train semantic models for road markings and make an even lower powered model that will fit our needs.

ENet (Efficient Neural Network)

This model is considered a primitive model with very aggressive computational load to come up with inferences. This is still a very small model data wise, but it requires a lot of compute to create even the least confident results, making it a good prototype mode, but one we may pass for others.

FastSCNN

This model is very accurate and used in many self-driving projects. It's an iteration above the ENet model, providing much superior accuracy and large parameter sizing. However it definitely consumes much more compute than ENet to reach such peaks. It excels in high resolution images as ENet takes more low resolution tokens for its inferences. This leads to the higher compute and generally superior quality.

HARD-S

Is a much more advanced design and is quite new in the semantic AI space. This model outperforms all other models whilst being smaller than FastSCNN. It accomplishes this by using a two branch inference architecture that struggles in varied resolution contexts, but with single high resolution contexts it excels. It was designed for edge computation, like built in car systems or even our MCU if it has enough overhead to accommodate.

Table 3.22: Semantic Segmentation Model Metrics for Embedded Deployment

Feature	ENet	FastSCNN	HARD-S
Accuracy (mIoU on Cityscapes)	58.3% (Lowest)	68.6% (Balanced)	72.8% (Highest)
Model Size (Parameters)	0.38 M (Smallest)	1.14 M (Largest)	0.76 M (Balanced)

Computational Cost (GFLOPs)	5.65 G	6.72 G (Highest)	3.24 G (Lowest)
--------------------------------	--------	------------------	-----------------

All of these models are a good example of models made for low compute systems handling semantic tasks, however due to modern advancement and advanced techniques, a clear winner is found. HARD-S's high accuracy and lowest compute cost makes it ideal for our system whilst also being relatively smaller than other systems. Our hope is to train this model on side walk and more generic pedestrian markings to lower compute and raise accuracy, as we can train out car centric recognition.

Chapter 4 Standards and Design Constraints

For any project, we need to keep in mind that we have many limitations. These limitations aren't merely inconveniences but fundamental boundaries that dictate what is possible, efficient, and safe. They can arise from the natural observations of the laws of physics or can be defined by standards established by various organizations and regulatory bodies. Adhering to these constraints is not just about complying to them but also for creating a robust, optimized and well established project. Through understanding and proactively addressing these limitations from the outset, we can prevent costly redesigns, mitigate risks, and deliver a solution that is both functional and reliable in its intended environment.

4.1 Industry Standard Constraints

Since we are utilizing off the shelf parts and components that must adhere to industry standards, we need to be conscious of the limitations these tools/parts have. Below are explorations of the specifications and constraints that we must follow and keep in mind while we design and assemble our project.

4.1.1 Universal Serial Bus (USB) 3.0 Standard

The USB Standard has existed since 1996, backed by many large industry giants in an effort to remove many connectors doing different and similar things. It has slowly taken over as the port and cable that connects most external peripherals, simplifying port selection and allows for interchangeable cables. In its infancy many other connectors existed that did what USB promised to do, but better. However over time the standard took over, mainly from its robust ability to deliver power after being fixed by the USB 1.1 Specification. Soon it overtook its competitors and prevented itself from becoming just another standard. It was so common to fail as a standard back then, that a popular comic artist poked fun at the phenomenon showing the usual downfall of standards with USB's mission. Since we are utilizing sensors and computers that utilize the usb protocol to communicate and get power, we need to be able to confirm the limitations that may pose.



Figure 4.1.1.1 : xkcd Standards Comic

USB 3.0 is backwards compatible with the USB 2.0 specification, using the same capability as the 2.0 standard if using a 2.0 port or cable. USB 2.0 only has 480Mbps data rate, which is quite fast but not quite fast enough for our uses. This is because the USB 2.0 standard uses 4 pins, whereas USB 3.0 uses 9 pins. USB 2.0 has a pinout of 2 data pins, one ground pin, and 1 power pin. If you can electrically wire these 4 pins, then you have a working USB connection. This is in contrast to the USB 3.0, which has more than double the pins. USB 3.0 has the same 4 pins electrically and locationally with 5 pins located a bit forward from the 4 legacy pins. These pins have 4 more data pins allowing for Superspeed transmission and then an extra ground pin specifically for the data pins. These extra pins provide a lot of complexity, but are required as the previous 4 pins are just not enough for sending 5Gbps of data.

USB 3.0 also introduced a new connector type. USB 2.0 has 3 major connector types; Type A which is the familiar rectangular port primarily for host systems, Type B which was made for client systems like printers, and Mini/Micro are connectors made for slim portable client systems such as music players and phones. USB 3.0 introduced variants of the 2.0 connectors that were backwards compatible with the cables for their respective connectors. USB 3.0 also introduced the USB Type-C connector. The Type-C connector is reversible, has a possible 21 pins, and was able to distinguish between host and client systems on both sides of the cables. This made the Type C connector incredibly versatile and is poised to replace many of the older connectors currently in use in the computing world. However since it is full of standards and capabilities itself, all of which are not standard on all cables/ports despite them being standards, it makes it difficult to know what cable or port is capable of.

Another important metric is power delivery. The USB 2.0 Spec can deliver 5V at 500mA which is about 2.5W. Where the USB 3.0 spec by default can deliver 5V at 900mA which is about 4.5W. While we don't plan to deliver power via the default protocol, this is important to note as even though this is standard, many implementations do not follow this spec. Many ports over or under deliver the current limits, which makes things quite annoying if you're dealing with a system that doesn't adhere closely with the spec.

Another standard implemented with the USB 3.0 Spec was also USB Power Delivery (USB-PD). USB-PD uses the data pins to allow communication between client and host systems for Power Delivery. With this communication the client can "ask" the host for higher voltages and currents to allow for more power down the power pins. The client asks for "profiles", with a maximum current of 3 Amps and voltages ranging from 5V to 20V. For example a laptop requires 45 watts of charging power as the host charger for 15V at 3 Amps or 20V at 1.5Amps out of the host charger. These profiles are variable, and not all chargers will have all the profiles so it is important to keep the profiles in mind when looking for a compatible system. A unique break to the 3 Amp maximum on the profiles is the 20V at 5 Amp profile meant for 100 watt requests. These are

commonly found in Macbooks and high-end systems.

Since we have so many considerations with USB, we need to make sure we keep it in consideration when deciding components and also finding out what needs to be implemented to actually utilize these technologies properly. We also are required to use if for many specific sensors as USB has become ubiquitous with the modern world

4.1.2 Bluetooth Standard

Bluetooth is a globally recognized form of a wireless communication standard that is managed by the Bluetooth Special Interest Group (SIG) with it being formally recognized as a standard in **IEEE 802.15.1**. This document defines the performance, interoperability, security, and profiles for short wave communication using Bluetooth.

Bluetooth functionality is organized by using profiles, which define specific application, behaviors, and protocols. Some examples of profiles that are commonly used with Bluetooth are Hands-Free Profile (HFP), Message Access Profile (MAP), Health Device Profile (HDP), and audio/video streaming profile. These profiles allow data to allow developers to easily set up an environment where the user can connect multiple devices using Bluetooth and automatically have them communicate using the same profile, such as a cell-phone connecting to a car using the HFP. These profiles will ensure compatibility across all devices as long as they are using the proper profile to communicate with each specified device.

There are also a multitude of Bluetooth versions with each having their own specific features that came with each update. They are all backwards compatible, meaning that Bluetooth devices operating with a newer version can communicate with any device running an older version, however the features of all devices connected are limited by the weakest node, in this case the oldest version. For instance, devices running version 5.2 or newer will have the LE Audio feature that uses a low power audio framework for functions such as multi-streaming and hearing-aid support. Version 6.0 introduces channel sounding and round-trip timing technologies which are useful for location-based services and proximity-triggered interactions. These features can be especially useful if implemented correctly with our sensors to expand upon the wireless features they provide.

Table 4.1: Evolution of Bluetooth Versions and Feature Enhancements

Version	Classic Data	LE Data	Range	Features
4.0	3 Mbps	1 Mbps	60 m	Bluetooth Low Energy
5.0	3 Mbps	2 Mbps	240 m	Increased Range & Speed

5.1	3 Mbps	2 Mbps	240 m	Enables direction-finding
5.2	3 Mbps	2 Mbps	240 m	Introduced LE Audio with LC3
5.3	3 Mbps	2 Mbps	240 m	Improved power management
5.4	3 Mbps	2 Mbps	240 m	Adds encrypted advertising
6.0	3 Mbps	3 Mbps	300 m	Round-Trip Timing
6.1	3 Mbps	3 Mbps	300 m	Privacy and power efficiency

4.1.3 LCD Screen Standard

Liquid Crystal Displays (LCDs) are widely used in embedded systems, mobile devices, instrumentation, and consumer electronics. In order to ensure their safe, reliable, and consistent performance, LCD modules are designed according to various international standards. These standards help guide design, manufacturing, testing, and ergonomic usability. For the purposes of this project, two relevant standard families were considered: **IEC 61747** and **ISO 9241-3xx**.

IEC 61747 – Liquid Crystal Display Devices

The IEC 61747 series is an international set of standards developed by the **International Electrotechnical Commission (IEC)** specifically for liquid crystal display devices. It defines performance characteristics, reliability testing, and construction guidelines for LCDs. This standard applies to a broad range of LCD types, including transmissive TFT displays commonly found in microcontroller-based projects.

Some of the key aspects addressed in the IEC 61747 standard include:

- **Optical Characteristics:** Brightness (luminance), contrast ratio, color uniformity, and viewing angles.
- **Electrical Performance:** Threshold voltages, response times, and interface signal timing.
- **Mechanical Construction:** Module thickness, display area dimensions, and component tolerances.
- **Environmental and Reliability Testing:** LCDs must undergo tests such as high/low temperature operation, thermal cycling, humidity exposure, and mechanical vibration/shock. These ensure durability in industrial or mobile use-cases.

Although many consumer-grade LCD modules may not be explicitly certified under IEC 61747, their datasheets often report performance values consistent with the standard's categories. For instance, typical display

parameters such as 250–400 cd/m² luminance, >500:1 contrast ratio, and >60° viewing angles align with the classifications laid out in the IEC documentation. Engineering teams can use this standard to assess display quality, validate specifications, and guide product selection.

ISO 9241-300 Series – Visual Ergonomics for Displays

The ISO 9241-300 series is a family of standards issued by the **International Organization for Standardization (ISO)** that focuses on visual display quality and ergonomic considerations. This standard is not limited to desktop monitors but also applies to smaller LCD modules used in embedded displays and handheld systems.

The following areas are addressed in this series:

- **Visual Comfort and Legibility:** Standards define acceptable limits for brightness, color accuracy, pixel sharpness, and reflectivity to reduce eye strain and improve readability.
- **Flicker and Refresh Rate:** LCDs must minimize flicker under normal use conditions to prevent user discomfort. A refresh rate of ≥60 Hz is generally acceptable.
- **Dead/Defective Pixels:** Standards such as ISO 9241-303 and 307 specify pixel fault classes (e.g., Class 1 allows a minimal number of stuck or dead pixels).
- **Surface Treatment:** Anti-glare coatings and reflective properties are also considered to ensure displays remain readable in varied lighting environments.

For example, an LCD used in outdoor or portable applications should have anti-reflective treatment and a sufficiently high contrast ratio (>250:1 minimum) to meet these requirements. While ISO certification is more common in medical and professional devices, using ISO 9241 as a design benchmark helps ensure good visual usability in consumer or industrial applications.

In this project, although no display is formally certified to these standards, selections were made based on compliance with key ISO and IEC display performance metrics, as provided in manufacturer datasheets and empirical evaluations.

4.1.4 Button Standards

Mechanical push-button switches are a universal and fundamental component found in almost every single piece of consumer grade electronics since the 20th century. These switches, like many other components, are subject to international standards of safety and design requirements and so we must use these standards to make sure that we are in compliance with designing our

project. The two primary standards that we will be using in regards to the button switches used in this project are **IEC 61058-1**, a global standard for international consumer grade electronic switches, and **UL 61058-1**, the North American standard for switches that will be important for creating this project in an American university.

IEC 61058-1 – Switches for Appliances

The first of these stands is provided by the International Electrotechnical Commission (IEC) named **IEC 61058-1: Switches for appliances – Part 1: General requirements**. This standard provides comprehensive guidelines for manually operated mechanical switches like the ones that were researched and will be implemented for this project. It applies to low current and low voltage switches that are at a voltage below 480V and a current below 63A. This covers the type of buttons that we want to implement and follows our designs as we are trying to create panel-mounted and tactile buttons found on our controller and other small consumer grade electronics.

Key aspects of this standard include:

- **Electrical Ratings:** The standard defines the maximum allowable voltage and current allowed for consumer grade switches. Most switches are rated for a current of around 3A and a maximum voltage of up to 125V which is more than what will be needed for our designs.
- **Contact Performance:** This defines the requirements for electrical performance such as contact reliability, bounce time, and insulation resistance. This is to ensure that the switch provides consistent performance over long periods of time and use and to ensure that the quality of the electronics do not decay.
- **Mechanical Endurance:** The standard tries to establish a minimum lifespan for the mechanical aspects of the design, often ranging from 10,000 to over 50,000 cycles (uses) depending on the switch class. It also sets standards for measurable physical factors regarding the switch such as actuation force, travel distance, and structural integrity of the housing and actuator.
- **Environmental Testing:** Switches are also required to have some sort of resistance to environmental factors that could damage the switch or reduce performance such as temperature cycling, humidity exposure, and vibration. For example, switches with a “T85” rating are rated for operation up to 85°C. Dust and moisture protection levels are classified using IP codes in accordance with another standard **IEC 60529**.
- **Safety Requirements:** This is in regards to strength testing, flame resistance, and insulation to prevent the switches from hurting people or damaging other components.
- **Marking and Identification:** This is just to ensure that basic information is marked onto the device such as switch type, and manufacturer identification, which ensures traceability and correct component selection.

IEC 61058-1 is frequently used in conjunction with additional parts of the same series that specify requirements for particular types of switches, such as push-buttons, toggles, or rocker switches. Using components that meet the IEC standards for safety and performance ensure that we are able to provide functionality to consumers while making sure that their safety is a top priority.

UL 61058-1 – North American Switch Safety Standard

Published by Underwriters Laboratories (UL), the **UL 61058-1** is the North American testing standard that follows the standards set by **IEC 61058-1**. It was designed to make sure that switches used in U.S. and Canadian consumer markets meet the country's domestic safety expectations. **UL 61058-1** includes all the requirements of **IEC 61058-1**, with slight variations to account for local laws.

Push-button switches that carry a UL Recognized or Listed mark are validated for use in commercial environments like the one our project will be tested in. UL compliance is not mandatory for prototyping a project or for academic use but using switches with known UL will ensure the the devices will operate safely under load and and prevent any further damage from happening to the prototype if it does still fail.

Common parameters that UL tests for are:

- Terminal strength and solder joint reliability
- Resistance to fire and flame propagation
- Endurance and reliability under repeated use

Choosing a button switch that conforms to the testing standards of UL 61058-1 saves us the time of having to test the product for basic functionality and ensures basic functional reliability during prototyping and testing. For this project, push-button switches were selected based on their alignment with IEC and UL performance standards, particularly in terms of actuation force, mechanical life cycle, contact bounce behavior, and safety under typical low-voltage conditions. Though formal third-party certification is not required for this application, datasheets were reviewed for parameters that indicate general compliance, including:

- Rated current and voltage
- Operating temperature range (T70–T85 typical)
- IP rating (e.g., IP40 or higher)
- Actuation cycles (typically $\geq 20,000$ cycles)
- Dielectric and insulation resistance

By referencing these standards during component selection, we can ensure that the functions it provides will be reliable and efficient over a long course of time and we won't have to worry about it endangering the rest of the

project. The inclusion of these standards also aligns with good engineering practice and enhances the documentation required for later design validation or system review.

4.2 Practical Constraints

We are developing this project in the real world, and as such we need to be aware of challenges and problems that will arise from being in an imperfect system. Whether that is from human error, physics, or other constraints.

4.2.1 Time

The main constraint that we have with this project is time, a common constraint for all projects. This is quite notable here since most Senior Design projects are done in the Fall and Spring semesters, where our project has the Summer and Fall cadence for the project. The Summer is a time constrained semester compared to the others, as such a higher average load is required in this time period. Fortunately Spring is a standard semester so the Building phase of our project is not on the fast track and hopefully problems can be avoided. Since time is constrained, we need to prioritize effort and work in different places and find efficient ways to accomplish this project. This may come at the cost of a perfect project and we must prioritize the Minimal Viable Product (MVP). The MVP is a point in our project we must accomplish before we can push for more ambitious goals.

It would be best to push for this MVP as soon as possible, and to focus on the feasibility of the stretch goals whilst developing the MVP. In our case it's a wheelchair that can drive forward automatically and can stop when an obstacle is near. This plain, single sentence description is what keeps the scope from blowing up and consuming time that could be used to actually deliver what we need. Feature Creep is a phenomenon in which many industries fall victim to where features are added on where core features are left to be neglected. Rejecting erroneous features can be the difference from going into development hell to actual delivery. I think by keeping this principle in focus we can ensure we prevent over commitment.

Another consideration is that we need time to correct any errors or account for unforeseen setbacks. Since this isn't something we can empirically account for, as problems don't give you a time table, we have two ways we can account for this. Preventative measures, such as heavy peer review and check-ins help prevent errors and keep issues from being ignored or unseen by a single individual. While this does add time overhead, we think preventative measures mean much more than simply dealing with the consequences. Another way we account for setbacks will be for over-provisioning time. We can finish tasks as early as we can manage and then provision time to tasks that may have issues. For example, ordering PCBs in the current political landscape leads to long shipping times and hold-ups that we can't control. So designing and

ordering them early will help mitigate the problems that will arise from slow shipping.

Lastly, is the human element. We need to make sure we don't end up creating an incongruence with how many man-hours we have. If we put everything on one person, it can be both unfair and also bottlenecked by that person. "Do what you say you will do" is a great example of this. While we may assign tasks to others, if somebody doesn't do what they say they will do then tasks will be incomplete and it'll slow down development

4.2.2 Economic

Probably the constraint that is most obvious in a project is the economics of the project. The components to assemble anything have a price tag, and you can dream up anything with a budget that is unlimited in nature. Of course we don't have that luxury in this case.

Since we didn't get a sponsor, we ended up on a budget that we all can pool together. We are fortunate to have means to afford this project, but we still wanted to limit the budget to be as cost effective as possible. This was a massive challenge when we originally generated ideas for our Senior Design. We ended up doing this project mainly due to the very generous donation from Dr. Wei of an electric wheelchair. This was by far the most expensive portion and difficult assembly piece of the project. The motors, construction, and research into making a wheelchair in the first place is immense. So this boon was definitely what we needed to actually put our idea into reality. It did however open up some other challenges, as the wheelchair we received had dead batteries, so we needed to budget and consider the compromises due to this. In fact, due to economic efficiency, we decided to use lower capacity batteries that are normally used in this chair. We calculated that this would not impact the MVP of our setup, especially as the previous batteries had very large capacities. We also used car batteries as prototype and debug reasons, which is quite the macgyver but working within our constraints.

Another consideration was to take inventory of our team's components we already had. Many of us had components that we could tap into to construct our project. This has significant savings. A good example is the Kinect v2 Sensor, the main sensor for our project. We decided to use it mainly due to the fact we had it on hand already, but also if we needed to get a new Kinect sensor, which are quite cost efficient on the used marketplace. If we have a larger budget, we would maybe want to get real Lidar equipment as that would be more accurate and inline with technology used today. However the kinect provides a depth map that is "good enough" for our purposes that we made the consideration of it.

Outside of our budget we also wanted to open source this project if possible. Since we are not beholden to a sponsor, we wanted to see if we could use the control we have of the project to be usable outside our project.

Providing open source solutions like this can make the time and economics of recreating our work lower for others, which we want. Lowering the burden of cost for people who may need this technology is what we strive for.

This was also a major consideration for our project. We want to find preferably cheap, off the shelf components so that other users can replicate what we have done, and actually upgrade their chairs to become self driving. Converting these older chairs can make them seem a lot more modern, using used parts (like the Kinect and other components) means we can keep electronic waste out of the landfill.

4.2.3 Environmental

Since we want to make a wheelchair that is capable of being outside, going up large inclines, or overall being used in many environments. We don't need to be concerned with extreme environments, it is unlikely this thing will be on the moon or anywhere exotic. However due to Florida weather (the location we will be mainly testing it in) we want to develop it with humidity and heat in mind. We need to make sure our sensors and MCU don't overheat or become dysfunctional. So we need to make sure to put our heat generating components in places with appropriate airflow. We also need to worry about aesthetics with this regard, as if we try to box everything up we may have issues cooling such systems.

We also need to worry about other environmental factors. While we will be manufacturing PCBs, we need to make sure the bare boards are not open to the elements as that is considered bad design and could cause issues. For humidity, boxed in electronics won't last long as moisture will cause damage and corrosion.

4.2.4 Ethical

While there is nothing unethical with designing a mobility apparatus for the benefit of the movement impaired, the biggest ethical concern we face with this project is the means of sourcing all the components and resources. While we can not ensure that every piece of circuitry and every strand of filament is taken directly from morally sound businesses, we can do our best to avoid such a dilemma by purchasing products from companies stationed in countries that have laws that prevent morally bankrupt people from exploiting people, animals and the Earth. There are many nations that still have a lack of human/workers rights, these countries allow their people to be abused by private firms and sometimes by the government directly. If we buy products from nations that are known to implement slave labor or harsh wages to meet the economic demands of resource extraction and product manufacturing we would be supporting these practices by providing the finances to keep the companies in business. The environmental factors our project were already discussed but we must take into account the fact that even though our product is environmentally friendly, the components used in the manufacturing of the chair should be sourced from businesses that practice sustainable and environmentally friendly standards that aim to reduce their damage on the environment and avoid damaging the wildlife of the region they operate in.

To make sure that we do our best to follow the ethical guidelines we listed above we will be investigating both the retailers we purchase our goods from and the companies that manufacture their products. Doing so will ensure that the sourcing of the components and the manufacturing of the chair are as ethical as they could possibly be. Buying from trusted suppliers with a good track-record of quality products and ethically sourced goods will also help ensure our standards are being met. If possible sourcing everything from local retailers will also help morally justify our purchases as we are going to be helping businesses that will need financing more than larger corporations.

4.2.5 Adaptation of Current Wheelchair

Modifications to the original wheelchairs electrical systems and PCB design are inherent constraints that we have to consider. If possible, inheriting and utilizing the old systems as much as possible would simplify design and increase electrical safety. The least invasive plan of attack we have is integrating with the 6 pin male connector on the wheelchairs PCB board. This will demand simulating the original joysticks outputs on the 6 wires properly so the wheelchair finds no issue with the signals being given. This will be the responsibility of our main brain that does the self computing algorithm which will most likely be the Raspberry PI. Pursuing this design philosophy will allow us to extend the sensing systems and automatic driving capabilities of our wheelchair while maintaining original functionality of its internal electronic and power systems.

4.3 ADA Constraints/Compliance

When modifying the designs of our wheelchair it is imperative we take into consideration the realities of the environments our users will interact with. Designing the wheelchair without these notions could actually reduce the effectiveness of our design and affect the users accessibility. An example being if our sensor housing is too big we could prevent the user from passing through standard sized doors. Integrating our design seamlessly is our main goal. We want to only improve and add to the functionality of the wheelchair so that it can navigate safely and comfortably.

4.3.1 ADA Physical Dimension Constraints

ADA(American with Disabilities Act) Establishes standards for equal opportunities for persons with disabilities. It also enacts accessible design constraints to public and commercial buildings. As a result our wheelchair and modifications must stay in line with the physical dimension requirements that ADA enacts.

Any additions we add to our wheelchair must fall within the ranges given to facilities by ADA(American with Disabilities Act)standards. Any alterations we make to the wheelchair shouldn't increase its width or size so much that it wouldn't be able to act as a regularly expected of an electric wheelchair in ADA compliant facilities. Protecting the rights of those that use the SmartStride is

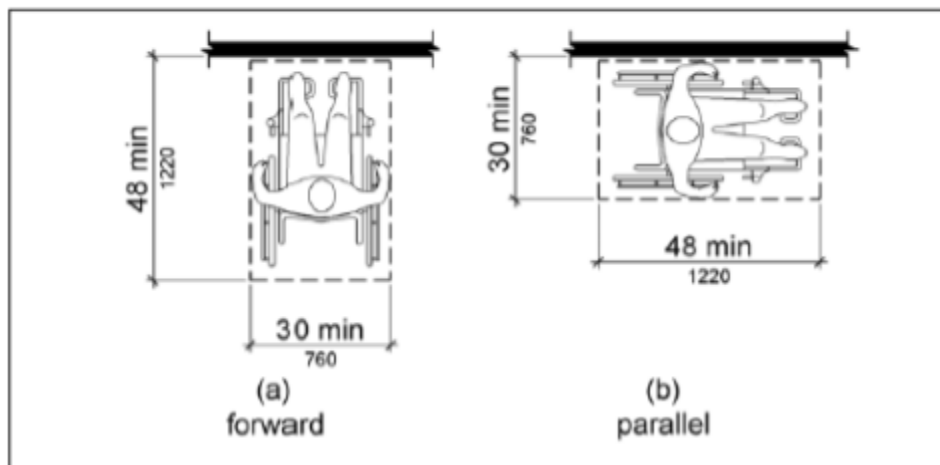
integral to comfortable use and accessibility that aligns with the policy and safety standards. This also helps ensure that SmartStride will continue to be able to act and operate as an normally expected electric wheelchair.

It's safest to assume that most facilities will conform with the minimum required design specifications. This allows our design and wheelchair to apply to as many environments as possible and succeed totally in areas that exceed such specifications.

ADA establishes key dimensional constraints that we must abide by. These include

1. **Clear Floor or Ground Space:** Clear floor spaces for forward or parallel approach to areas in buildings. These dimensions include a 30 inch minimum width and 48 inch length minimum.

Figure 305.5

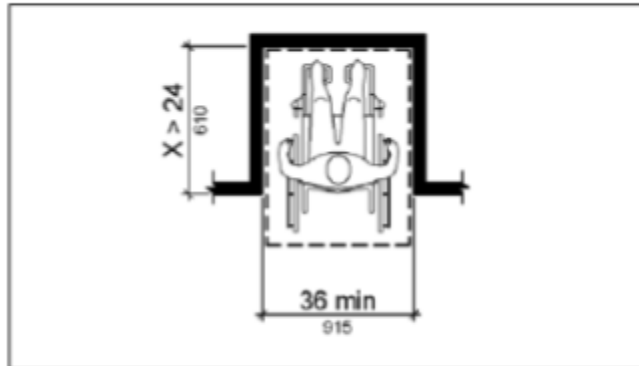


Position of Clear Floor or Ground Space

Figure 4.3.1.1: ADA Floor Space Diagram

2. **Maneuvering Clearance** : The SmartStride needs to be capable of maneuvering in the ADA mandated 60 inches wide minimum with a depth exceeding 15 inches.

Figure 305.7.1

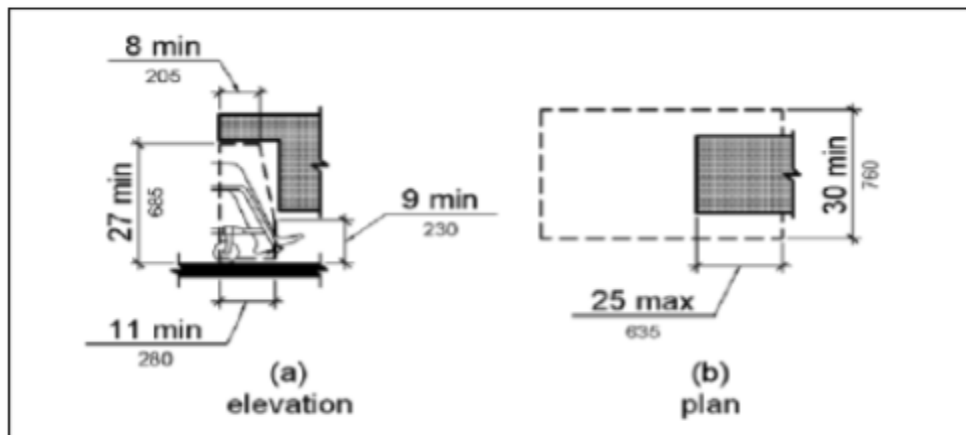


Maneuvering Clearance in an Alcove, Forward Approach

Figure 4.3.1.2: ADA Clearance Chart

3. **Knee Clearance**: ADA requirements deem necessary clearance for knees when wheelchairs approach specific elements designed for them. There is minimal knee widened clearance of 30 inches

Figure 306.3



Knee Clearance

Figure 4.3.1.3: ADA Knee Clearance Chart

4.3.2 ADA Visual Constraints

While ADA regulations primarily focus on the actual environment a disabled person would interact with, taking into consideration its design philosophy is crucial to SmartStrides accessibility in all form factors. The “ADA Requirements: Effective Communication” is where we will pull visual and auditory designs for the modifications we aim to make with our SmartStride. We cannot make assumptions about what type of impairments a person using the SmartStride may have so we need to cover as many bases as reasonably possible. “The key to communicating effectively is to consider the nature, length, complexity, and context of the communication and the person’s normal method(s) of communication” (“ADA Requirements”).

1. **Visual Cues/Clarity:** Taking into account the ADA philosophy our symbols and control methods on the wheel chair should have clear text. With a good bright color and a background that allows for high contrast. Big Fonts and short clear text is integral to the design. Color can also not be the sole indicator for certain systems or emergency functions.
2. **Auditory Cues/Clarity:** Verbal announcements like the engagement of automated driving as well as potentially a regular auditory hum or tone to indicate that it is currently operating in self driving mode would be a good idea. Having clearly distinct noises like beeps and chimes relating to specific functions will also better allow visually impaired users to feel comfortable and in control. Warnings sounds for object detection during automotive driving are also useful to brace the user for a sudden change in direction or a maneuver different than just going forward. Battery status, system warnings and core system announcements should fall in line with these requirements.

4.4 Personal & Mission Constraints

Since this is a project that has the ability to do good, our team decided to make a few personal constraints we wanted to adhere to, to allow for good to be created from our contribution to the world. We wanted to accomplish the following with this project:

Modularity

We wanted to apply this work to as many different types of electric wheelchairs as possible. Rather than designing a completely new chair from the ground up, our goal is to create a modular system that can be retrofitted onto existing, common wheelchair models. This approach ensures that our solution can be implemented as an upgrade, reducing waste and making it accessible to individuals who already own a basic electric wheelchair. This constraint forces us to think in terms of a universal “kit” rather than a single, proprietary product.

Affordability

A core motivation for this project is to address the high cost of advanced mobility aids. From the outset, we are imposing a strict budget constraint on ourselves. Every component selection and design decision will be weighed against its cost-effectiveness. Our mission is to prove that advanced features like obstacle avoidance and autonomous navigation do not have to be prohibitively expensive. We aim to keep the total cost of our augmentation system significantly lower than commercially available smart wheelchairs, making it a viable option for lower-income individuals and communities.

User-Centric Design

The system must be intuitive and genuinely helpful for a user with significant physical limitations. This means we are constraining ourselves to design solutions that are simple to operate and do not require complex technical knowledge. Features will be designed with direct input from potential users in mind, focusing on practical benefits like an easily accessible manual override, clear status indicators (e.g., battery life), and straightforward controls. The technology should empower the user, not overwhelm them.

Safety and Reliability

Given that this system will be responsible for a person's mobility, safety is a non-negotiable constraint. We are committed to using reliable components and implementing robust software logic. This includes building in multiple fail-safes, such as edge detection to prevent falls and an immediate stop function if the system encounters an unexpected obstacle or error. Every goal, from basic "cruise control" to advanced navigation, must be built on a foundation of safe and predictable behavior.

Chapter 5 ChatGPT and other Similar Platforms

Within the last handful of years, there has been a big change in the world thanks to AI chatbots being released to the public. Almost every industry has been affected in some way. Manufacturing, cybersecurity, education, energy, and software have been majorly changed by AI. This tool has been used in every possible way to make things better. It has made production lines more efficient, coding projects ten times easier and less time consuming. AI has also helped improve traffic by analyzing patterns and adjusting traffic light timing to reduce congestion. It has been used a lot in higher educational research and in article publications ranging all over the place from physics to music theory. The main ai chatbot used is ChatGPT, with other platforms like Google's Gemini and Poe following not too far off.

Around November of 2022 was when the world saw the first AI chatbot of a decent caliber, ChatGPT. Developed by OpenAI, this tool took the world by surprise. At first it took a moment for the chatbot program to become popular, but as soon as it gained traction, it took off. As of right now, ChatGPT sees around 300 million users a week and is still quickly going. 2024 was a busy year for OpenAI as it released its newest version, GPT-4o and partnered with the company Apple.

Pros

This tool offers so many benefits to the user. For one, ChatGPT has access to the largest collection of information in the world, the internet. Almost anything imaginable the user wants to ask a question about, ChatGPT can answer. All sorts of prompts can be answered: "Who was the most ruthless ruler of the roman empire?", "How do you calculate the capacitance needed to supply a surge current for 1 ms?", "Tell me about some cutting edge research that might be nominated for a Nobel". Another great benefit of this program is that the user doesn't have to search the internet thoroughly anymore to get their questions answered. In the past, it was crazy the amount of websites one person would have to click through just to find what they were looking for. Could've been 2-whole pages worth of links alone till you found your answer. This type of convenience that ChatGPT offers is making a lot of people turn away from the internet search bar. The sheer convenience means less time searching for your answer which also means more productivity which brings us to our next point

What may be 8 hours to work on turns into 3 hours with ChatGPT and other similar platforms. Students are able to type up research papers quicker than ever before. No need to go to a library or look through endless scholarly articles to find and cite what information you are looking for. Asking an LLM what type of information you are looking for and for it to only cite valid sources makes the paper writing process 10 times quicker.

Cons

While there are many benefits to an LLM, there are also many cons. One of the main issues is people taking advantage of the platform and relying on it too heavily. This can lead to people having subpar problem solving skills. Depending on what age people start heavily relying on LLM, it may cause even further issues with cognitive functionality. According to an article published by AIM, MIT Media Lab did a study and found a correlation that might suggest that LLM like ChatGPT are leading to our cognitive decline. In the study done by MIT, they split their test group into three different sections. The first section was allowed to use an LLM. The second section could only use a general search engine like google, and finally the third group could only use their brains and no external tools. Each of these three groups were expected to write an essay on the same topic while an EEG was hooked up to each of the participants. The study found that people who used the LLM's had slightly lower neural activity than the other two groups. The group who had the strongest neural connectivity was the group who solely had to rely on their brains to get the essay done. A surprising statistic was also grabbed from this study: 83.3% of the participants who used an LLM for their essays couldn't even recall quotes from their own essay. The other two groups only had about a tenth of the whole group have a hard time remembering their own quotes. There is quite the notable difference between the LLM group and the other two.

A big reason that there is a correlation that can be measured between LLM users and lower brain activity is that the LLM flips through ideas itself and finds out the best one for the user so they don't really have to do any thinking for themselves. The groups who went without an LLM had to figure out how to piece their whole essay together. What were their main ideas going to be about? How many arguments were they going to put down for each? Is there a weakness with my argument? If there is a weakness, how can I fill in the gaps to make my argument stronger? All of these questions the non-LLM-using groups had to think about without any outside help. While one of the two groups still had a search engine, that group still had to think about what they were going to ask for and figure out whether the information would be valid enough to help their argument and or get their main points across.

Another big argument against LLMs is the higher percentage of people relying on it at the academic level. ChatGPT and other similar tools are a really great source of information for kids who are having a hard time understanding something directly from a class. For example, if you are having a hard time with understanding a double integral in your calculus class, you could ask ChatGPT to help you understand it and dumb it down for you as well as provide examples. This is a great resource in that case, but if someone is using ChatGPT to help them do their homework or do example problems while studying for an exam, this could hinder the person's actual education for that class. If administrators don't place caps on LLM or provide rules for using them, there could very likely be a definite drop off in the quality of education for students. Times are changing. More people than not right now are using an LLM for school in some type of way whether it is ethical or not. This heavy reliance on LLMs will definitely hurt this

generation plus the future generations if something is not done about it. The plot of the movie Idiocracy is getting more and more realistic in this world everyday.

Using Google Gemini vs ChatGPT

We also used Gemini's Deep Research function to find and corroborate many different sources, since Gemini uses Google's Search to find information its data is usually much more reliable than other popular LLMs. The searches it conducted and information it presented tended to fall in line with what we would find personally searching the web and utilizing Google Search and other search engines. It also helped sort and present the sources it used clearly for proper citing of material and easy comparison of the actual article to what the LLM is telling us. This led to less hallucination of information which ChatGPT can tend to do.

Using LLMs for our project

There are many pros and cons of using LLMs for our project. We went ahead and took the initiative to bounce some questions off of ChatGPT for our research portion of the project to get a good idea of what we need to do. While some of these prompts ChatGPT gave us helped us out, there were a few prompts that it gave us that could've hurt us if we didn't know better or do more research. Below, we will show some ways ChatGPT has both helped and hindered us in our research for this project.

Example #1

Our first example of this is when we were trying to figure out if voltage regulators were needed for our components (mainly the Kinect V2) given our Sealed-Lead Acid batteries we picked. We knew there were built-in drivers in the motors and controller but didn't know if we still needed to hook up regulators to protect our components. The question we asked ChatGPT was: **"We have a 24V wheelchair with two sealed lead acid batteries supplying. We plan on putting in a 12 V Kinect V2 sensor on it as well as other components that will probably range in voltage. Will we need voltage regulators?"** We got a pretty big response from ChatGPT (***Check Appendix ## for the entire prompt***). ChatGPT said yes, we will definitely need voltage regulators. For one, the power source is different from the loads. The batteries together supply 24V while the load devices will range in voltage so we need something like a regulator to step down the voltage. Also with this question, it told us what type of regulator we would need to use and also made a table gauging what voltage is needed for each component. Lastly in the LLM's response, it gives us recommendations regarding the certain model regulator to use depending on amount of current and also to check the Kinect V2 specs to make sure it will operate smoothly or will need a separate power adapter.

This question being answered by ChatGPT was very helpful. If we decided not to do research on voltage regulators, we would more than likely have had our components fried when sticking them to the chair and possibly have more components within the chair itself that could've been fried causing us to be set back even further on the project. A bad thing though about the prompt that was

given, is that there were no sources that were cited. We had to ask ChatGPT where it got this information from (**Check appendix A for where we got our sources from for the first prompt.**). One downside to LLMs is that when it is giving answers to questions, half of the time, if not more, it doesn't cite where it got its information from. This could lead to people accidentally stealing information from others without giving credit where it is due. Also, when a new invention or idea is being used, and that person publishes their work, it could lead to them being sued because their idea they got from these LLMs were copied from another person's published idea without them realizing it.

Example #2

For our second example, when doing research for the type of battery we should pick, we gave ChatGPT a basic question: **“Between Sealed Lead Acid, Gel Cell, and Lithium Ion for two-12V batteries, which one would you use for a wheelchair?”** ChatGPT gave a rather lengthy answer. Check appendix A for the actual prompt. To sum up, the answer it gave us, it said when picking the batteries, we must prioritize reliability, safety and range. Right after saying this, it goes ahead and tells me that the best choice out of the three batteries is Lithium-Ion. ChatGPT explains its reasoning for picking Lithium-Ion is because it reduces the weight of the wheelchair, has a longer lifespan, faster charging, and higher usable capability. It states the downsides of going with this choice as well: more expensive upfront, requires proper battery management system, and special charger needed. Next the LLM goes to the next best choice which in its case is the Gel Cell battery. It goes into both the upsides and downsides of the Gel Cell as well. Next, it says it doesn't recommend the sealed lead acid battery then states why. ChatGPT didn't provide any upsides to the SLA in its worded explanation. Next, the LLM created a table. The rows of the tables were the three different battery types while the columns of the table were the different properties of the batteries that we should rate when looking for a battery. Of course in the table the LLM showed the Lithium-Ion battery in the top row as having most of the better properties with the SLA in the bottom row having the worst ratings. Lastly the prompt had given a two sentence summary for why it would choose Lithium-Ion batteries over the other two.

When dissecting the prompt ChatGPT gave us, overall, it gave us a good answer. It told us why it picked lithium ion and then went into a lengthy explanation as to why and even compared the three types of batteries. A downside of this answer that we did not like is the attention to detail. We weren't given the price range of each type of battery, the amount of time each battery would last as well as the weight we were looking at for the batteries. It was very convenient of ChatGPT to give us a general idea of which battery is going to have the lowest or highest type of property value out of the three but we needed to go farther into our own research to figure out the most suitable option. If we would've gone with lithium-ion as the LLMs choice, it would've taken a bigger hit out of our budget that we really wouldn't have been prepared for. We should've put down on the user interface that cost is higher on the priority list to keep in mind when choosing a suitable battery. Then ChatGPT would've given us a more preferred answer. ChatGPT's response in this case is a good example of needing

to give the tool more detail to give a better answer. In going through research, we need to be very specific with an LLM or it could send us down the wrong path.

Another thing we liked about the answer ChatGPT gave us is it compared all three batteries together when all we asked is which is the more suitable option. This general oversight gives us a good direction to go so if we don't want to go with the answer ChatGPT provided us with. We can look through the comparisons it gave us and pick one of the other two batteries to do further research on. Overall, this example really goes to show how LLMs could benefit our research and learning experience for this project.

Example 3:

A very important piece of information we decided to run by ChatGPT to make sure our idea for the project was on the right track was about the rules and regulations for designing electric wheelchairs. **"What are the current rules/codes in place for designing or adding onto electric wheelchairs? We are in the U.S. and in the State of Florida"** ChatGPT gave us a very lengthy and informative response to this question laid out in four sections. In the first section, it talks about Federal Standards for Electric Wheelchairs. Here, it talks about the expected safety standards, power mobility devices, durability testing. For each of these, the LLM has a one to two sentence explanation for each topic not going too much into detail. In the Federal Standards section, it also brings up ANSI standards that are expected for electric wheelchairs and things relating to them. In the Florida Regulations and Codes section, it talks about ramp design, clearance requirements, parking spaces, electrical components, and Florida motor vehicle codes. In the third section, key considerations for modifying electric wheelchairs, it talks about the battery regulations, motor and controller modifications, and ADA compliance. In the final section "other considerations for wheelchair modifications", it talks about insurance and reimbursement, warranties, and safety. Finally in the ChatGPT prompt, it makes a table summarizing the relevant regulations to our electric wheelchair project. In the table, it shows what the name of the code or rule is called and then goes into detail on what it talks about.

When trying to make an autonomous moving electric wheelchair, it is very important for us to keep in mind the rules and regulations so we can comply with the law and make a safe and user-friendly certified wheelchair. While we already checked through the web to make sure we complied with the regulations, we wanted to run our question through ChatGPT to see if it would pick up on something we might've missed. One major benefit of using ChatGPT for our project is quality control. Using LLMs in this type of way for our project/research is ethical and a good way to make sure the direction we are going in is right. While we used quality control to make sure our project followed guidelines, we also used it for some other things: spelling and grammar mistakes, verifying component compatibility and connection, assessing code functionality, and selecting the appropriate voltage regulators for specific components, etc.

ChatGPT actually was able to point out something that we missed regarding rules and regulations. Apparently, American National Standards Institute (ANSI) accredited the Rehabilitation Engineering and Assistive

Technology Society of North America (RESNA) with making standards on wheelchairs. Even though standards are not required, it would be very conscientious to comply with these standards so it will be a socially acceptable design. These standards are talked about more in chapter 4.

While ChatGPT does good with quality control, its answer for us might not be of any use. By this we mean ChatGPT isn't up to date with the info in its database. The current version of this LLM is updated with information all the way up to 2023. While this is still relatively up to date, rules and regulations are getting updated pretty frequently so it is important to have access to the newest released rules. This is a way ChatGPT hinders our project. Thankfully, not that many updates and additions to the rules or codes are added every year so it is easy to keep up with current ones for this project.

Using LLMs For Our Project

Table 5.1: Pros Vs. Cons of Using LLMs For Our Project

Pros	Cons
Quality Control	Only has information up to October 2023
Makes Comparisons Well	Quality of Input Affects Output (Have to be very specific to get what you are looking for)
Retrieve Information without going to find it.	Doesn't Usually Cite its information
Helps with Component Selection	Potential for Inaccuracy
Code Assistance (Assists in debugging and optimizing our code for microcontroller)	Ethical Dilemmas (providing advice that might not be appropriate in a given situation)
Converting links to proper citations and References	Understanding of Complexity of Multi-Component System

Chapter 6 Hardware Design

6.1 Power Supply Systems

The wheelchair base design is powered by two individual 12v batteries. Each one connects its positive and negative leads to contacts at the bottom of the chair's wire system. Since we intend to incorporate the original PCB design we will be utilizing the original power delivery from batteries to controls and signals from controls back to the motors. As discussed earlier the battery composition will affect charge capacity and battery weight. The original wheelchairs battery level reader is based on its original voltage ratings so we will need to implement our own that properly outputs a reading against the battery capacity we're using.

Since we also have sensors and Control Units that require different voltages and nonstandard power supplies not provided by the wheelchair, we will need to adapt the 12V and 24V rails into 5V and 3.3V that is needed by these systems. We may be able to get away with utilizing the 12V of a singular battery for any sensors that require 12V. However that comes with the consideration for unbalanced batteries and could cause issues. So it maybe best to take the 24V and convert it to the desired voltages requires

One thing we may not need to worry about is the charging circuitry of the batteries. Since the wheelchair comes with a fully kitted out charging system we can utilize that to charge the batteries. That of course is dependent on the adaptations we can make to the smaller battery capacities. Based on preliminary testing it seems to work well, however we will properly test to confirm this.

6.1.1 3.3V Regulator

In our design, the 3.3V regulator will be tasked with stepping down the higher input voltage from the battery system. This is especially important for devices like the ESP32 microcontroller and various sensors that require 3.3V for proper operation. By utilizing a buck converter for voltage regulation, we can achieve efficient energy management, minimizing wasted power and heat generation. This efficiency is vital, especially in a battery-operated device, as it extends the overall battery life and enhances the system's reliability.

During the implementation phase, careful consideration will be given to the selection of the regulator. It must support the necessary current ratings and ensure low dropout voltage, which is essential for maintaining performance under varying load conditions. Additionally, protective features such as overcurrent and thermal protection will be integrated to safeguard the circuit from potential damage. Ultimately, the 3.3V regulator not only plays a vital role in the overall functionality of the SmartStride wheelchair, but it also contributes to the safety and longevity of the electronic components involved in the project.

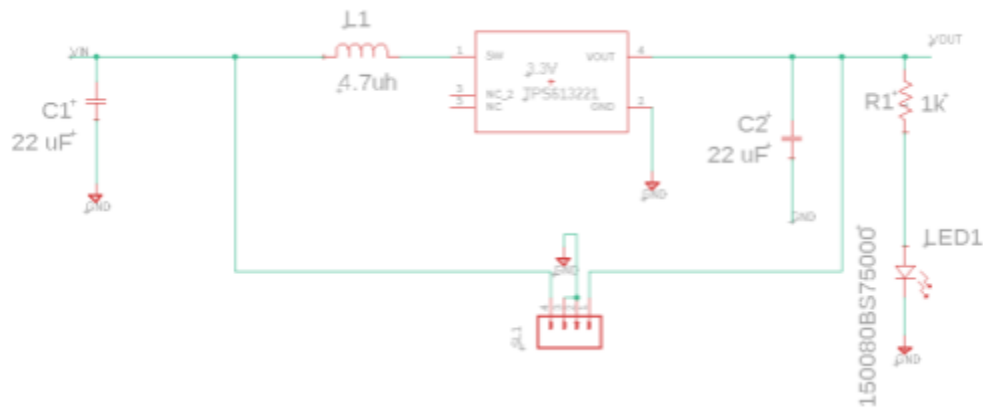


Figure 6.1.1.1: 3 Voltage Regulator

At the core of the circuit is a 3.3V voltage regulator, which is essential for maintaining a stable output voltage suitable for powering sensitive components. Supporting this regulator are capacitors (C1 and C2) that filter and stabilize the voltage, smoothing out fluctuations to ensure a consistent power supply to the load. An inductor (L1) is also included as part of the voltage regulation circuit, managing current flow and contributing to the overall efficiency of the power supply.

Additionally, a resistor (R1) is used to limit current and divide voltages within the circuit, likely controlling the current flowing to an LED (LED1), which serves as an indicator to show the status of the circuit. A diode (D1) is incorporated for protection, preventing reverse current flow that could damage other components. Overall, this schematic illustrates a simple yet effective power supply circuit, designed to convert a higher voltage input into a steady 3.3V output suitable for various electronic devices.

6.1.2 5 V regulator

At the center of this circuit is the 5V regulator, which transforms a higher input voltage into a consistent 5V output, making it suitable for powering various electronic devices that require this voltage level. Supporting the regulator are two capacitors, which serve to filter and stabilize the output voltage, effectively smoothing out any fluctuations that may occur during operation. An inductor is included in the circuit to help manage current flow, enhancing the efficiency of the power supply. Additionally, a resistor is utilized to limit current and divide voltages, likely regulating the current supplied to an LED, which acts as an indicator for the circuit's status.

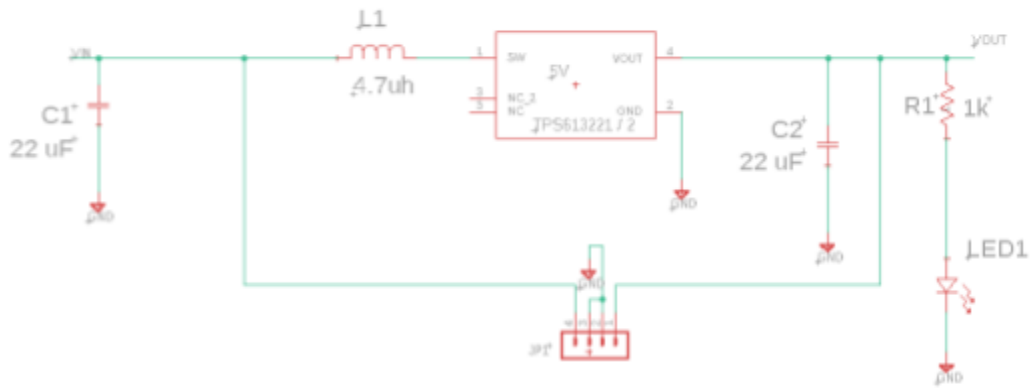


Figure 6.1.2.1: 5 Volt Regulator

6.1.3 Battery Level Indicator

The LM3915 senses varying voltage from a battery and lights up a corresponding number of LEDs to indicate the charge level. A resistor divider network (comprising R2, R3, R4, and R1) scales the battery voltage appropriately for the IC's input. As the battery voltage increases, more LEDs light up in sequence, giving a clear visual cue about the battery's state of charge.

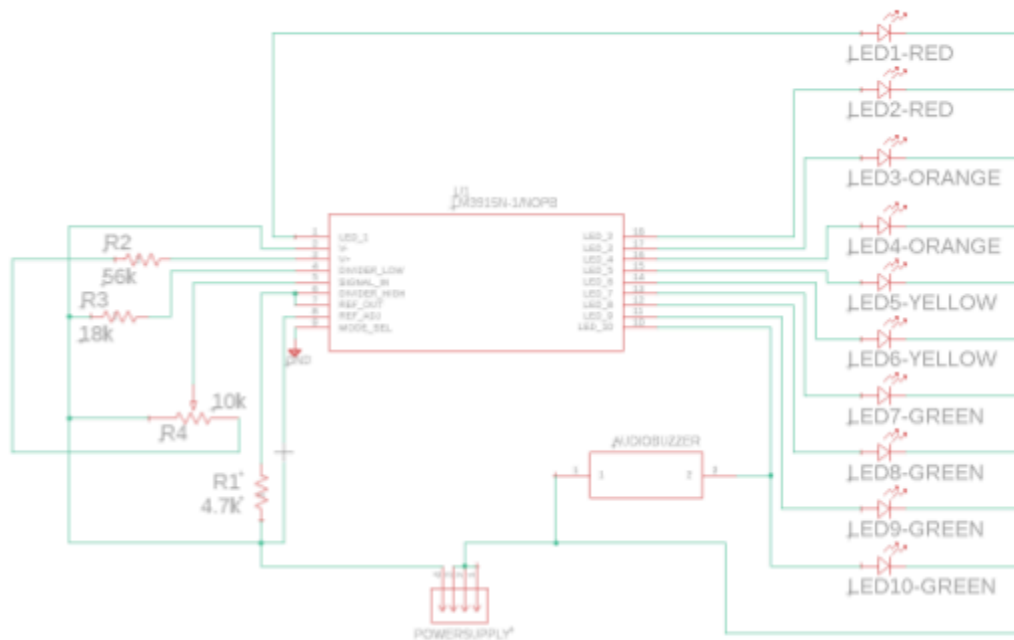


Figure 6.1.3.1: Battery Level Indicator Schematic

The circuit includes ten LEDs arranged in three color zones to represent different charge levels: red LEDs indicate a low battery level, orange LEDs signal a medium-low level, yellow LEDs show a moderate level, and green LEDs reflect a healthy or full charge. These LEDs are driven directly from the output pins of the LM3915, with each pin corresponding to a specific voltage threshold. Additionally, an audio buzzer is connected to provide an audible warning when the battery drops below a specific voltage, alerting the user to recharge or replace the battery.

Overall, this design is a practical and user-friendly solution for monitoring battery voltage in real time. It's often used in portable electronics, renewable energy systems, and any application where maintaining awareness of battery health is important. The LM3915's ability to convert analog input into a bar graph LED display makes it especially useful for quick visual diagnostics without the need for a digital voltmeter, hence why we are using this for our battery indicator.

6.2 Interfacing with WheelChair PCB

Our approach to integrating with the wheelchairs original PCB will involve interfacing with the 6 pin connector that the original joystick connected with. Simulating these signals correctly with the Raspberry PI will be crucial for adequate response times and proper translation of our desired movements. This will need to be the cause in either autonomous mode or manual control. This pin connector is expecting 1 ground ,1 5 volt pin and 4 axis inputs.

The brains of our self driving will be a separate PCB with separate power so as to minimize risk of shorting any critical components on the original wheelchair PCB. Mitigating electrical damage and risk to the user.

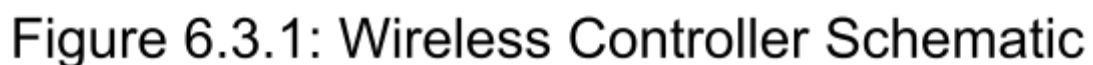


Figure 6.2.1: Original Controller PCB

6.3 Wireless Controller Schematic

Since we are overriding the original controller in favor of our own controller mechanism, we need to give the user an alternative method to control the

This schematic is fairly straight forward, we have a ESP32 on board with a bluetooth receiver built into it, we then have 4 AA batteries on board to power the ESP32 and the joystick. We need to power regulate the 6V from the batteries to 3.3V for the ESP32 and 5V for the joystick. We then have a voltage divider connected to the Power supply and the ESP since the ESP cannot take more than 3.3V input and then measure the divided signal to then use the value to estimate the battery life. We leave a bit of resolution of the voltage on the table since we are using a voltage divider rather than a dedicated chip, however this simplifies design and cost. We finally then have 4 LEDs for a battery indicator as



we want to forgo a display on the controller for ease of use. Of course we have the 4 controller pins connected to the ESP directly since the ESP as a ADC on board to convert the analog joystick signals to a signal we can read. Finally we have programming pins for us to flash the ESP's onboard memory.

6.4 Pi DAC & Power Delivery

Since we will be using a SBC, the Raspberry Pi 5 needs to connect to a PCB so that we can route its pins to components and “breakout” the connections. We use a 2 pin input so that we can take in the 24 Volt rail from the main electrical system in the wheelchair, and then break those out into 2 types of voltages for the Kinect and Pi. We have a 24V to 5V converter to power the Raspberry Pi, the converter is able to make 5V@5Amps without issue so we can run this Pi at full bar if possible. We then have 3 24V to 12V converters to power the connection to share the load for the Kinect. We then port the 12V rail off to other pins since we will be sending it to the Kinect.

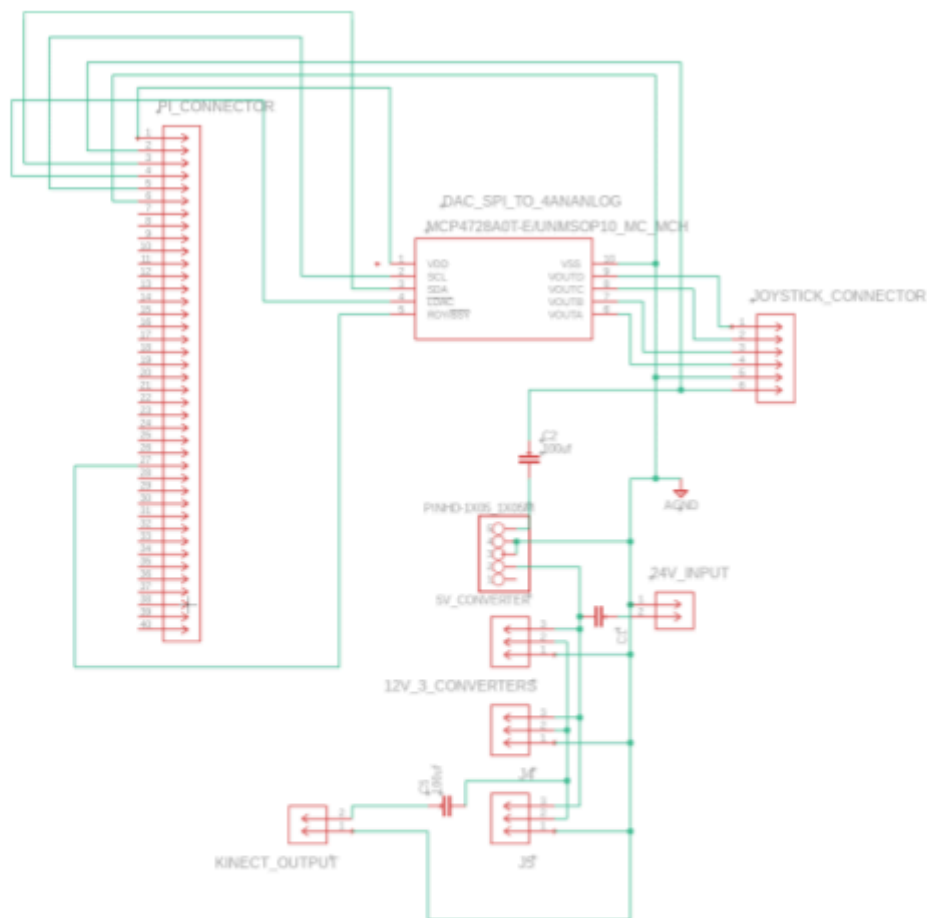


Figure 6.4.1: Pi to DAC & Power Supply Schematic

We then have the Pi connected via SPI to a 12 Bit DAC that is capable of 4 outputs. This is required since we need to output 4 separate voltages to signal the chair similarly to the original joystick. We can use the Raspberry Pi's built in 3.3V rail that it powers to the DAC, as typically the DAC needs a very small amount of amperage via this 3.3V rail. This reduces the complexity of our schematics by reducing the amount of on board regulators.

Finally we use 100uf capacitors to smooth out our voltages as we need our voltages to be clean for our DAC, so keeping ripple at a minimum is a must. These DACs are originally made for audio equipment, but using it in this way is quite useful.

Chapter 7 Software Design

7.1 Emergency Stop Button

In order to ensure the safety of the driver we need to ensure that the wheelchair has a way to stop all movement and terminate all instructions being run when the chair is in its automatic movement mode. This will be done using a button, roughly the size of the tip of one's finger, built right into the controls to allow ease of access to the user. This button, when pressed, will automatically activate the brakes and cease all motor function when the chair is actively running and if it's in the automatic mode it will stop taking instructions and reset to manual mode. The buzzer will also be activated to alert the user and other pedestrians that aren't aware that there may be an emergency caused by the chair's current functions.

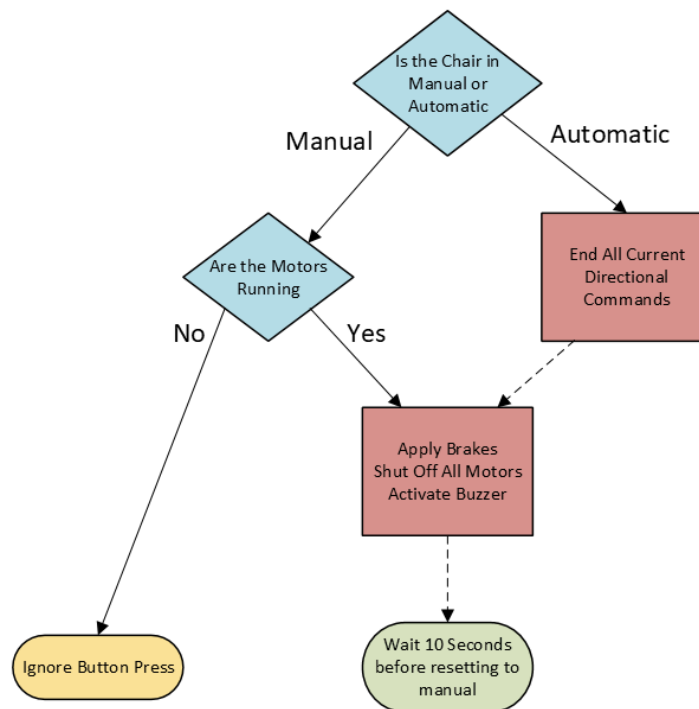


Figure 7.1.1: Emergency Stop Visual

The operation of how the button will perceive that it is being pressed will be relatively simple. All that is required is a basic understanding of how buttons work in the first place and how to use them properly when reacting with the ESP32. When the button is not being pressed the input signal of the button will be read as high. Here's a simple diagram showing the process:

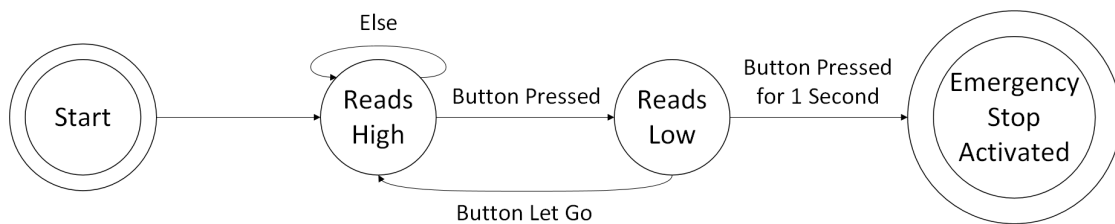


Figure 7.1.2 Button Programming Diagram

7.2 Selfdriving/Cruise Control Algorithm

For our self driving algorithm we have a few prototypes in motion. This is the most simplistic design we came up with. We have 4 conditions our algorithm looks out for any of these conditions being set and then applies the brakes and gives control back to the user.

- If we detect any User Input from our Wireless Controller
- If we detect any depthmap data that is within 1 meter
- If we detect if our Ultrasonic Sensor's reading is within 1 meter
 - Smoothed over a short period of time
- If our Object Detection Model's detect an object within 1 Meter
 - Last condition due to inaccuracy

These conditions make up the bedrock of the Cruise Control Self Driving algorithm. This is intentionally simple, as we don't want to have our self-driving system be built in a way to take the autonomy away from the user. Our number one priority is to give control to the user, so our first condition needs to be looking for user input, and never stealing it from the user. This also keeps things responsive and also keeps the Algorithm from making any errors that the user cannot override and fix.

This kind of methodology exists for self-driving cars and delivery buggies, often having the driver or remote user respectively controlling these machines when an issue arises. We don't need the user to distrust our system or ever feel like they are being a passenger in their chair. Especially since this will be their main form of movement, we need to be mindful of the user's autonomy.

We also use the Depth Map as our main condition when deciding to kill the algorithm. It is by far our most reliable and high resolution view into the real world, as such it is our main source of truth and the first consideration with selfdriving.

We have ultrasonic sensors on board to measure the distance in front of the user. Our hope is to use this when the depthmap fails or simply doesn't detect a small object. We then have the Ultrasonic Sensor's output normalized as it can often give erroneous readings that need to be addressed. However as a second check it is adequate and simple to implement.

We finally then have our Object Detection system used as a last resort. Since it is least reliable, we don't want to tap into it unless we have to. This is still a very capable condition to check, and means we can be sure that the Depth Map and Ultrasonic sensor is being covered on their weaknesses.

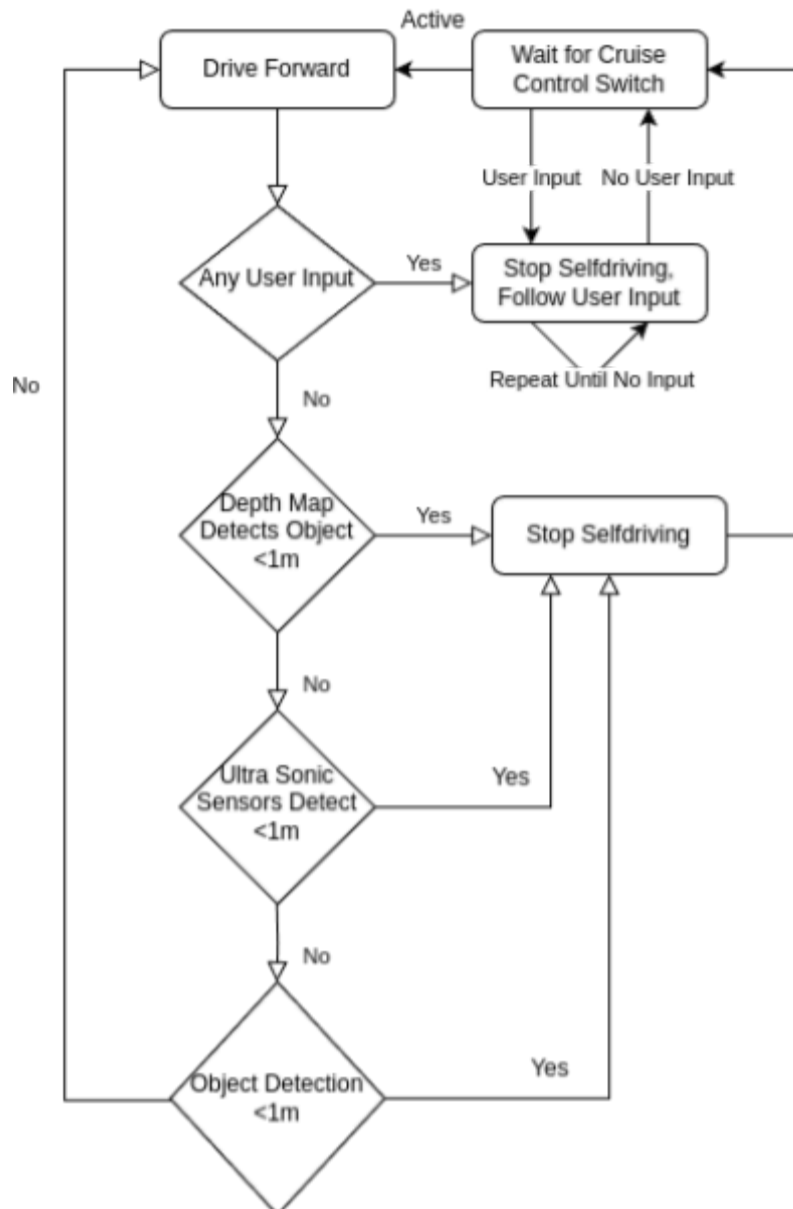


Figure 7.2.1: Self Driving/Cruise Control Algorithm Flowchart

Supplemental Design

Early in our development, we had a much more simple system we wanted to integrate. We decided to try out a prioritizing system with FreeRTOS on an ESP32 system with a sensor that didn't require lots of processing. The idea was to make modular systems in place that would remotely communicate with the Pi and since they are not tightly integrated into our MCU, we could use them and scale up as we needed. We decided to go for a more centralized design as it simplifies development and leads to more accurate Fusion algorithms since we can tailor for these sensors.

We still plan to integrate one or so of these designs into the system, however the idea is for them to be Supplemental to the main sensor design and Selfdriving Algorithm. Here is an online prototype of this code so that a visualization on how this supplemental design may work in our design. The override button was disabled as that would be handed on the Pi itself, but it will inform the user of the current status of the system. Click on the Ultrasonic Sensor and adjust the value to visualize a hypothetical scenario.

Wowki Online Design: <https://wokwi.com/projects/437672071848409089>

7.3 Joystick Workflow

Since we are utilizing the electronics on the system, we need to design a way to communicate with the system that was never intended to be communicated with. Especially since this system is much older (2003) the likes of the Raspberry Pi and ESP32 didn't exist. However when we examined the controller PCB of the wheelchair, we learned that the joystick is a typical joystick that we can un slot easily, utilizing a 6 pin. We learned it uses analog signals as the joystick is just a bunch of potentiometers, so we need to design a way to communicate to the wheelchair through this controller port. We need to send a signal down these pins and basically emulate the joystick to the wheelchair so we can control it.

Utilizing a DAC, we can send an SDI or I2C connection to communicate with this wheelchair. Since we can't move 75% forward, we need to get creative with how we send signals. We will map the controller on an XY plane, and then move a point around the origin, a 2d representation of the joystick. Then if we move it up we then know we need to modify the voltage value from a range of 2.5 to 3.3. By default we tell the DAC to constantly send a voltage of 2.5 volts. Then we would lower the inverse direction.

We can't send a voltage value to the DAC, but a numerical value of 12bits (4096 to 0) to address the voltage. The default we would send to each rail would be 2048, then use the I2C communication protocol to address the rail we want to modify, then move along the numerical range to change the voltage along those rails. We would be moving between the ranges 2234 to 2703. This is due to the limited voltage range of 1.8V to 3.3V of the original joystick

Chapter 8 System Fabrication/Prototype Construction

This portion of our project we need to make sure we do right as the feel of our project is most important in making a final project. It also has the most area to go wrong, as a non-rigid construction or verified design can lead to headaches later we will need to debug or buy new parts. Otherwise the general “vibe” or the construction of the prototype will also lend itself for better construction and overall a better product for our users.

8.1 PCB Fabrication

When it comes to the PCB the most challenging task will be designing it, the actual fabrication of the boards is relatively simple. Using online retailers, a shipment of working PCBs can be easily shipped to our location by simply creating a detailed fabrication document and submitting it along with a request of a specified amount of that design’s board. Along with the actual board we will need a soldering stencil which will also be ordered from the same company we are ordering the boards from. The stencil will be just as important as the PCB as without it we won’t be able to install any specific components to the boards such as resistors and capacitors. Due to the most recent development of tariffs imposed on countries that are known to manufacture boards, we expect the boards to be quite pricey but luckily there is a plethora of options to choose from, including pcbway.com, so we will be investigating which best suits our economic needs.

Since the boards are not fully installed with all the components we require, we will be doing two things, ordering the components online and soldering them manually. As stated before the boards can be soldered using nothing more than the stencil, soldering paste and an applicator, all of which can be easily purchased or found at UCF’s engineering labs. The components that will be soldered onto the board must be ordered online and would have already been taken into consideration when designing the PCBs making the only task regarding them is sourcing them from an online retailer as most of the specific components we will need will not be found in the UCF labs. When the components have been placed onto the board the final step would be to use a soldering oven to set all the components in place. From there any extra components, such as the ESP32, pin connectors, etc. can be manually soldered using a soldering iron.

8.2 Controller Casing Prototyping

The controller casing will be constructed using a 3D printer, both in resin and filament to determine which type of material would be best for the final design. Using 3D printers gives us the ability to quickly fabricate new designs in a mere couple of hours, saving us the hassle of having to order our design online

from other retailers which will save us both time and money due to no longer having to ship the parts from another location. We have access to both types of printers, UCF's filament printers located in the TI lab and a set of personal resin printers that we have 24/7 access to. It would only take a couple of hours to fabricate new casings so the prototyping phase should be finished relatively quickly once we have our PCB design completed.

8.3 Sensor Casing/Mounting Fabrication

The sensors do not come with any mounting devices that will allow us to immediately be able to attach them to the chair or any other component we plan on using, therefore we will have to design our own methods for attaching the sensors to the wheelchair. Like the controller casing we plan on using 3D printing materials for creating our mounting components. Due to the fact that these components will have to be more rigid than the controller casing we will have to use mainly filament when fabricating them. This is due to the fact that filament is less brittle than resin, giving the devices some more tension strength so that it will be more reliable at maintaining its shape and not breaking from anything hitting the sensors mounting devices. The mounting devices may take a more awkward shape in order to either suspend itself on the chair or conform to the shape of the sensors, as the Kinect will have a longer body that may need to be held completely still in order to stop it from moving too much and moving things out of vision. Other non-printed components may also be used to ensure that the casings and mounting devices are able to stick onto the chair, such as adding holes to the prototypes to allow us to use screws to keep them locked in place.

8.4 Pi to DAC & Power Delivery PCB

This PCB is a prototype of our Raspberry Pi to DAC and Power Delivery PCB put together. Since we need a 40 pin connector for our Raspberry Pi, it makes the board much wider than wanted. However we require to keep the Pi a distance away from the converter boards and other electronics due to the high thermal footprint of the Pi. We also need the 24V and 12V rails close together since we want to minimize wiring in our system and keep it far from the 5V rails we have set up for the pi, and the 3.3V rail for the DAC. With more creative wiring we may be able to reduce the footprint of this PCB, but keeping the converter boards and the SBC needs to be considered.

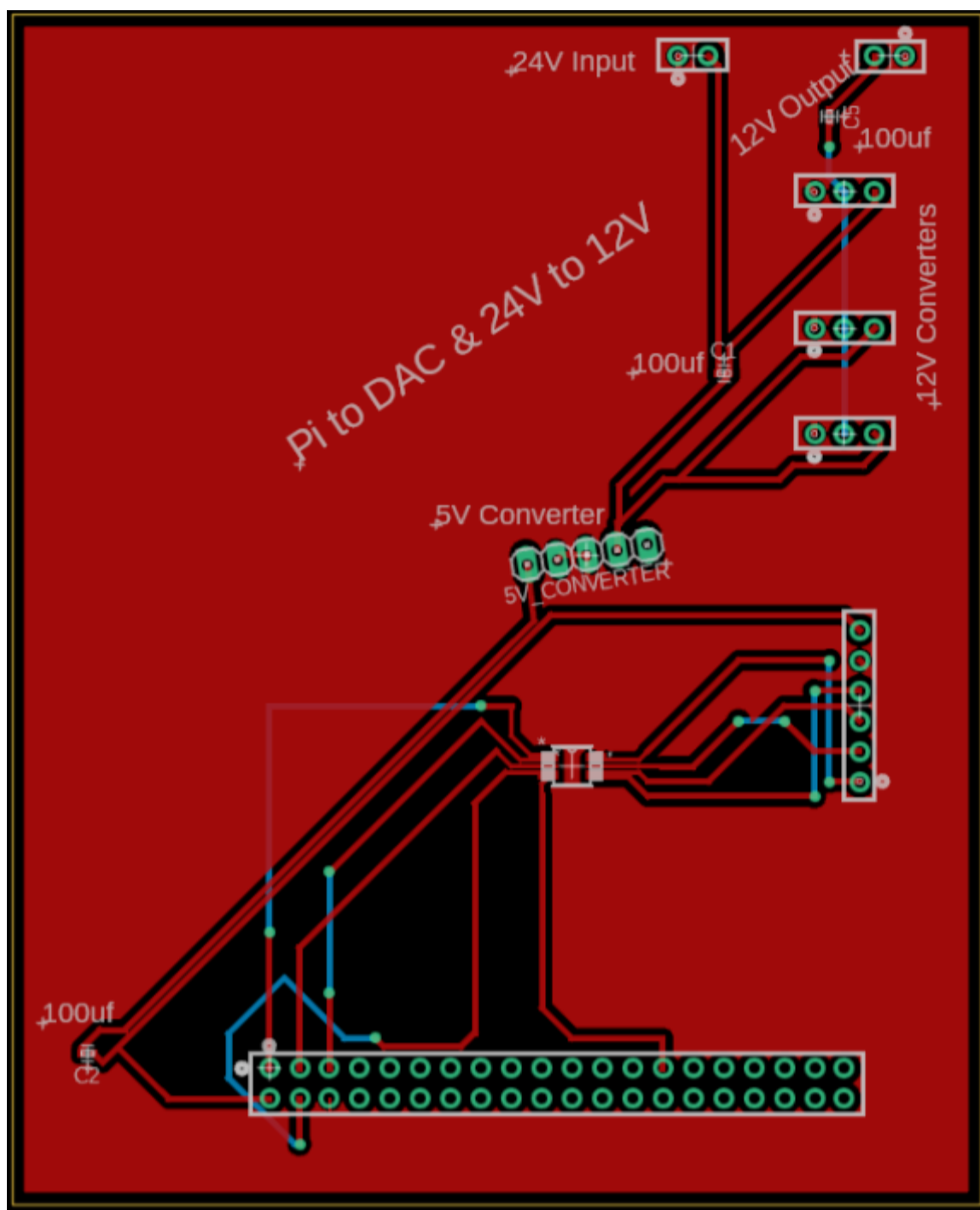


Figure 8.4.1: Pi to DAC & Power Delivery PCB Design

8.5 Wireless ESP32 Controller PCB

This is the prototype PCB for the wireless controller. It utilizes the ESP32 to communicate via bluetooth to the RaspberryPi. This design incorporates a voltage regulator in order to supply the ESP32 with the proper 3.3v and a controller that draws 5v. This will be achieved with two different regulators and a power source of 3 AA batteries.

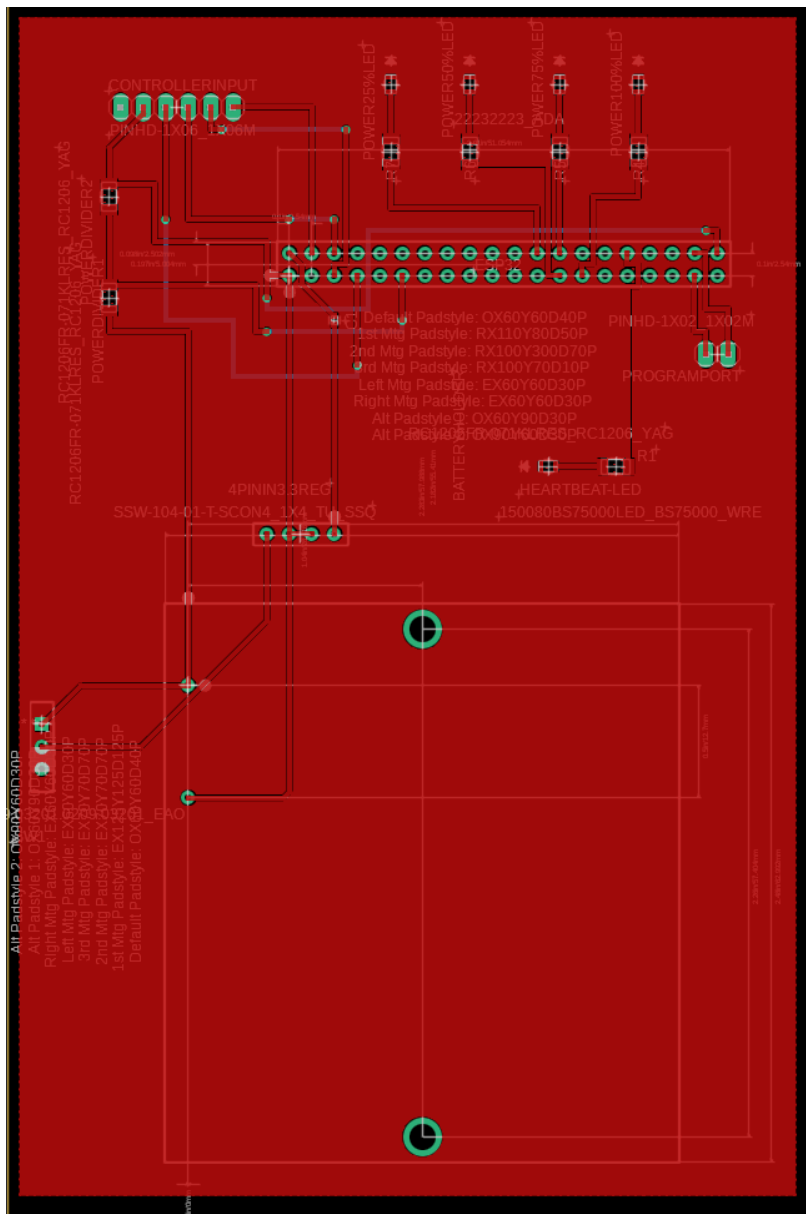


Figure 8.4.2 ESP32 Wireless Controller PCB

Chapter 9 System Testing and Evaluation

The testing and evaluation phase overall is a super important aspect to this project. It is also a very fundamental process part of the design phase that needs to be done in order for a product to be successful. For the SmartStride, the testing and evaluation is grouped into two different parts: hardware and software. For hardware, we will deal with a focus on tolerance in our testing; for example, for the joystick, pushing it farther away from its default center position to get a feel for its sensitivity. We will test the motor tolerance on different speed settings as well as to how the batteries handle stress and the length of time they can push out a consistent current until they discharge completely. For the software side, it will be more of debugging and making sure the emergency stop functions and code for the Kinect's depth rendering and sensing work well. While we are looking for our software to run well, we placed a bit of emphasis on getting our code to operate in the most efficient way possible to minimize power output.

9.1 Hardware Testing

Hardware testing for the SmartStride involves a list of very in depth evaluations of individual components to ensure their functionality, performance, and adherence to design specifications. Each component underwent specific tests specifically designed for it to make sure it upholds our standards as well as ADA standards. Another reason we go through specific tests is to identify potential issues that may arise that we don't see while operating the whole entire prototype at once. The testing protocols for the hardware covers a wide range of components: LCD screen display, emergency stop button, buzzer button, batteries, motors, and joystick. The individual component testing is important in ensuring the reliability and safety of the wheelchair.

9.1.1 LCD Screen Testing

The LCD screen is relatively easy to test for, all we need to be able to do is go through each of the LCD's listed features and make sure they are in a presentable fashion or create a response that can be detected by the hardware. The LCD will mainly need to be outputting readable text so a couple of programs that output different words in different sizes would be run to test its full capabilities. The fonts and image sizes that appear the best to the user will be selected and used for formatting in the final program of display features on the LCD. Since images will also be shown on the LCD we will need to run some programs that will show the full color capabilities of the screen so that users can more easily differentiate between the features displayed on the images.

We also need the display to either take in a pinout system, or can hook into a micro-HDMI connection since we do have access to this on our MCU. Being able to actually connect the display reliably and program it properly is important for showing the user what stats they need to see.

LCDs need good viewing angles too and must have a high brightness to accomplish what we want in an outside setting. Since direct sunlight can prevent our display from being visible easily, we need it to be bright enough to overcome direct glare. As well as viewing angles must be solid since we don't want our user to need to adjust themselves to a better angle to view potentially important information on the display.

9.1.2 Button Testing

The emergency stop button needs only one important feature that needs to be tested, that being the ability to display a high and low signal. All that needs to be done is connect the button to an ESP32 or other programmable board and ensure that a high signal is going through when the button is not being pressed and a low signal when someone is pushing down on the button. Whether or not the emergency stop itself works will need to be done in software testing but with the physical button working it shouldn't be too much of a problem.

9.1.3 Buzzer Testing

While the buzzer itself is not a necessary component for the functionality of the emergency stop signal or even the user's ability to activate it, it is still an important component to test whether or not it works and whether or not someone could hear it may even save someone's life. The buzzer needs to be mainly checked for functionality, if we can get a sound going through it then that means it is working properly. Now after that what needs to be tested is the range that someone will be alerted that the emergency stop has been activated, to indicate to other people besides the user that something dangerous could have occurred. Preferably the buzzer would need to be heard from a couple of feet away so as long as we can get people's attention from around 10 feet the buzzer should be usable.

9.1.4 Battery Testing

The main goal of this section is to ensure the two Mighty Max ML35-12 batteries we obtained for the project can output the correct amp-hour capacity it was marketed to be and ensure it can constantly and consistently supply power to the load for at least 1.5 hours. In addition to these main objectives, the battery testing also serves several secondary purposes: confirming safe charging/discharging behavior, validating suitability for estimated power draw, and ensuring there are no abnormal drops in voltage. The testing setup consists of the current load of the wheelchair and all of the devices we added onto it as well as a multimeter, timer, and safe testing environment. To clarify, our safe testing environment is in an indoor location (proper temperature conditions) with a well ventilated area.

Table 9.1: Battery Testing on Load

Test #	Load Current	Initial Voltage	Cutoff Voltage	Time to Discharge (Hour:Minute)	Measurement Capacity (Ah)
Test 1	25.5 A	12.68 V	10.5 V	1:32	39.10
Test 2	25.5 A	12.70 V	10.54 V	1:29	37.82
Test 3	25.5 A	12.65 V	10.5 V	1:39	42.08

Looking at the table above, the testing of our batteries went very well. We got better than expected results for both the discharge time and the measurement capacity. The results were not too much better but it still surprised us. The measurement capacity was obtained by multiplying the load current by the time to discharge (converted to decimal form). We want to clarify that the measurement capacity is different from the battery rated expected capacity. They are different because of the measurement capacity taking the load current into account while the rated battery doesn't take into account a load. When rating a battery, the manufacturer makes their rate around testing the battery by discharging it at 1.75 A for 20 hours. If the battery lasts for the full 20-hours, it passes as a "35 Ah" rated battery or whatever specific rated battery it is.

Another factor we would like to bring up from the battery testing is the initial voltage as compared to the cutoff voltage. In an earlier section, we predicted the battery to fluctuate +/- 2V from its rated value (12 V - 10V). From the table above, we see the highest charge it had was 12.7 V while the lowest value it had was 10.5 V. Fortunately we got the voltage values we were expecting in all three of the battery tests. This helps us to know as we researched our voltage regulators around what we researched for the expected fluctuating voltage values. Knowing this gives us the confidence to know that our components and motors are pretty safe from the batteries we got.

9.1.5 Motor Testing

There are many purposes for why we need to test our motors. For one reason, MP3R is a 20-plus year old wheelchair and we want to make sure everything is still working properly. Another reason for motor testing is to verify the load capacity of the chair. When the chair first came out, the load of both the original batteries and the base came out to a little bit over 140 pounds. The max load that the chair can also carry with its weight is 300-pounds. When adding on the kinect and replacing the old batteries with the new batteries and adding on the other components, we probably will only be adding about ten more pounds to the wheelchair load by the time we get a fully functional prototype. It is important

that we keep the weight in mind of all of our components so we can see how much strain we are actually putting on the wheelchair.

Another thing motor testing is important for is efficiency evaluation. We need to be able to determine how efficiently the motor converts electrical energy into mechanical energy and minimize energy waste. An additional thing we would like to assess with the motor testing is the voltage fluctuation and vibrational testing. Each of the motors already has a built-in regulator. While adding on another regulator ourselves and already having the built-in regulator, there might be some issues with the voltage fluctuation causing the regulators in the motor to be fried with all of our new components. We need to verify that running the motor on different intensities will keep it within normal operation. Also with the vibration, we want to make sure the motor doesn't vibrate too much to where the components we add on will have a hard time with them.

There are many different ways to test the motor for all of the many purposes to go by. We want to find a motor testing technique to go by that can check off most if not all of the boxes for the different purposes for testing. We found a pretty easy way to test the motors that would check off all of these boxes. We ended up hooking our new batteries into the base of the wheelchair and then used our old joystick that has three different variable speeds to see how well our motors could perform. When testing our motors, we made sure to stay in a relatively cool location (room temperature or less).

For the first test of our motors, we laid the base of the wheelchair on the floor. From there we hooked up our joystick controller to the base with the batteries. From there on the controller, it has an area on the back-side of it (separate from the joystick) where you have this dial that controls the intensity of

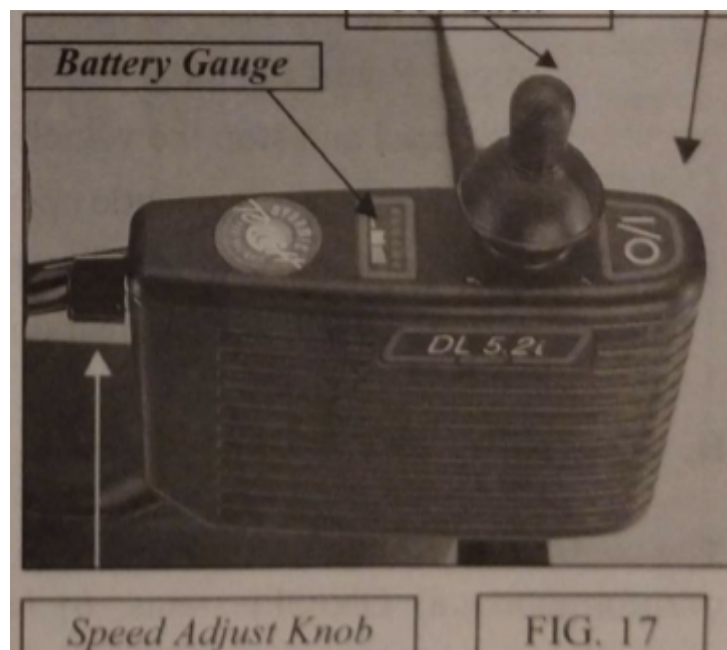


Figure 9.1.5.1: Controller Visual

the motors. The picture below shows where one can find this dial on the controller. This picture was taken from the user manual. In the picture, the “Speed Adjust Knob” is what we are referring to.

The first setting which is the farthest counter-clockwise turn on the dial is the lowest intensity setting. We started off by driving just the base of the wheelchair around without this setting. We drove it forward first, then performed a sharp turn with it to make sure it could handle a pretty good speed. With the joystick pushing all the way forward, we got a good idle speed of about 2 mph which is a good idle speed. We operated the motors on the first setting for about a minute then went onto test the next setting. For the second test, we did the same thing but with the second (middle) setting which is pointed upright on the dial. We drove it forward for about 30-feet, then performed a sharp turn with it. The speed of the second setting was not much faster than the first. We got about another 1 mph faster (3mph total) out of it. This second setting is around the average walking speed of a human. The third and final setting, we did the same thing for. Ran it for about a minute and ran it on a straight path as well as performed a sharp turn with it. Not much changed from either the first to second test at all other than the speed. We got about 4.7 MPH out of it which is pretty good. This speed is equivalent to a fast walk or super light jog.

One other thing we would like to note is that after testing each setting with the chair on the ground, we turned it upside down on its base while the wheels were facing the sky. A multimeter was taken out and was placed between the motor controller and motor connection. We measured the amount of current drawn from one of the motors to get an ideal for the power fluctuation of the motors on different settings. The table below notes all of the important observations from each different intensity setting. We want to make a note that on the current draw column of the table, we are referring to the current drawn from each motor while that setting was on its max speed (joystick pushed all the way forward).

Table 9.2: Motor Performance cross Speed Settings

Speed Setting	Max Speed	Current Draw	Observations
1st (lowest)	2.1 mph	2.4 A	Very slow use, mainly for indoor use.
2nd	3.3 mph	5.7 A	Very comfortable operating range. Easy to control on this setting but still able to go

			pretty fast.
3rd (highest)	4.7 mph	9.2 A	The joystick is highly sensitive to quick maneuvers. This setting seems like it is mainly used for quick travel.

From the table above, the current draw was right around what we expected. In section 3.2.2 when researching the batteries we needed to get for the wheelchair, we predicted how much total current each motor would draw which was right around 10A on the highest intensity. 9.2 Amps measured on the highest setting is not too far off. There is an 8% difference between the experiment value and the predicted value. A good thing about the motors drawing less than 10 Amps is that we might have more battery life than expected.

9.1.6 JoyStick Emulation Testing

Since our main goal is to utilize the existing control PCB of the wheelchair and emulate the original's joysticks movement signals we need to know what sort of voltage values it is expecting. Once we can mimic these signals with output from our central processing unit we'll be able to control the wheelchair with software and it will operate as if the original joystick were still connected.

The original joystick in the wheelchair has 6 lines coming out of it that connect to the original PCB. An Arduino Mega will be utilized. The original joystick has labeling indicating that it receives 5 volts in. Red is connected to the 5v output and Black to ground. To determine which wire corresponds to which axis we will then connect the 4 remaining lines to the analog input of the arduino and monitor the voltage value of each one on serial output. Depending on the direction the joystick is pushed for example in the forward direction then whichever output gives the highest voltage(in this case 3.0+/-) will be the one corresponding to it.

9.2 Software Testing

Software testing is a critical phase of this project, as the software is the crucial link that translates sensor data and user commands into physical action. Without functional, well-integrated software, the hardware components cannot operate as a cohesive system. Given that SmartStride is an autonomous mobility aid, the software must be exceptionally responsive, resilient, and, above all, safe. This testing phase focuses on verifying the logic of our control algorithms, the reliability of communication protocols, and the seamless integration of each

software module with its corresponding hardware. Our goal is to ensure the system behaves predictably and safely under all operating conditions.

This process is divided into distinct modules to validate each core function of the SmartStride system. Key areas of evaluation include the performance and readability of the LCD screen, the responsiveness and override capability of the emergency stop function, and the accuracy of the Freenect2 library in processing depth data from the Kinect sensor. Each test is designed to confirm that the software not only meets its functional requirements but also adheres to the safety and user-centric principles that are fundamental to the project's success.

9.2.1 LCD Screen Testing

While the LCD may be fully functional it is also important that the controller module connected to the screen is able to relay the proper information and images we desire to the user's viewability. One of the most basic but useful tools will be to be able to display text that indicates whether the chair is in a manual or automatic mode. The battery percentage of the chair would also be important to display as if there is no indicator to how long the battery may last the driver may be stranded without many options of getting around. Having the LCD screen also display that the emergency stop operation has been activated would also be crucial to the functionality of both the screen and the emergency stop button, however this feature will most likely be tested with the emergency stop function and not with the LCD programming.

9.2.2 Emergency Stop Testing

One of the most important functions that we must ensure full efficiency of is the emergency stop function. With there being unforeseeable accidents during the product's lifecycle it will be very reassuring to us and the users that the emergency stop works in any situation the driver may find themselves in. The first benchmark in testing its functionality would be to ensure that the button will override any and all commands being run through the automatic mode. After that it's just a matter of verifying that the brakes will be activated whenever the user is in manual and in motion, if for some reason the user needs to immediately stop while they're driving they should be able to rely on the button for assistance. The last and most difficult benchmark would be verifying that the emergency stop can still function if there is an error with the controller or main computer module. We'd have to make sure that the button has the ability to override any function or process that the main module is currently doing and if needs be, have the ability to reset the functionality of the wheelchair as a whole so that it may be able to possibly fix any problems affecting the chair.

9.2.3 Freenect2

Freenect2 is the library that we pull from when interfacing and utilizing the KinectV2 sensor. Specifically we needed to interface with its IR sensor and

create depthmaps that we can utilize and analyse the distances in the map. In the figure you can see the many dots that make up the visual for the wall. This is our IR sensor from the KinectV2 that is mapping out a space and in this testing we create a PCL file that we can view and open. PCL files are a common file format used for storing depth maps and engineering 3d maps of environments.

Protonect Viewer

The primary tool for this initial check is the Protonect viewer, which comes bundled with the freenect2 library. Running this executable will open a window displaying three distinct video streams from the Kinect:

- **RGB Image:** A standard color video feed.
- **IR Image:** The infrared data stream, which will appear as a grayscale image.
- **Depth Map:** A visual representation of the distance of objects from the sensor, typically color-coded for depth.

This visualization, shown in figure 9.2.3.1, means that our Kinect is functioning properly and we are able to get a very clear depthmap. The PCL file creation process is as follows:

1. Capturing a depth frame from the Kinect.
2. Using the camera's intrinsic parameters (focal length, principal point) to convert each pixel's depth value into a 3D coordinate (X, Y, Z).
3. Organizing these 3D points into a PCL data structure.
4. Saving the data structure to a .pcd file.

The final product will not utilize PCL files unless we want to transmit the data to another device which we are not.

We then run the Kinect's IR sensor on a loop and we can then get out a stream of IR images. To make sure we get solid distance data we then find the closest point in the PCL file and measure the distance from point (0,0,0). We then physically measure the distance to the closest point and compare our results. While it can have some error we generally got a value that was close to the output values.

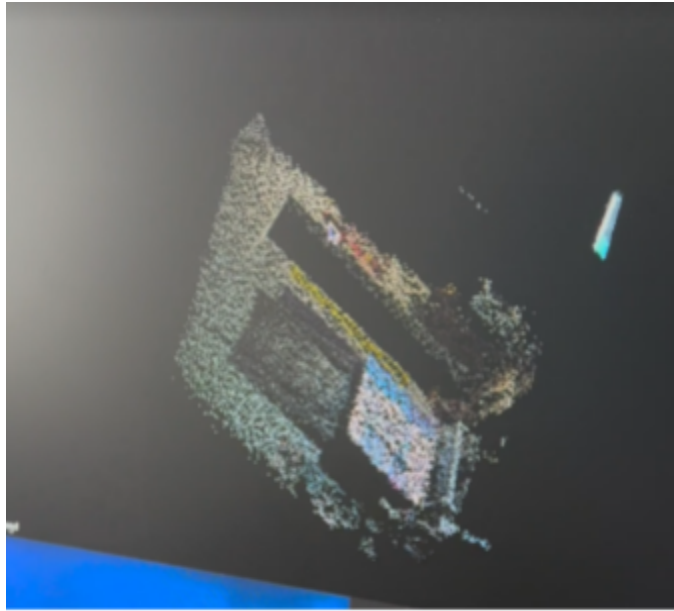


Figure 9.2.3.1: Kinect Depth Map

9.2.4 ESP32 BLE Server

To validate the wireless communication protocol for the controller, we conducted a test using an ESP32 as a Bluetooth Low Energy (BLE) server and a Raspberry Pi as the BLE client. The goal was to establish a connection and have the Raspberry Pi send simple commands to control a peripheral on the ESP32.

An LED was connected to a GPIO pin on the ESP32. The ESP32 was programmed to create a BLE server with a specific service and characteristic that could receive data. The Raspberry Pi was then programmed to scan for this specific BLE device, connect to it, and send data.

The test was executed by sending numerical values from the Raspberry Pi to the ESP32. Sending a '1' would signal the ESP32 to turn the LED on, while sending a '0' would signal it to turn the LED off. The test was successful, with the LED responding instantly to the commands sent from the Raspberry Pi. This confirmed the viability of using BLE for communication between the main processing unit and the wireless controller, demonstrating a stable connection and low-latency command execution.

We used Bleak, a bluetooth library, on the Pi to code the Pi side in python. A consideration too with the BLE protocol is that after some time (default 15 seconds) the BLE server will no longer listen for a response from its client. This is an intentional behavior of the system, as to maintain low energy usage it kills

open connections with little activity. This is to our benefit, but means in deeper integration we will need to program either constant polling or design a time out system that reactivates a handshake.

Testing latency reveals that the BLE protocol is very fast for our uses, which was an original concern when deciding a wireless communication system. However that shouldn't have been a problem since most technologies exist that utilize it for low latency transmission like game controllers and audio.

Chapter 10 Administrative Content

This chapter is the administrative content we have left to go over. This ranges from budgetary, scheduling, and overall division of labor due to our varied talents. This chapter while last outside of the conclusion is most important to note to make sure success is achieved.

10.1 Project Milestones

With our project being on such a big scale with a lot to do in a limited amount of time, we thought it would be best to create a schedule that details each major milestone along the way. Our 120-page document is going to be a big point of focus throughout the Senior Design 1 milestone calendar.

Senior Design 1 (Summer 2025)

Task	Time Period	Dates	Description
Project Selection and Group Formation	1 Week	5/13/2025 - 5/16/2025	This is where we will all come together and form a group, establish group roles/expectations, then form an idea we set on
D&C Documentation	2 Weeks	5/16/2025 - 5/30/2025	We research and come up with the preliminary solutions and constraints of our project
Form Committee	1 Week	5/24/2025 - 5/30/2025	During Writing the D&C document, we contact UCF faculty to form a recommendation and formation committee
D&C Meeting	1 Day	6/3/2025	Meet with Dr. Wei and find out what need to be fixed in our D&C
60-Page Draft	6 Weeks	5/31/2025 - 7/7/2025	Start the 60 page draft. This document will be part of the 120 pager and be a refinement of our solutions

Verify D&C Research	2 Weeks	5/31/2025 - 6/14/2025	During the 60 page draft, we will go back and verify if the work we did in our D&C is fully feasible and adjust solutions based on analysis
Start Design of Control Unit & Battery	2 Weeks	6/7/2025 - 6/21/2025	We start designing the brains of our design. Make sure the PCB is mocked up in CAD software so we can decide what to do further
Start Design of the Motor Units & Mounting	2 Weeks	6/14/2025 - 6/28/2025	We start designing the motors and mounting hardware for our design. This will also house the battery and control designs
Begin Work on Software Designs	5 Weeks	6/14/2025 - 7/9/2025	After we design most of the control unit, we start doing the magic software design for our control unit. We also make sure we integrate everything together
60-Page Draft Meeting	1 Day	7/9/2025	After we finish the 60 page draft we meet with our committee to verify if we are on track and if any alterations need to be made
Begin Work on 120-Page Document	4 Weeks	7/10/2025 - 7/28/2025	We finish the designs and start work on the 120-page document. This will be a documentation of everything so far
Finalize 120-Page Meeting	1 day	7/29/2025	We have a final meeting and demo to verify our design is complete and ready to be realized

Senior Design 2 (Fall 2025)

Task	Time Period	Dates	Description
Verify and Polish Prototype Designs	1 Week	8/19/2025 - 8/26/2025	Take the designs we made in SD1 and properly spruce them up
Order PCBs and Parts	1 Week	8/26/2025 - 9/2/2025	We order PCBs and Parts from major retailers and manufacturers
Use Development Boards & Parts to Test Design	1 Week	9/2/2025 - 9/9/2025	While we wait for things to ship, we use development boards and sub optimal parts on hand to test the design and refine

Assemble with Production Parts	2 Weeks	9/9/2025 - 9/23/2025	After we retrieve our parts, we will properly assemble them together and look for an incompatibilities
Start Planning and Constructing Presentation	3 Weeks	9/23/3035 - 10/6/2025	We need to start thinking about the Demo and extra documentation we need to bring
Intensive Testing & Bug Fixing	3 Weeks	9/23/3035 - 10/6/2025	We continue to develop and fix any unforeseen issues that pop up from production building
Final Order Call for Broken or Defective Parts	1 day	10/7/2025	After we build our prototype, this will be the final threshold of parts/PCBs that need to be ordered if we want them to get here in time
Buffer for Final Order and/or Optimize	4 Weeks	10/8/2025 - 11/4/2025	We will wait for parts to come in, while working on what we can. We will also work on documentation at this point if we can't do any more
Final Adjustments and Test Live Demo + Stress Testing	5 Weeks	11/5/2025 - 12/4/2025	After getting the parts, we make our final adjustments. Remove any extra features that aren't cooked fully or add any quick ones
Day of Rest and Mentally Prepare	1 Day	12/5/2025	The calm before the storm, this is also a day or testing our final presentation to make sure its solid
Final Presentation	1 Day	12/6/2025	The final presentation to the committee and others

10.2 Budget Estimates

Below is a rough estimate of our budget. We included all essential parts that we will need for the project. We estimated how much each part will be. We want to make a note that this budget includes only hardware components. A row in the table below was made to account for additional costs we may have. Our budget focuses on the balance between low-cost and quality.

Table 10.1: Project Bill of Materials (BOM) with Estimated Costs

Item	Unit Count	Estimate	BOM
Wheelchair	1	\$5,000	\$0.00
Wheels	2	\$0	\$0.00
Motor	2	\$45	\$89.99
Battery	2	\$150	\$299.99
Joystick	1	\$14	\$14.99
Controller Batteries	8	\$10	\$9.99
ESP32	10	\$17	\$34.99
PCB	10	\$40	\$79.99
Raspberry Pi	1	\$120	\$119.99
Sensors	5	\$10	\$9.99
Miscellaneous	NA	\$300	
Total	-	\$5,429	\$959.92

10.3 Division of Labor

Since this project can't be done by one person, we will distribute it amongst ourselves so that we can get this done. We have a team mix of one Electrical Engineer Major and three Computer Engineer Majors. This means we will try our best to move many hardware tasks to software when possible, but our Computer Engineers will need to do some electrical work as it is the most important part to get right. Here is a breakdown of the of the labor:

Table 3.18: T

Electrical Engineering	Responsibilities
Adam Lilly	-PCB Design & Implementation
	-Power Technologies Research and Verification
	-Battery Level Indicators
	Hardware testing

Computer Engineering	Responsibilities
Matthew Itskovich	-ESP32 Development and Edge Compute
	-Power System Verification
	-Wireless Controller Development
	-Software Research
Electrical Engineering	Responsibilities
Arturo Lara	-Budgetary and Administrative Content
	-Website Development
	-DAC and Minor Components Selection
	-Display Development
Electrical Engineering	Responsibilities
Evan Rees	-Raspberry Pi Development
	-Sensor Integration
	-MCU Selection
	-Project Lead

Chapter 11 Conclusion

LLM Gemini was used in the drafting of this Conclusion:

The SmartStride project was founded on the mission to enhance the mobility and independence of individuals with physical impairments by developing an affordable, hands-free, and intelligent upgrade for motorized wheelchairs. The primary objective was to retrofit an existing electric wheelchair with modern sensing and computing capabilities to enable safe, autonomous navigation features without the prohibitive cost associated with commercially available smart wheelchairs. Through extensive research, meticulous design, and rigorous testing, this project has successfully demonstrated a viable and functional prototype that achieves these core goals.

The final design of the SmartStride system is a testament to the integration of carefully selected modern technologies. At its core, the Raspberry Pi 5 was chosen as the main processing unit, providing the necessary computational power for real-time processing, including OpenCL acceleration for GPU-intensive tasks. For environmental perception, the Microsoft Xbox Kinect V2 was selected as the primary sensor, offering a cost-effective yet powerful combination of an RGB camera and an IR depth sensor capable of generating detailed depth maps and skeletal data for obstacle and pedestrian detection. This was supplemented by ultrasonic sensors to create a simple, reliable "virtual bumper" for close-range obstacle detection.

A significant portion of the hardware design focused on creating a robust and efficient power delivery system. Acknowledging the economic constraints, cost-effective Sealed Lead-Acid (SLA) batteries were chosen, and custom PCBs featuring buck-boost and buck converters were designed to step down the 24V supply to the stable 12V, 5V, and 3.3V rails required by the Kinect, Raspberry Pi, and other microcontrollers. User interaction was facilitated through a custom-built wireless controller, utilizing an ESP32 for Bluetooth communication, allowing for manual override and control. One of the key integration challenges was successfully interfacing with the wheelchair's original motor controller. This was achieved by designing a DAC system capable of emulating the analog voltage signals of the original joystick, allowing the Raspberry Pi to take control of the chair's movements seamlessly.

The software architecture was built to prioritize safety and user control. A self-driving "cruise control" algorithm was developed to drive the chair forward while continuously scanning for obstacles using a fusion of data from the Kinect's depth map, ultrasonic sensors, and an object detection model. In keeping with our user-centric design philosophy, the system is designed to immediately cede control to the user upon any joystick input and includes a critical, hard-wired emergency stop button that can override all automated functions and bring the wheelchair to a safe halt. Testing confirmed that the hardware components, from

the batteries to the motors, perform within expected parameters and that the core software functionalities operate as intended.

While the SmartStride project has successfully met its basic and most critical goals, the modular design provides a strong foundation for future enhancements. The next logical step would be to evolve the self-driving algorithm from simply stopping at obstacles to dynamically routing around them, a stretch goal initially identified for this project. Further advancements could include implementing incline detection to automatically adjust motor power, edge detection to prevent driving off curbs, and doorway alignment for easier navigation through buildings. The system could also be enhanced by training and implementing more sophisticated and efficient AI models, such as the HARD-S model for semantic segmentation, to better distinguish between drivable and non-drivable surfaces.

In conclusion, the SmartStride project has effectively transformed a standard electric wheelchair into an intelligent mobility platform. It has proven that with thoughtful engineering and the strategic use of accessible components, it is possible to create a low-cost, modular, and safe system that can significantly improve the quality of life for its users. The work documented in this report not only represents a successful academic endeavor but also serves as an open-source blueprint for making advanced mobility aids more accessible to all.

Works Cited

1. The Editors of Encyclopaedia Britannica. (2015, February 11). *History of the wheelchair*. Encyclopaedia Britannica.
<https://www.britannica.com/technology/history-of-the-wheelchair>
2. St George, T. (2022, February 7). *The history of the electric wheelchair*. LITH-TECH Mobility.
<https://lith-tech.com/2022/02/history-of-the-electric-wheelchair/Lith Tech>
3. VOCIC. (2024, November 27). *How to convert a manual wheelchair into an electric wheelchair?*
<https://vocic.com/blogs/news-and-blogs/how-to-convert-a-manual-wheelchair-into-an-electric-wheelchair>
4. Tiny Big Dog Designs. (2019, December 23). *DIY electric powered wheelchair*.
[https://tinybigdogdesigns.com/2019/12/23/diy-electric-powered-wheelchair /](https://tinybigdogdesigns.com/2019/12/23/diy-electric-powered-wheelchair/)
5. Mobilis. (n.d.). *Wheelchair Autopilot*. Mobilis.
<http://mobilis.io/wheelchairautopilot>
6. Battery Group Expert. (n.d.). *Battery group size chart and cross-reference table*. Retrieved July 5, 2025, from
<https://batterygroupexpert.com/battery-group-size-chart-and-cross-reference-table/>
7. No0bert. (2009, February 1). *How much current does a typical wheelchair motor draw?* Picaxe Forum. Retrieved July 5, 2025, from
<https://picaxeforum.co.uk/threads/how-much-current-does-a-typical-wheelchair-motor-draw.11535/>
8. Linden, D. (n.d.). *BU-201: How does the lead-acid battery work?* Battery University. Retrieved July 5, 2025, from
<https://batteryuniversity.com/article/bu-201-how-does-the-lead-acid-battery-work>
9. *Kinect V Vision optics development underground scout robot* (n.d.). 1Library. Retrieved July 5, 2025, from
<https://1library.net/article/kinect-v-vision-optics-development-underground-scout-robot.qv6k2ory>
10. Walker, T. (2020, August 25). *What is a linear voltage regulator?* All About Circuits. Retrieved July 5, 2025, from
<https://www.allaboutcircuits.com/technical-articles/what-is-a-linear-voltage-regulator/>
11. IBM. (n.d.). *AI in manufacturing*. IBM. Retrieved July 5, 2025, from
<https://www.ibm.com/think/topics/ai-in-manufacturing>
12. Arrow Electronics. (n.d.). *Flyback protection diodes*. Arrow.com. Retrieved July 5, 2025, from
<https://www.arrow.com/en/research-and-events/articles/flyback-protection-diodes>
13. Lunden, I. (2025, June 30). *ChatGPT: Everything to know about the AI chatbot*. TechCrunch. Retrieved July 5, 2025, from

- <https://techcrunch.com/2025/06/30/chatgpt-everything-to-know-about-the-ai-chatbot/>
14. witnessmenow. (n.d.). *ESP32-cheap-yellow-display*. GitHub. Retrieved July 5, 2025, from <https://github.com/witnessmenow/ESP32-Cheap-Yellow-Display>
 15. Esp32io. (n.d.-a). *ESP32 button tutorial*. Retrieved July 5, 2025, from <https://esp32io.com/tutorials/esp32-button>
 16. Esp32io. (n.d.-b). *ESP32 piezo buzzer tutorial*. Retrieved July 5, 2025, from <https://esp32io.com/tutorials/esp32-piezo-buzzer>
 17. Analytics India Magazine. (n.d.). MIT study warns of cognitive decline with LLM use. *Analytics India Magazine*. Retrieved July 5, 2025, from <https://analyticsindiamag.com/ai-news-updates/mit-study-warns-of-cognitive-decline-with-llm-use/>
 18. USB-Implementers Forum. (2013, December 30). USB-IF developers documents archive. Retrieved July 5, 2025, from https://web.archive.org/web/20131230035108/http://www.usb.org/developers/docs/documents_archive/
 19. U.S. Department of Justice. (2010). *ADA standards for accessible design*. <https://www.ada.gov/law-and-regs/design-standards/>
 20. Raspberry Pi Foundation. (2025, July 1). *Pico 2 and Pico W Documentation*. Raspberry Pi. Retrieved from <https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>
 21. Espressif Systems. (n.d.). *ESP32-WROOM-32 Datasheet*. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf
 22. GerTech. (n.d.). *ESP32 Specifications*. Retrieved from https://gertech.se/gertech/files/ESP32_Specification.pdf
 23. Blues Wireless. (n.d.). *Getting started with PIO on Raspberry Pi Pico*. Retrieved from <https://blues.com/blog/raspberry-pi-pico-pio/>
 24. U.S. Department of Justice. (2010). *2010 ADA Standards for Accessible Design*. Retrieved from <https://www.ada.gov/law-and-regs/design-standards/2010-stds/#701-general>
 25. U.S. Department of Justice. (n.d.). *ADA Effective Communication*. Retrieved from <https://www.ada.gov/resources/effective-communication/>
 26. Qirky. (n.d.). *PyKinectTk [GitHub repository]*. GitHub. Retrieved from <https://github.com/Qirky/PyKinectTk>
 27. Swapnil0908. (n.d.). *Kinect Skeleton Tracking [GitHub repository]*. GitHub. Retrieved from <https://github.com/swapnil0908/Kinect-Skeleton-Tracking>
 28. Kinect. (n.d.). *PyKinect2 [GitHub repository]*. GitHub. Retrieved from <https://github.com/Kinect/PyKinect2>

29. GeeksforGeeks. (n.d.). *Introduction to GitHub*. Retrieved from <https://www.geeksforgeeks.org/git/introduction-to-github/>
30. GeeksforGeeks. (n.d.). *Introduction to GitLab*. Retrieved from <https://www.geeksforgeeks.org/devops/gitlab/>
31. GeeksforGeeks. (n.d.). *Introduction to GitLab*. Retrieved from <https://www.geeksforgeeks.org/devops/gitlab/>
32. GeeksforGeeks. (n.d.). *Introduction to Bitbucket*. Retrieved from <https://www.geeksforgeeks.org/git/introduction-to-bitbucket/>
33. Xiang, L., et al. (2021, August 6). *Openkinect/libfreenect2: v0.2.1* [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.5167182>
34. Jetson Nano
35. NVIDIA. (n.d.). *Jetson Nano Developer Kit*. Retrieved from <https://developer.nvidia.com/buy-jetson>
36. Anderson, J. (2020, August 14). *What's the average walking speed?*. Healthline. Retrieved from <https://www.healthline.com/health/exercise-fitness/average-walking-speed#takeaway>
37. Leo_FC. (n.d.). *Make Your Own LED Battery Level Indicator*. Instructables. Retrieved from <https://www.instructables.com/Make-Your-Own-LED-Battery-Level-Indicator/>
38. DFRobot. (n.d.). *LattePanda Delta 864*. Retrieved from <https://www.dfrobot.com/product-2594.html>
39. Abeykoon, C., & Jayawardena, C. (2024). *Performance analysis of YOLOv3, YOLOv4, and MobileNet SSD for real-time object detection*. ResearchGate. Retrieved from https://www.researchgate.net/publication/381851712_Performance_Analysis_of_YOLOv3_YOLOv4_and_MobileNet_SSD_for_Real_Time_Object_Detection
40. Digi International. (n.d.). *Zigbee Wireless Standard Overview*. Retrieved from <https://www.digi.com/solutions/by-technology/zigbee-wireless-standard>
41. Docker. (n.d.). *What is a Container?*. Retrieved from <https://www.docker.com/resources/what-container/>
42. Red Hat. (n.d.). *What is Podman?*. Retrieved from <https://www.redhat.com/en/topics/containers/what-is-podman>

Appendix

Appendix A: Chat GPT Logs

Chat Log A:

Yes, **you will absolutely need voltage regulation** for your setup. Here's why and how:

Your Current Setup:

- **Power Source:** 24V (from two 12V Sealed Lead Acid batteries in series).
- **Load Devices:** Kinect V2 (12V input), and other components with varying voltage requirements.

Why You Need Voltage Regulators:

1. **Voltage Mismatch**
 - Your power supply is 24V.
 - Kinect V2 **requires 12V**, and other components may require **5V**, **3.3V**, etc.
 - **Directly connecting 24V to these devices will likely damage them.**
2. **Battery Voltage Fluctuation**
 - LA batteries don't supply a constant 12V; a full 12V battery can be ~13.2V and discharge down to ~10.5V.
 - So, a 24V system might swing from ~26.4V to ~21V.
3. **Component Safety**
 - Sensitive electronics need stable, well-regulated voltage to function properly and avoid overheating or burning out.

What You Need

1. Step-Down (Buck) Voltage Regulators

Use DC-DC buck converters to convert 24V down to:

- **12V** for Kinect V2
- **5V / 3.3V / etc.** for microcontrollers, sensors, etc.

Look for:

- Efficient switching regulators (not linear regulators — they waste too much power and get hot).
- Adequate current rating (Kinect V2 alone can draw **2–2.5A at 12V**).

Component	Voltage Needed	Regulator Example
Kinect V2	12V	24→12V buck converter (at least 3A rated)

Microcontroller	5V or 3.3V	24→5V or 24→3.3V buck converter
Sensors/Peripherals	Varies	Use dedicated buck converter

Recommendations

- **Adjustable Buck Converters:** e.g., LM2596 modules for low current; XL4015 for higher currents.
- **Fuse or overcurrent protection** between battery and regulators.
- **Heat sinks** or active cooling for high-power regulators.
- **Check Kinect V2 specs** — it actually needs a **separate power adapter**, and may not run directly off 12V without a modified setup.

Would you like help picking specific regulators for your system, or a wiring diagram suggestion?

Chat Log B:

For a wheelchair, especially one that will see regular use and must prioritize reliability, safety, and range, here's a direct recommendation based on real-world use cases:

Best Choice: Lithium-Ion (Li-ion)

Why:

- **Much lighter:** Reduces the total weight of the wheelchair by 20–40%, which means:
 - Easier maneuverability
 - Better range per charge
 - Less strain on the motors
- **Longer lifespan:** Typically 2,000–3,000 cycles (vs. 300–500 for SLA). That means:
 - Fewer replacements over the life of the chair
 - Better long-term cost savings despite higher upfront cost
- **Faster charging:** Can charge in 2–4 hours, vs. 8–12 hours for SLA.
- **Higher usable capacity:** Li-ion batteries can discharge to 80–90% without damage, unlike SLA, which you should only discharge to ~50%.

Downsides:

- **More expensive upfront**
- **Requires proper Battery Management System (BMS)** to prevent overcharge/discharge or overheating
- **Special charger needed**

Acceptable Alternative: Gel Cell (a type of SLA)

Why:

- **Safer than standard SLA:** More resistant to vibration and leaks (ideal for wheelchairs)
- **Maintenance-free**

- Can tolerate deeper discharges than traditional SLA (~50–60% DoD safely)
- **Cost-effective** for short-to-medium term use

Downsides:

- Still **heavy**
- Shorter lifespan (**300–500 cycles**)
- Slower charging (6–10 hours)

Not Recommended: Standard Sealed Lead Acid (SLA)

Why:

- Cheapest, but:
 - **Heaviest**
 - Shortest lifespan
 - Not good for regular deep discharging (which wheelchairs do)

- Often leads to early battery failure and reduced range over time

Real-World Wheelchair Battery Summary:

Battery Type	Weight	Lifespan (Cycles)	Depth of Discharge	Cost	Ideal For
Lithium-Ion	Lightest	2000–3000	80–90%	High	Daily/long-term use, modern electric wheelchairs
Gel Cell	Heavy	400–600	~60%	Moderate	Moderate use, budget-constrained users
SLA	Heaviest	300–500	~50%	Cheap	Short-term use, tight budgets

What I Would Use for a Wheelchair:

Lithium-Ion, no question—**especially** if the wheelchair is used daily or for mobility-critical needs. The reduced weight, longer lifespan, and higher efficiency make it a better investment despite the higher initial cost.

Want help identifying specific Li-ion battery packs that are wheelchair-compatible?

Chat Log C:

Current Rules and Codes for Designing or Modifying Electric Wheelchairs

Location: United States – State of Florida

1. Federal Standards for Electric Wheelchairs

a. Safety & Classification

- Electric wheelchairs are regulated as **Class II medical devices** by the **FDA**.
- They must comply with general medical device safety standards and undergo FDA registration for legal sale.

b. Power Mobility Devices & ADA

- The **Americans with Disabilities Act (ADA)** recognizes power wheelchairs and **Other Power-Driven Mobility Devices (OPDMDs)** as valid mobility aids.
- ADA guidelines require public and commercial facilities to accommodate these devices unless legitimate safety risks are involved.

c. Durability & Performance Standards

- **ISO 7176 Series:** International standards that define mechanical strength, braking, stability, and fatigue resistance testing of power wheelchairs.
- **ANSI/RESNA Standards:** U.S.-specific standards such as:
 - **WC-01 / WC-19:** Securement systems and transportation safety.
 - **RESNA WC-3:** Testing procedures for battery and electronics durability.

2. Florida State Regulations and Codes

a. Ramp & Access Design

- Ramps must meet **ADA slope requirements:** 1:12 rise-to-run ratio, with landings at every 30 feet of length.
- Minimum clear width: **36 inches**. Handrails required if ramp rise exceeds 6 inches.
- Doorway clearance minimum: **32 inches** wide at a 90° angle.

b. Electrical Systems & Modifications

- If modifications affect battery voltage, wiring, or safety systems, they must maintain **UL** and **manufacturer's specifications**.
- Any rewiring should preserve **IP ratings** for resistance to water/dust, especially in outdoor wheelchairs.

c. Florida Motor Vehicle Code

- Under **Florida Statute §316.2068**, electric mobility devices may be operated:
 - On sidewalks, paths, and certain roads (≤ 25 mph speed zones).
 - No license or registration required, but users under 16 must wear a helmet.
 - Devices must yield to pedestrians and follow local municipality rules.

d. Parking and Public Access

- Designated wheelchair-accessible parking must follow:
 - Minimum **8-foot-wide** access aisle,
 - Proper **vertical signage** and **ground markings**.
- Shared-use paths and sidewalks must allow safe passage of wheelchairs, including curb cuts and tactile indicators.

3. Key Considerations for Modifying Electric Wheelchairs

a. Battery Regulations

- Batteries must meet **UN 38.3** transport safety standards if shipped, especially lithium-ion types.
- Charging systems should be **smart-regulated** to avoid overvoltage or overheating.

b. Motor and Controller Changes

- Any controller or motor upgrades must maintain safe speed limits (typically ≤6 mph in indoor models).
- Reprogramming or replacing the controller can void the warranty unless approved by the manufacturer.

c. ADA Compliance

- Modifications must not remove or reduce accessibility features such as seat elevation, tilt, or securement compatibility.
- Must maintain **maneuverability** in confined public spaces (e.g., turning radius ≤5 feet).

4. Other Considerations for Wheelchair Modifications

a. Insurance and Reimbursement

- Medicare and Medicaid may deny claims if the device is modified outside approved channels.
- It is recommended to check **HCPCS codes** and ensure any new component is pre-approved.

b. Warranty Implications

- Manufacturer warranties may be voided by user-performed electrical or mechanical modifications.
- Use of OEM (original equipment manufacturer) parts is often required for service support.

c. Safety and Recall Risk

- Aftermarket components (e.g., lights, horns, mounts) should be tested for **electromagnetic compatibility (EMC)**.
- Modifications must not interfere with braking systems or cause power surges.

Summary Table of Relevant Codes and Standards

Standard/Rule	Description
FDA Class II Regulation	Classifies power wheelchairs as medical devices; requires registration & controls.
ADA Title II & III	Ensures public access and protection of wheelchair users.
ISO 7176 Series	International standards for mechanical durability, strength, and testing.

ANSI/RESNA WC-19	Transportation safety standards for securing wheelchairs in vehicles.
Florida Statute 316.2068	Allows electric mobility devices on sidewalks and roads under 25 mph.
ADA Ramp Requirements	Requires 1:12 slope ratio, handrails, and landings for ramps.
UL/IEC Electrical Safety Codes	Ensures batteries, controllers, and motors operate safely and reliably.
UN 38.3 Battery Transport Standard	Required for safe lithium-ion battery shipment and handling.
Medicare HCPCS Code Compliance	Needed for reimbursement approval for new wheelchair components.